

Vorlesung (WS 2016/17) *Softwarekonstruktion*

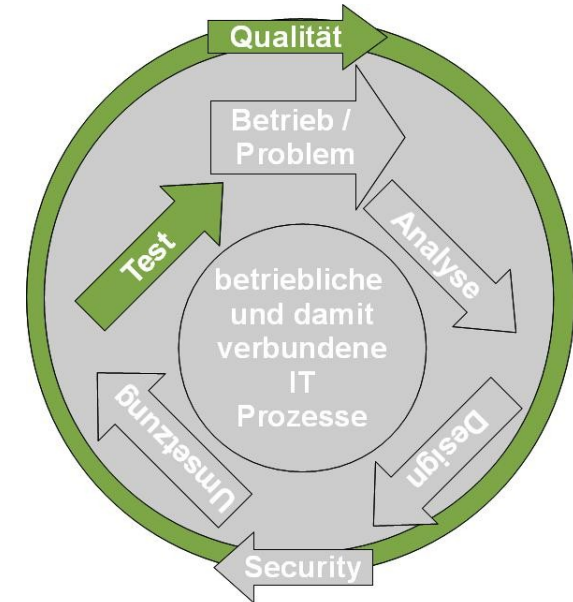
Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

2.4: White-Box-Test

v. 19.12.2016 und v. 9.1.2017

- Modellgetriebene SW-Entwicklung
- Qualitätsmanagement
- **Testen**
 - Grundlagen Softwareverifikation
 - Softwariemetriken
 - Black-Box-Test
 - **White-Box-Test**
 - Testen im Softwarelebenszyklus



[Basierend auf dem Foliensatz „Basiswissen Softwaretest - Certified Tester“ des „German Testing Board“, 2011]

Literatur (s. Webseite):

- Andreas Spillner, Tilo Linz: Basiswissen Softwaretest. **Kapitel 5.**
- Eike Riedemann: Testmethoden für sequentielle und nebenläufige Software-Systeme. **Kapitel 7,8,9.**

- Verständnis für die Motivation für das Testen von Software entwickeln
- Übersicht über Testmethoden (Black-Box / White-Box)
- Fertigkeit entwickeln die korrekte Testmethode bedarfsabhängig auszuwählen

- **Voheriger Abschnitt:** Dynamischer Test ohne Kenntnis der Programmlogik
- **Dieser Abschnitt:** Analyse interner Struktur des Testobjekts mit Hilfe:
 - Kontrollflussbasierter Test
 - Datenflussbasierter Test

Was Sie wiederholen sollten!

Softwarekonzepte
WS 2016/17



Wiederholung
SWT

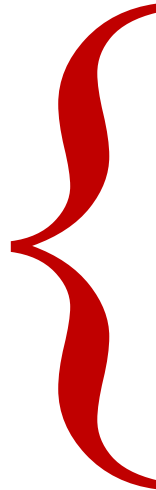
Was Sie für diesen Abschnitt von der Vorlesung SWT wiederholen sollten:

Testen mit Hilfe der

- **Anweisungsüberdeckung**
- **Zweigüberdeckung**
- **Pfadüberdeckung**
- **Mehrfachbedingungsüberdeckung**

Diese Themen werden wir noch einmal kurz behandeln, es ist aber sinnvoll sich die Methoden vorher noch einmal ins Gedächtnis zu rufen.

2.4 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Test der Bedingungen

Datenflussbasierter Test

Statische Analyse

White-Box-Test: Dynamisches Testverfahren, basiert auf Analyse interner Struktur der Testobjekts.

»**Fehleraufdeckende**« **Stichproben** möglicher Programmabläufe und Datenverwendungen suchen.

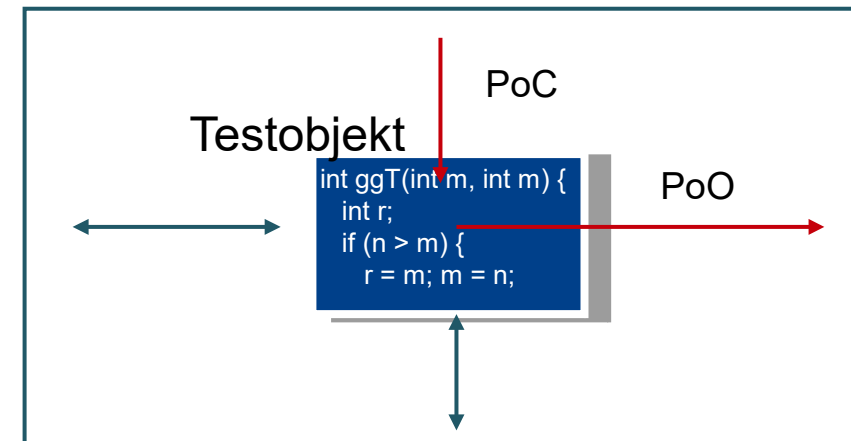
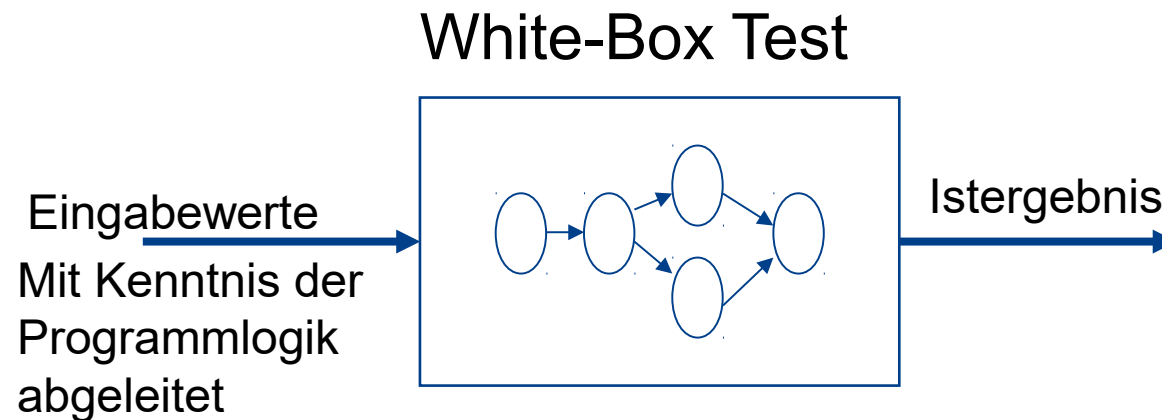
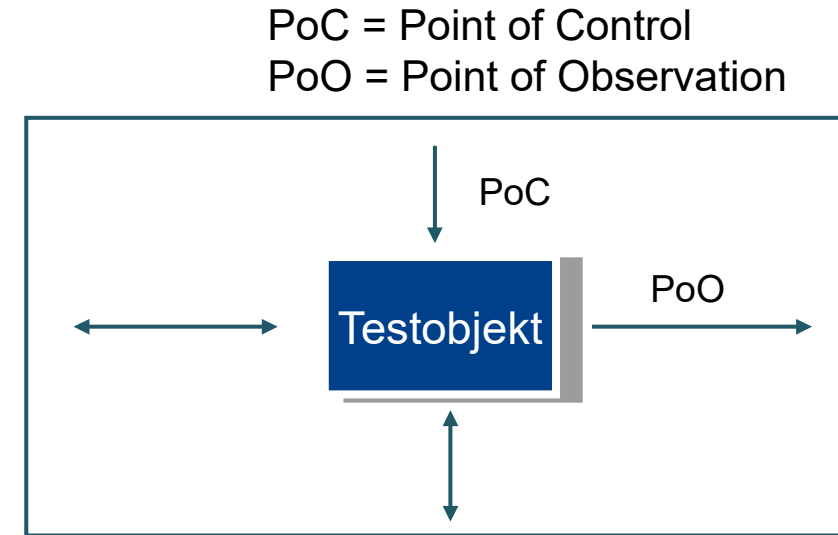
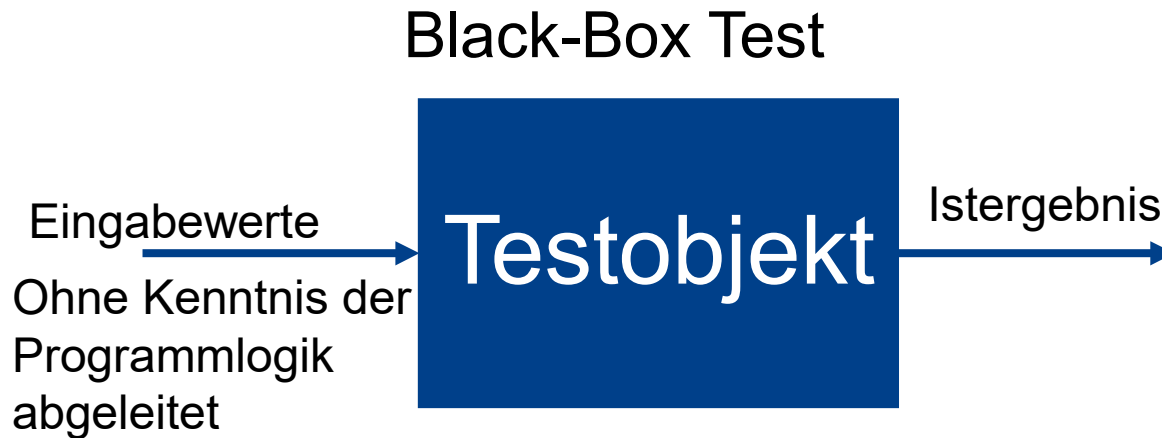
Zur

- Herleitung der Testfälle
- **Bestimmung der Vollständigkeit der Prüfung** (Überdeckungsgrad)

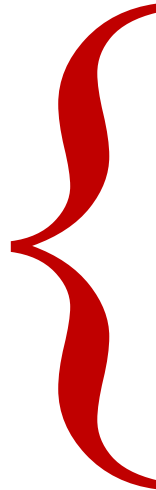
wird Information über innere Struktur des Testobjekts herangezogen.

→ Strukturorientierte (strukturelle) Testentwurfsverfahren.

Black-Box Test vs. White-Box Test



2.4 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Test der Bedingungen

Datenflussbasierter Test

Statische Analyse

Kontrollflussbezogenes Testen:

- **Dynamisches** Verfahren: Basiert auf Ausführung des Programms.
- **White-Box-Verfahren:** Nutzt Kenntnis der Programmstruktur aus.

Analysemittel:

- **Kontrollflussgraph:** Verdeutlicht Kontrollfluss im Programm.

Idee:

- **Auswahl der Testdaten:** Viele Durchläufe durch Kontrollfluss des Programms testen; dafür wenig Testfälle gebrauchen.

Varianten, differenzieren nach:

- **Art** der verwendeten **Kontrollflusswege** oder **-wegstücke**.
- Art und Weise der Überdeckung.
- angestrebtem Überdeckungsgrad

Kontrollflussgraph eines Programms P: **Gerichteter Graph**

$$G = (N, E)$$

mit 2 ausgezeichneten Knoten *nstart*, *nfinal*

- Knoten stellt **Anweisungen** / sequenzielle Anweisungsfolgen dar.
- Kante aus Menge der Kanten $E \subseteq N \times N$ beschreibt möglichen **Kontrollfluss zwischen zwei Anweisungen**.
- Kante aus E auch **Zweig** genannt.
- Beide ausgezeichneten Knoten stellen **Anfangs- und Endeanweisung** eines Programms dar.

Block: Nichtleere Folge von Knoten.

- Nur durch ersten Knoten betretbar,
- Vom ersten Knoten ausgehend in vorgegebener Reihenfolge genau einmal deterministisch durchlaufbar.
- Maximal bezüglich ersten beiden Eigenschaften.

Pfad / vollständiger Weg:

- Folge von Knoten und Kanten, die mit Startknoten beginnt und Endknoten endet.

Wege (T,P):

- Menge vollständiger, endlicher Wege w des Kontrollflussgraphen, für die Testdatum t aus Menge der Testdaten T existiert, das Weg w ausführt.

- **Entscheidungsknoten:** Knoten mit mindestens 2 Nachfolgeknoten.
- **Entscheidungskanten:** Kanten, die ihren Ursprung in einem Entscheidungsknoten haben.
- **Zyklus:** Weg im Kontrollflussgraphen mit mindestens zwei Knoten, der an demselben Knoten beginnt und endet.
- **Einfacher Zyklus:** Zyklus, bei dem alle Knoten (außer Anfangs- und Endknoten) verschieden sind.

Beispielprogramm: Funktion **ggT** ()

Bestimmung des **größten gemeinsamen Teilers (ggT)** zweier ganzer Zahlen $m, n > 0$:

$\text{ggT}(4,8)=4$; $\text{ggT}(5,8)=1$; $\text{ggT}(15,35)=5$

Spezifikation in Java (JML):

```
public int ggt(int m, int n) {  
  // pre: m > 0 and n > 0  
  // post: return > 0 and  
  // m@pre.mod(return) = 0 and  
  // n@pre.mod(return) = 0 and  
  // forall(i : int | i > return implies  
  // (m@pre.mod(i) > 0 or n@pre.mod(i) > 0)  
  ... }
```

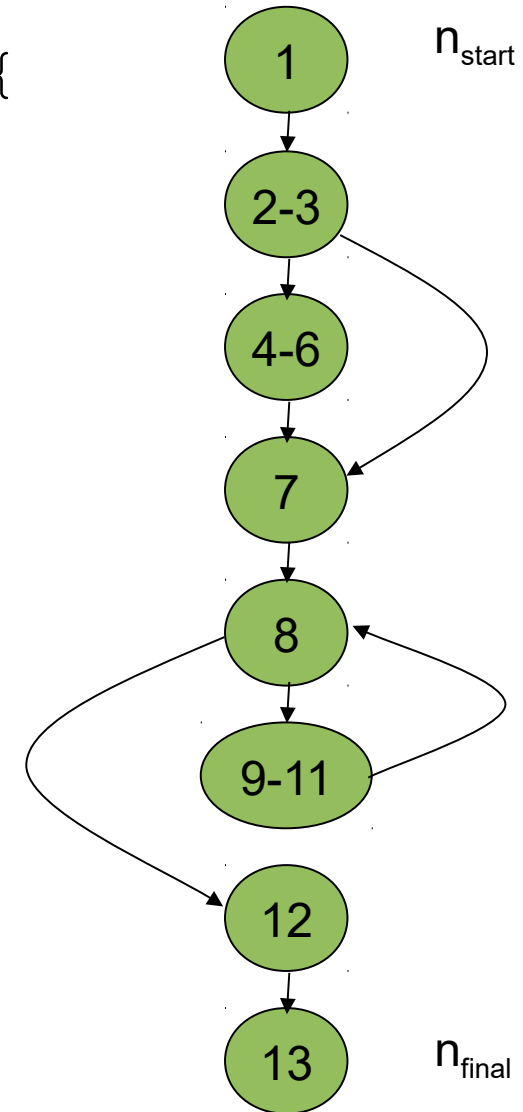
Beispielprogramm: Kontrollflussgraph von ggt ()

```
1. public int ggt (int m, int n) {  
2.   int r;  
3.   if (n > m) {  
4.       r = m;  
5.       m = n;  
6.       n = r;  
7.   }  
8.   while (r != 0) {  
9.       m = n;  
10.      n = r;  
11.      r = m % n;  
12.  }  
12. return n;  
13. }
```

Block

Block

Block



Zunächst: Bestimmung der Pfade durch Kontrollflussgraphen, die durch Testfälle »zur Ausführung gebracht werden sollen«.

Frage: Mit welchen Eingaben werden diese Pfade erzwungen ?

Idee:

- Bedingungen **kontrollflussbestimmender Anweisungen** betrachten.
- Damit **Aussagen über Programmvariablen** »berechnen«.

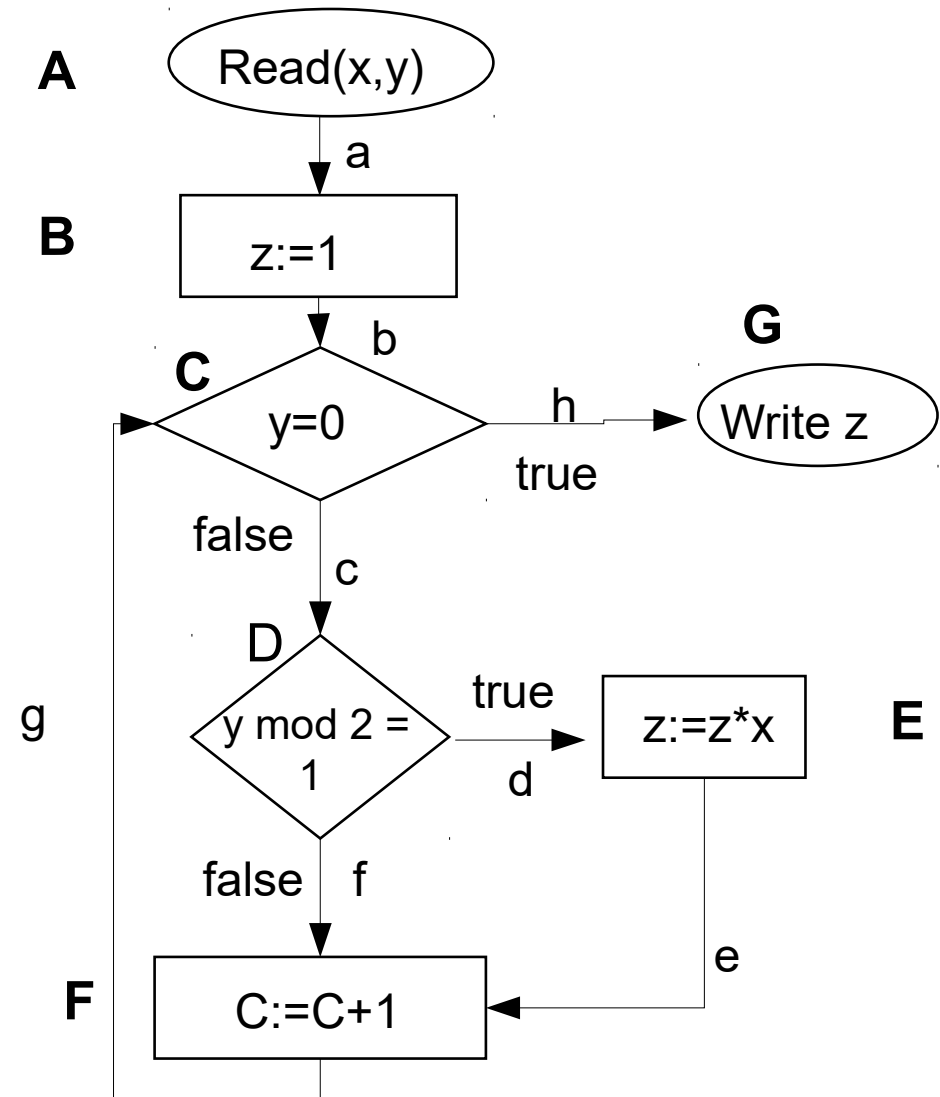
Achtung: Nicht alle im Graphen vorhandenen Pfade (Zweige, Wegstücke, ...) können für das gegebene Programm tatsächlich ausgeführt werden (z.B. nicht erreichbare Programmstücke; nicht erreichbare Anzahl von Schleifendurchläufen), d.h. es kann Pfade ohne zugehörigen Testfall geben.

Entscheidungs-Entscheidungsweg

Entscheidungs-Entscheidungsweg:

- Wegstück, welches bei **Entscheidungsknoten** oder **Anfangsknoten** beginnt
- Und alle folgenden Knoten und Kanten bis zum nächsten **Entscheidungsknoten** bzw. bis zum **Endknoten** des Kontrollflussgraphen (einschließlich) enthält.

Was sind hier die Entscheidungs-Entscheidungswege ?

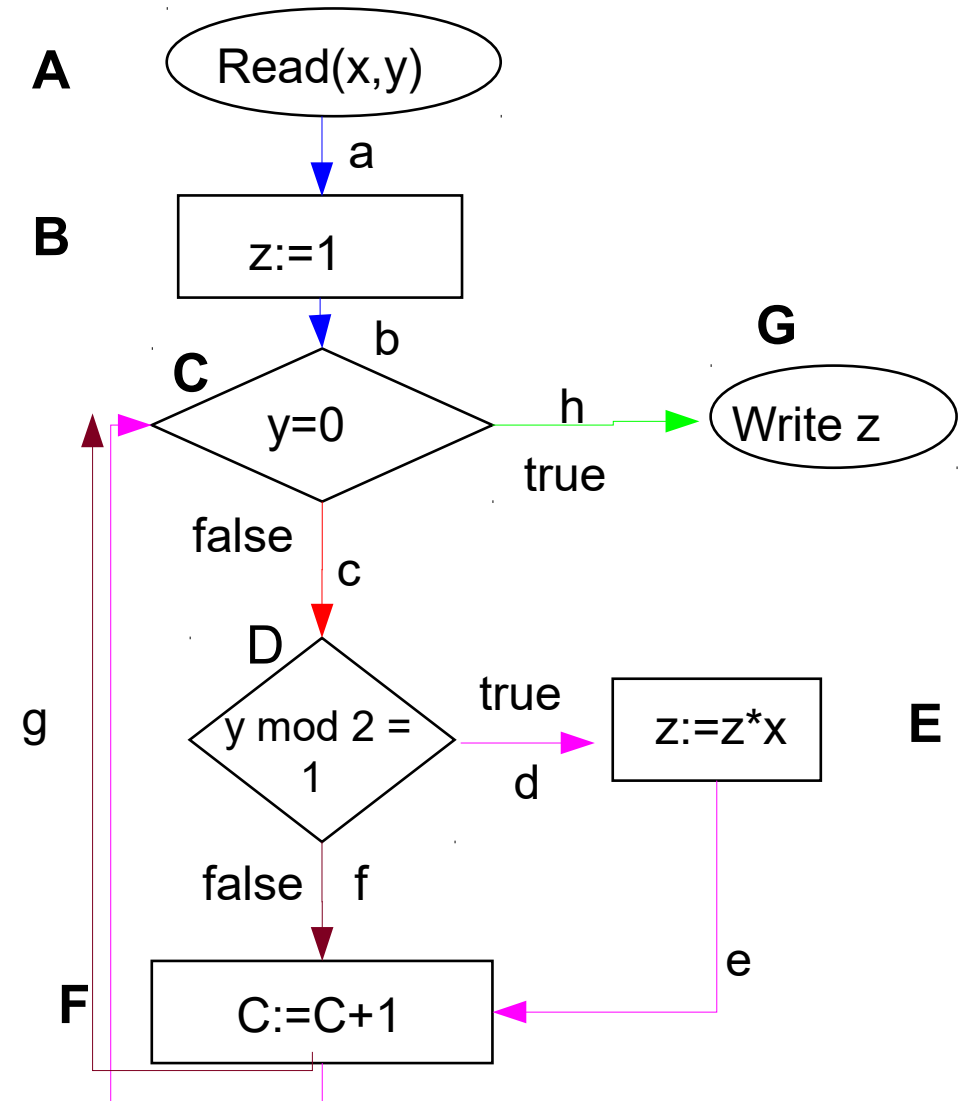


Entscheidungs-Entscheidungsweg

Entscheidungs-Entscheidungsweg:

- Wegstück, welches bei **Entscheidungsknoten** oder **Anfangsknoten** beginnt
- Und alle folgenden Knoten und Kanten bis zum nächsten **Entscheidungsknoten** bzw. bis zum **Endknoten** des Kontrollflussgraphen (einschließlich) enthält.

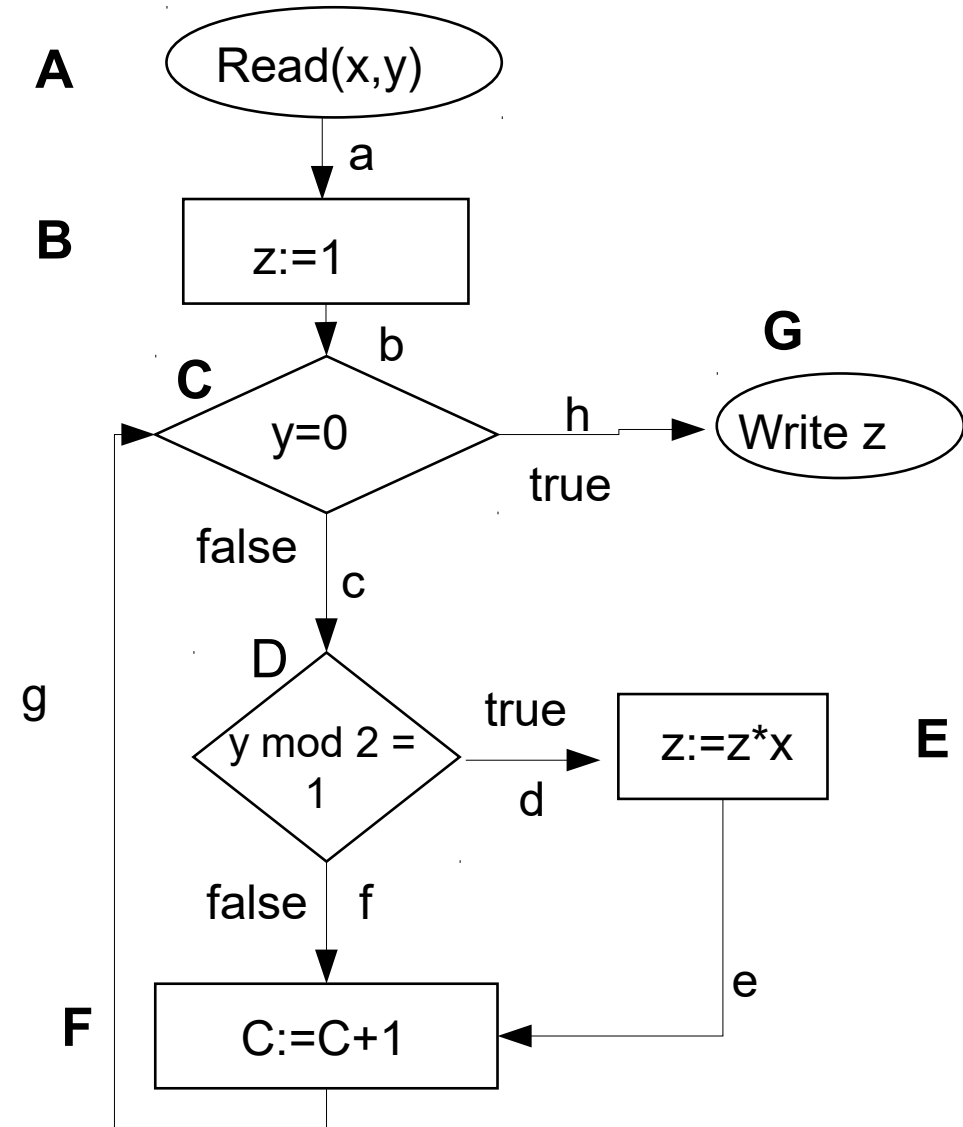
Hier: $\{A, B, C\}$; $\{C, G\}$; $\{C, D\}$; $\{D, E, F, C\}$;
 $\{D, F, C\}$



Segment: Wegstück mit Eigenschaften:

- **Erster Knoten** des Wegstücks: Anfangsknoten des Kontrollflussgraphen oder Entscheidungsknoten oder Vereinigungsknoten (mehrere Inputs).
- **Letzter Knoten** des Wegstücks: Endknoten des Kontrollflussgraphen oder Entscheidungsknoten oder Vereinigungsknoten.
- Alle **andere Knoten** haben nur Eingangs- und Ausgangskante.

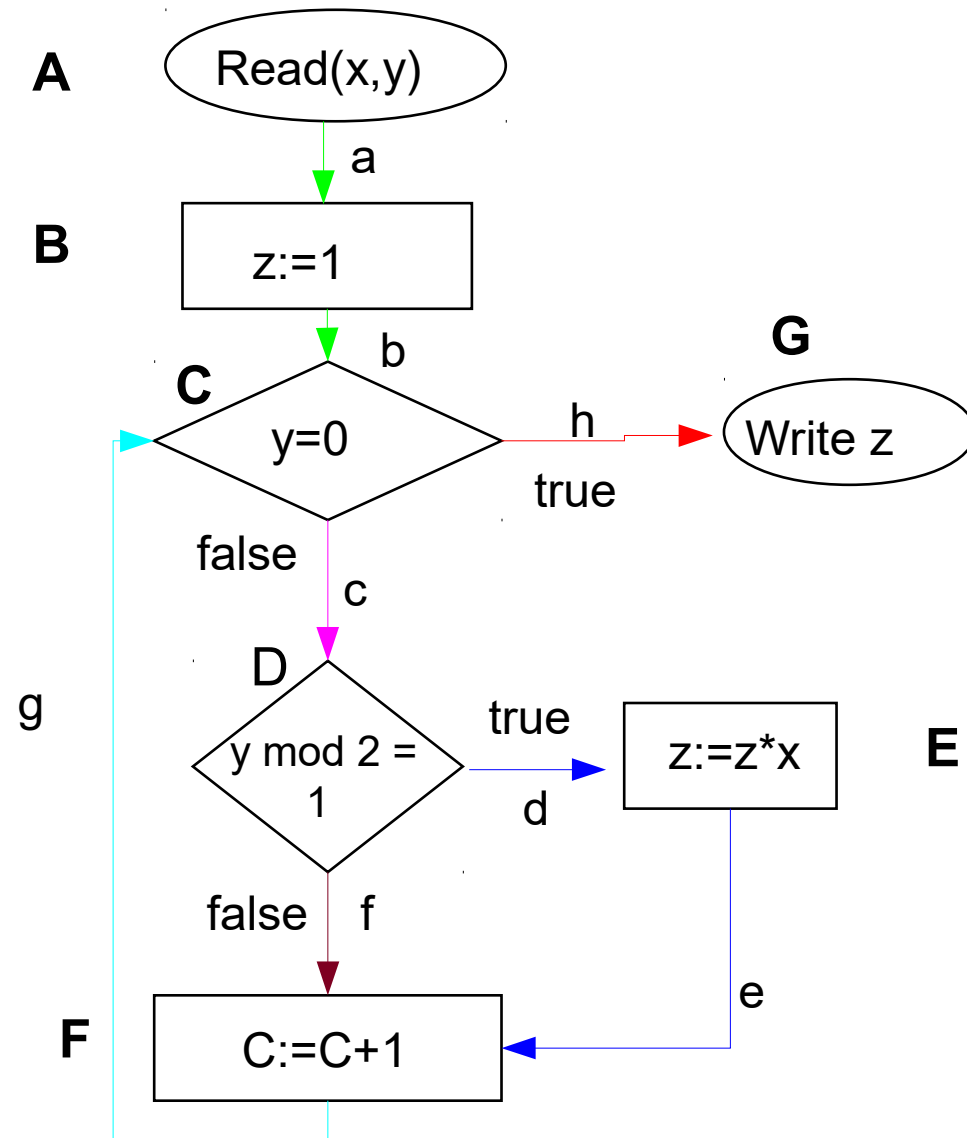
Was sind hier die Segmente ?



Segment: Wegstück mit Eigenschaften:

- **Erster Knoten** des Wegstücks: Anfangsknoten des Kontrollflussgraphen oder Entscheidungsknoten oder Vereinigungsknoten (mehrere Inputs).
- **Letzter Knoten** des Wegstücks: Endknoten des Kontrollflussgraphen oder Entscheidungsknoten oder Vereinigungsknoten.
- Alle **andere Knoten** haben nur Eingangs- und Ausgangskante.

Hier: {A,B,C}; {C,G}; {C,D}; {D,E,F}; {D,F}; {F,C}



Durch kontrollflussbezogenes Verfahren aufdeckbare Fehler:

Berechnungsfehler:

- Richtiger Kontrollflussweg im Programm ausgeführt, aber mindestens **ein berechneter Variablenwert falsch**.

Bereichsfehler:

- **Falscher Kontrollflussweg** im Programm ausgeführt.
(→ Eingabebereich des vorliegenden Kontrollflussweges stimmt nicht mit Eingabebereich im korrekten Programm überein.)

Unterbereichsfehler:

- Spezielle Bereichsfehler.
- **„Zuviel / zuwenig“ Kontrollfluss:**
→ Abfrage fehlt oder es gibt zusätzliche, falsche Abfrage.

- **Anweisungsüberdeckung**
(*statement coverage; C_0 -Überdeckung; alle Knoten*).
- **Entscheidungs-/Zweigüberdeckung**
(*decision coverage; C_1 -Überdeckung, alle Zweige*).
- **Grenze-Inneres-Test**
(*boundary interior coverage*).
- **Pfadüberdeckung**
(*path coverage; C_∞ -Überdeckung, alle Pfade*).

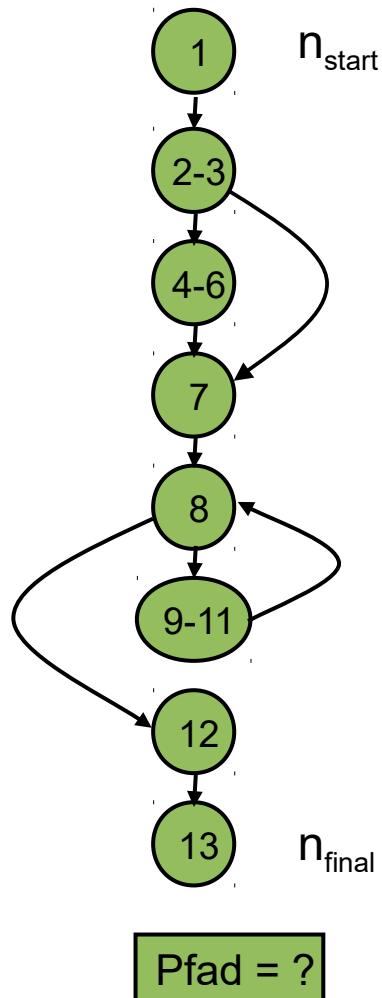


Anweisungsüberdeckung (C_0 -Test):

- Testdatenmenge T erfüllt C_0 -Überdeckung für Programm P g.d.w. es für **jede Anweisung** A des Programms P mind. ein **Testdatum** t aus T gibt, das Anweisung A **ausführt**.
- Anweisung A unter T ausgeführt g.d.w. der zur **Anweisung** A gehörende **Knoten** k in mind. einem **Weg** in $Wege(T)$ vorkommt.
- **Testwirksamkeitsmaß TWM_0** : Anweisungsüberdeckungsgrad (Verhältnis besuchter Knoten zur Gesamtzahl von Knoten):

$$TWM_0 =_{\text{def}} \frac{\text{Zahl der unter } T \text{ überdeckten Anweisungen}}{\text{Zahl aller Anweisungen}}$$

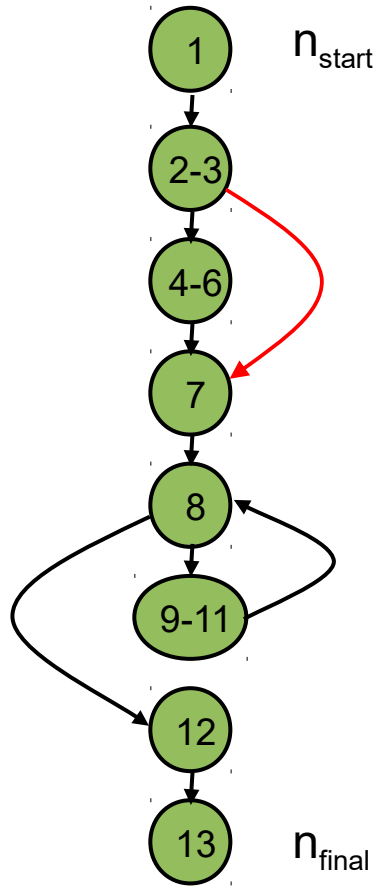
Beispiel: Anweisungsüberdeckung für `ggt()`



```
1. public int ggt(int m, int n) {  
2.     int r;  
3.     if (n > m) {  
4.         r = m;  
5.         m = n;  
6.         n = r;  
7.     }  
8.     r = m % n;  
9.     while (r != 0) {  
10.        m = n;  
11.        n = r;  
12.        r = m % n;  
13.    }  
14. return n;  
15. }
```

Beispiel für Pfad mit Anweisungsüberdeckung ? Zugehörige Testdaten ?

Beispiel: Anweisungsüberdeckung für `ggt()`: Pfad

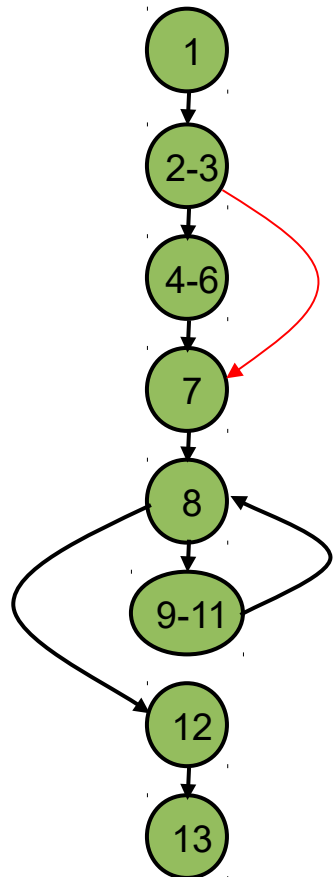


```
1. public int ggt(int m, int n) {  
2.     int r;  
3.     if (n > m) {  
4.         r = m;  
5.         m = n;  
6.         n = r;  
7.     }  
8.     r = m % n;  
9.     while (r != 0) {  
10.        m = n;  
11.        n = r;  
12.        r = m % n;  
13.    }  
14. return n;  
15. }
```

Pfad = (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13)

(Roter Zweig wird nicht ausgeführt.)

Beispiel: Anweisungsüberdeckung für `ggt()` : Testdaten



Pfad: (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13)

Logischer Testfall: $\{ n > m \wedge n \bmod m \neq 0 \wedge n \bmod (n \bmod m) = 0 ; \text{ggt}(m,n) \}$

Konkreter Beispiel-Testfall:
 $\{ m = 1, n = 2; \text{ggt}(m,n)=2 \}$

Oder:
 $\{ m = 4, n = 6; \text{ggt}(m,n)=2 \}$

m	n	r
4	6	2

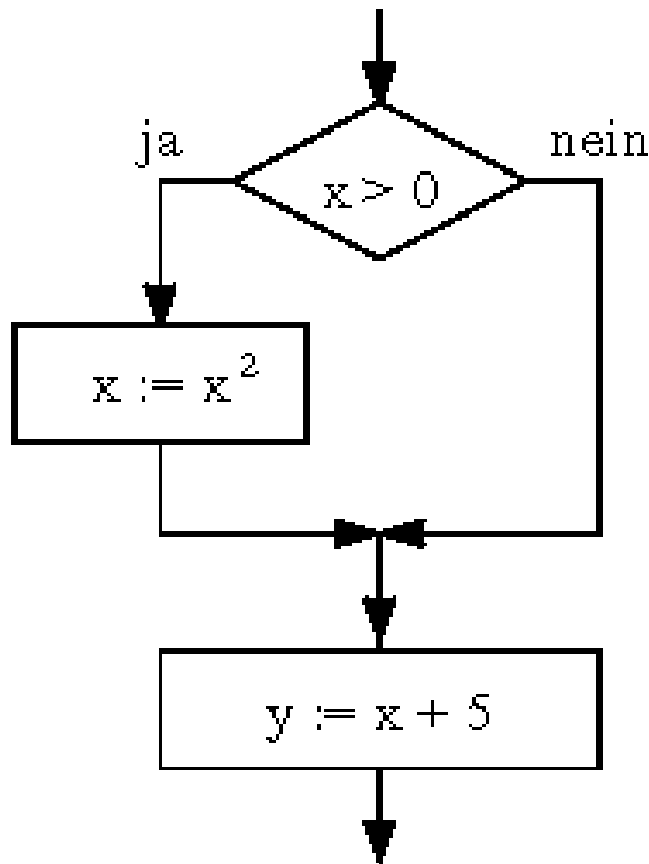
```
1. public int ggt(int m, int n) {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
7.     }
8.     r = m % n;
9.     while (r != 0) {
10.        m = n;
11.        n = r;
12.        r = m % n;
13.    }
14.    return n;
15. }
```

100%ige Anweisungsüberdeckung nicht immer erreichbar.

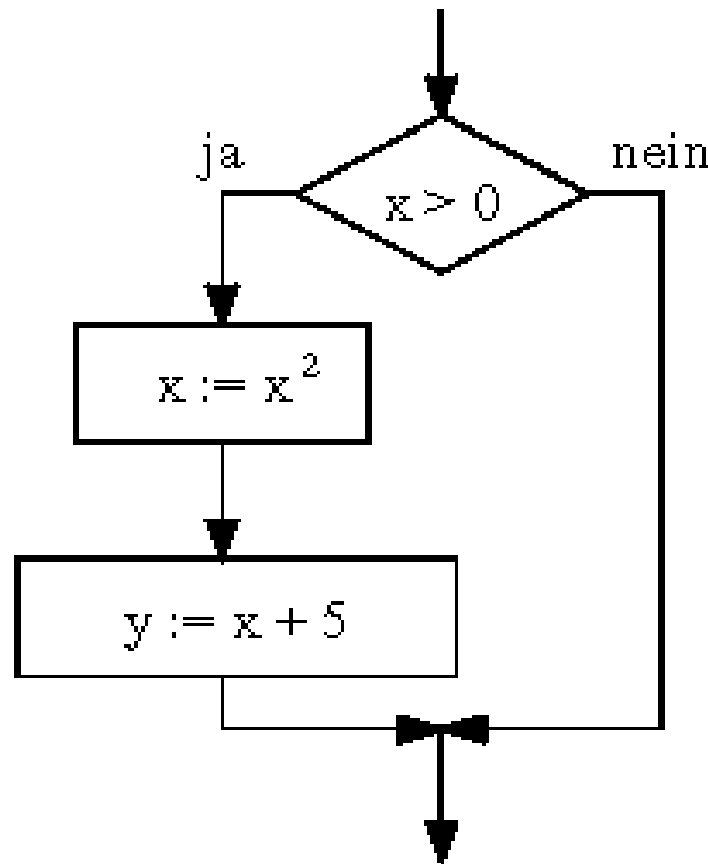
- Z.B.: Ausnahmebedingungen kommen im Programm vor, die während Testphase mit erheblichem Aufwand oder gar nicht herzustellen sind.
- Kann auf **nicht erreichbare Anweisungen** („dead code“) hindeuten (ggf. statische Analyse durchführen).

Anweisungsüberdeckung: Aussagekraft (Beispiel)

Richtiges Programm



Falsches Programm



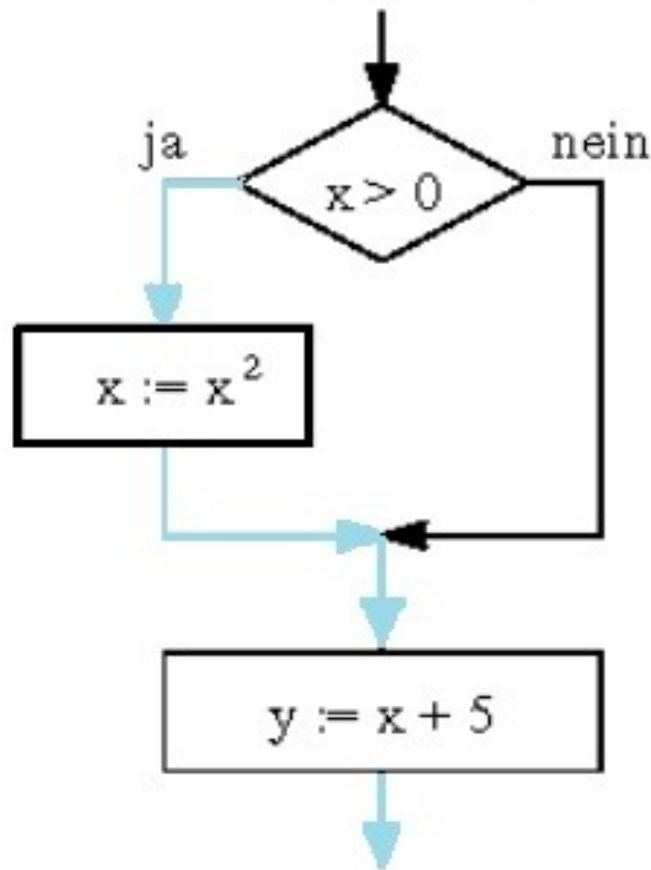
Testdatum:

Wie könnte ein einzelner **Testfall** aussehen, der **C₀-Überdeckung** erfüllt ?

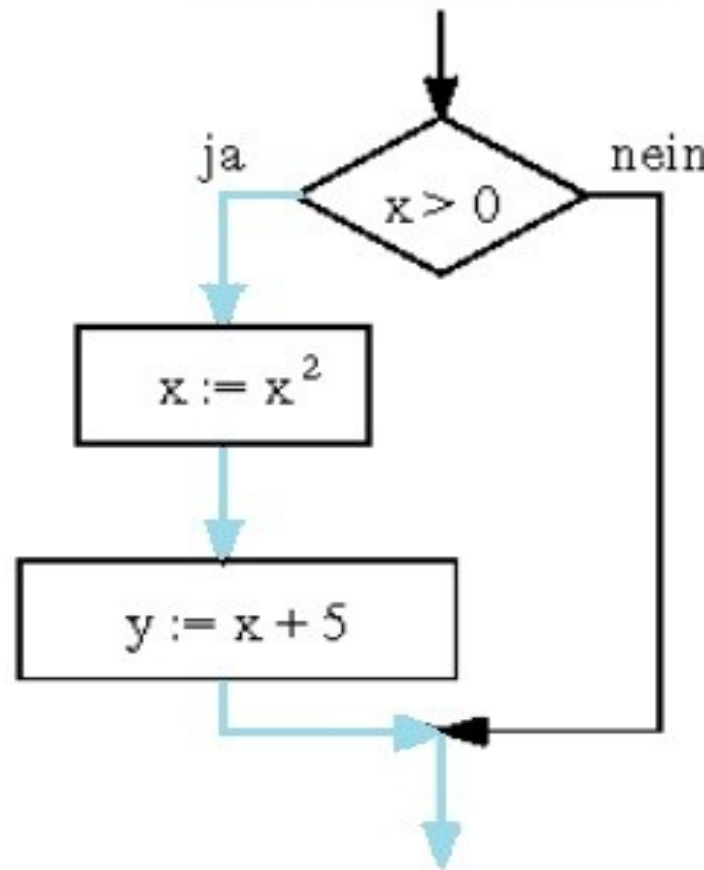
Kann er das richtige vom falschen Programm unterscheiden ?

Anweisungsüberdeckung: Aussagekraft (Beispiel)

Richtiges Programm



Falsches Programm



Testfall T1:

Eingabedaten:

$x=5, y=0$

Solldaten: $x=25, y=30$

erfüllt zwar **C₀-Überdeckung**,
deckt aber nicht
Fehler im
falschen
Programm auf.

Schwaches Kriterium, z.B. weil:

- „**Leere**“ **Kante** (Kante überbrückt Knoten) nicht berücksichtigt.
 - **Beispiele:**
 - ELSE-Kante (zwischen IF und ENDIF) mit leerem ELSE-Teil (siehe Beispiel oben)
 - Rücksprung zum Anfang einer Repeat-Schleife
 - BREAK.
 - **Fehlende** Anweisungen nicht erkannt !

Bewertung:

- Geringe Zahl von Eingabedaten.
 - Notwendiges, aber **kein hinreichendes Testkriterium.**
 - **Mangelhafte Aussagekraft:** Nicht-ausführbare Programmteile werden entdeckt, alle anderen Fehler nur zufällig entdeckt.
- => Stärkeres Kriterium notwendig: Zweigüberdeckung !

Zweigüberdeckung (C_1 -Test, auch genannt Entscheidungsüberdeckung):

- Testdatenmenge T erfüllt C_1 -Überdeckung für Programm P , g.d.w. es **für jede Kante k** im Kontrollflussgraphen von P mind. einen **Weg** in $Wege(T, P)$ gibt, zu dem k gehört.
- **Testwirksamkeitsmaß C_{Zweig} (Zweigüberdeckungsgrad, Verhältnis der besuchten Zweige zur Gesamtzahl von Zweigen):**

$$C_{\text{Zweig}} =_{\text{def}} \frac{\text{Anzahl der besuchten Zweige}}{\text{Gesamtanzahl der Zweige}}$$

- Ein anderes **Testwirksamkeitsmaß** für die **C₁-Überdeckung**

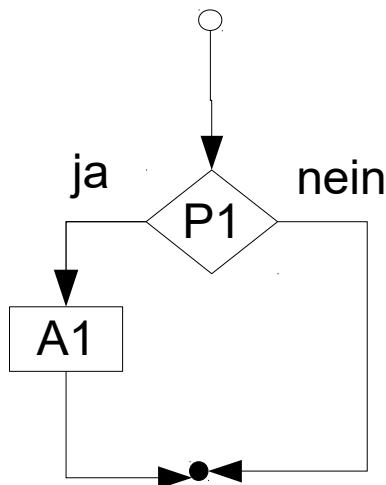
$$\text{TWM}_1 =_{\text{def}} \frac{\text{Anzahl der überdeckten Entscheidungskanten}}{\text{Zahl aller Entscheidungskanten}}$$

- Im Gegensatz zum Testwirksamkeitsmaß C_{Zweig} werden nicht die Zweige gezählt, sondern die **Entscheidungskanten** gezählt.
→ Werte der Maße können unterschiedlich sein

- Ein anderes **Testwirksamkeitsmaß** für die C_1 -Überdeckung

$$TWM_1 =_{\text{def}} \frac{\text{Anzahl der überdeckten Entscheidungskanten}}{\text{Zahl aller Entscheidungskanten}}$$

- Im Gegensatz zum Testwirksamkeitsmaß C_{Zweig} werden nicht die Zweige gezählt, sondern die **Entscheidungskanten** gezählt.
→ Werte der Maße können unterschiedlich sein



Frage: Testwirksamkeitsmaß, wenn beim Test nur der „ja“-Zweig durchlaufen wird?

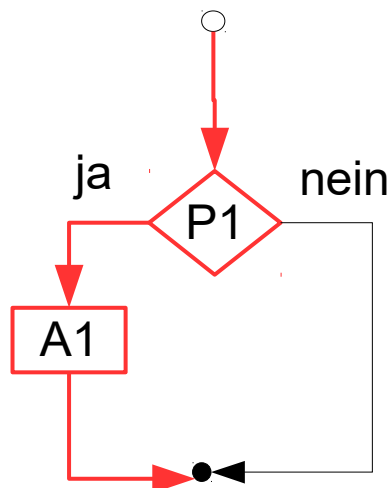
$$TWM_1 = ?$$

$$C_{\text{Zweig}} = ?$$

- Ein anderes **Testwirksamkeitsmaß** für die C_1 -Überdeckung

$$TWM_1 =_{def} \frac{\text{Anzahl der überdeckten Entscheidungskanten}}{\text{Zahl aller Entscheidungskanten}}$$

- Im Gegensatz zum Testwirksamkeitsmaß C_{Zweig} werden nicht die Zweige gezählt, sondern die **Entscheidungskanten** gezählt.
→ Werte der Maße können unterschiedlich sein



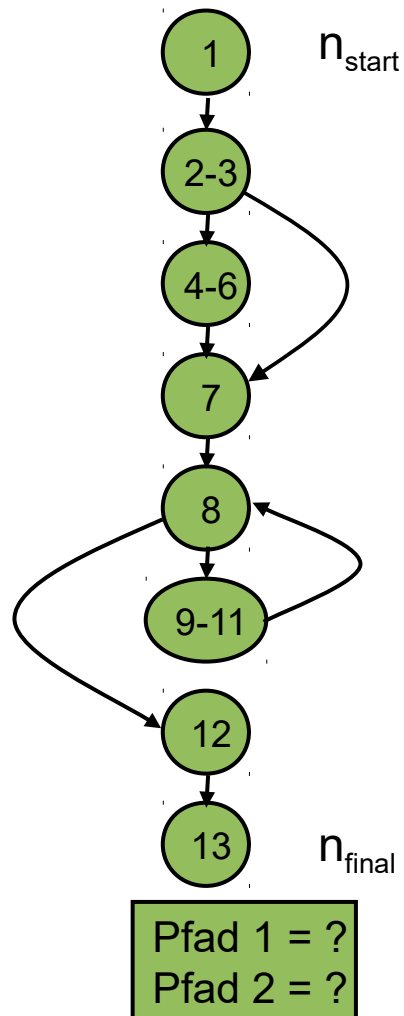
Frage: Testwirksamkeitsmaß, wenn beim Test nur der „ja“-Zweig durchlaufen wird?

$$TWM_1 = 50\%$$

$$C_{\text{Zweig}} = 75\%$$

→ **Aussagekraft** von Testwirksamkeitsmaßen?

Beispiel: Zweigüberdeckung für `ggt()`

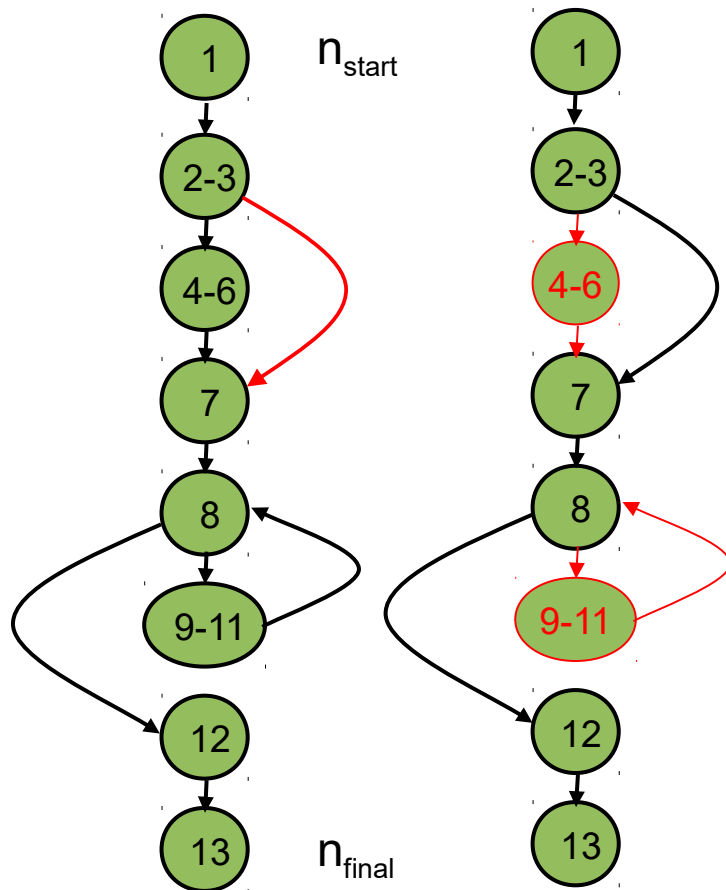


```
1. public int ggt (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
       }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
       }
12.    return n;
13. }
```

Entscheidung

Beispiel für Pfadmenge mit Zweigüberdeckung ? Testdaten ?

Beispiel: Zweigüberdeckung für `ggt()`: Pfadmenge



Pfad 1 = (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13)
Pfad 2 = (1, 2-3, 7, 8, 12, 13)

```
1. public int ggt (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
7.     }
8.     r = m % n;
9.     while (r != 0) {
10.        m = n;
11.        n = r;
12.        r = m % n;
13.    }
14.     return n;
15. }
```

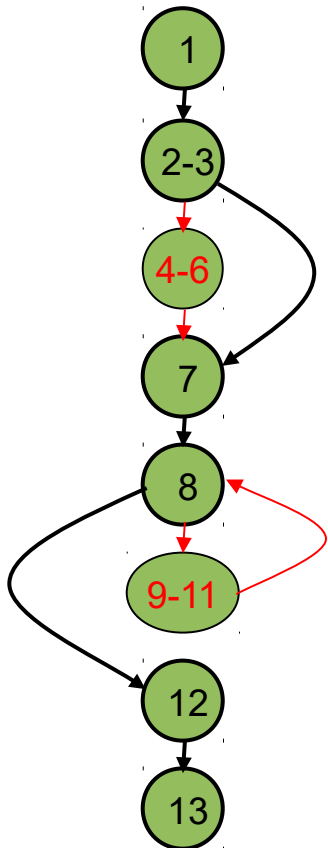
Entscheidung

Beispiel: Zweigüberdeckung für `ggt()`: Testdaten

Pfad: (1, 2-3, 7, 8, 12, 13)

Logischer Testfall: $\{n \leq m \wedge m \bmod n = 0 ;$
 $\text{ggt}(m, n) \}$

Konkreter Testfall: $\{ m = 4, n = 4; 4 \}$



m	n	r
4	4	4

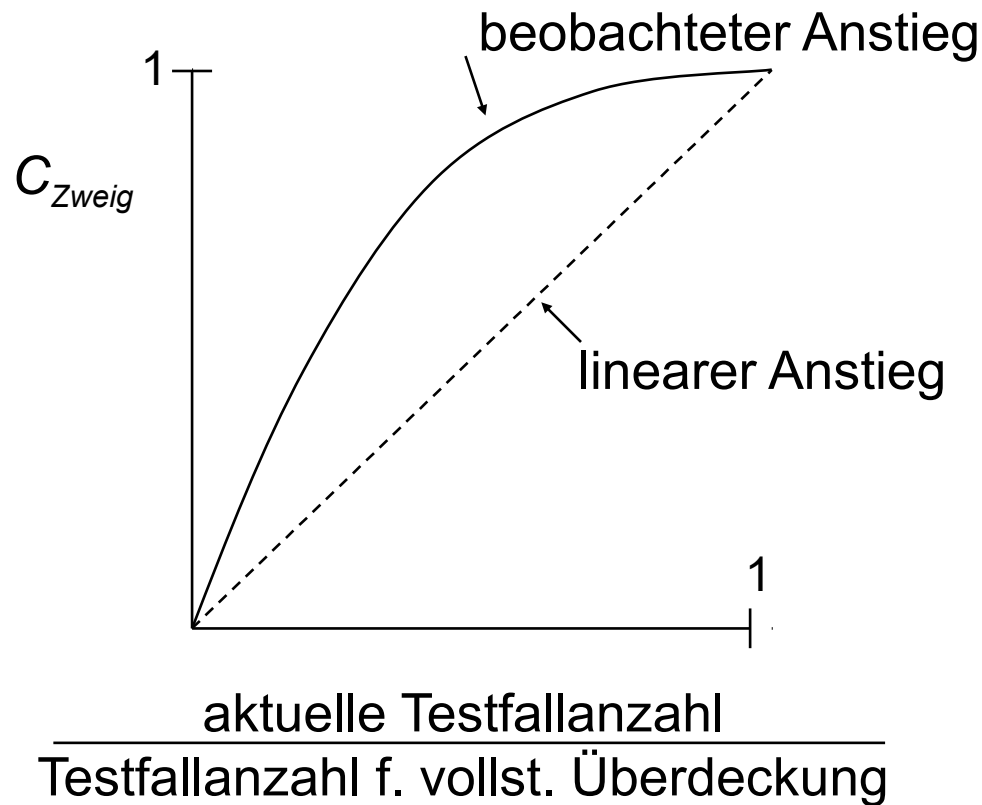
```
1. public int ggt(int m, int n) {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
7.     }
8.     r = m % n;
9.     while (r != 0) {
10.        m = n;
11.        n = r;
12.        r = m % n;
13.    }
14.    return n;
15. }
```

Bewertung:

- Geringe Zahl von Eingabedaten.
- **Aussagekraft besser als C_0 :** Nicht-ausführbare Knoten und Zweige sicher entdeckt, alle anderen Fehler nur zufällig.
- **Durchlaufene Programmteile erkennbar** und ggf. optimierbar.
- Wie üblich: toter Code → keine 100%ige Überdeckung.
- **Unzureichend für Test von Schleifen.** (→ Grenze-Inneres-Überdeckung.)
- Abhängigkeiten zwischen Zweigen nicht berücksichtigt.
(→ Pfadüberdeckungstest.)
- Kein Test komplexer, zusammengesetzter Bedingungen.
(→ Bedingungsüberdeckungstest.)
- **Empirische Untersuchungen:** Bei 100%iger Abdeckung nur 25% der Fehler erkennbar.
→ Aber immerhin noch besser als völlig unsystematisches Testen !

Zweigüberdeckungsrate vs. Anzahl Testfälle

Zweigüberdeckungsrate als Funktion der Testfallanzahl
(gilt in ähnlicher Weise für andere Überdeckungsmaße) [Lig90]:

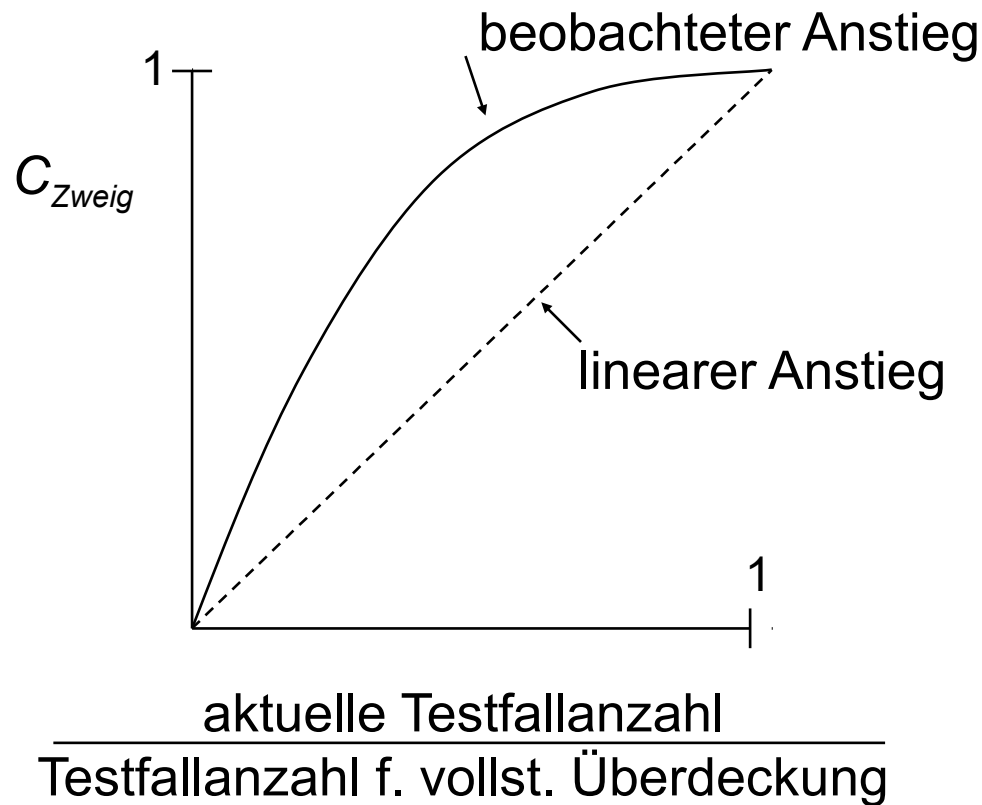


Empirisch beobachteter
Zusammenhang (keine streng
mathematische Notwendigkeit).

**Zur Diskussion:
Was bedeutet das ?**

Zweigüberdeckungsrate vs. Anzahl Testfälle

Zweigüberdeckungsrate als Funktion der Testfallanzahl
(gilt in ähnlicher Weise für andere Überdeckungsmaße) [Lig90]:



Empirisch beobachteter
Zusammenhang (keine streng
mathematische Notwendigkeit).

Zur Diskussion: Was bedeutet das ?

→ Anfangs erhöht sich C_{Zweig}
bei Erhöhung der Testanzahl
schnell, später langsam.
(Vgl. „80-20-Regel“.)

1) Kann man mit einer **100%iger Anweisungsüberdeckung** eine **100%ige Zweigüberdeckung** garantieren?

2) Und umgekehrt ?

Antwort:

1)

2)

1) Kann man mit einer **100%iger Anweisungsüberdeckung** eine **100%ige Zweigüberdeckung** garantieren?

2) Und umgekehrt ?

Antwort:

1) **Nein.** Vgl. Beispiel auf Folie 25/26.

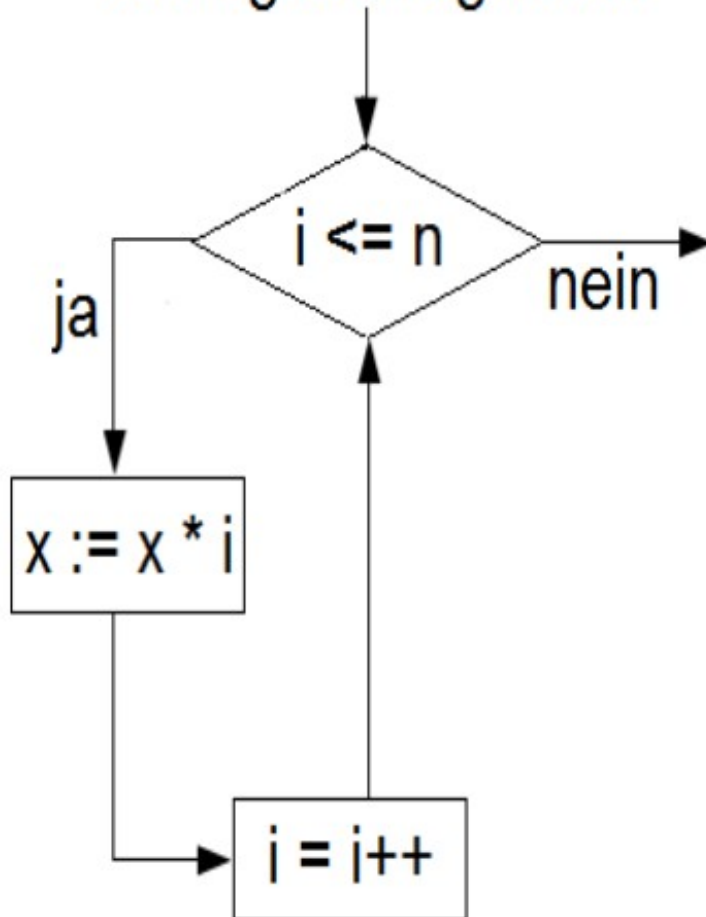
2) **Ja:** Mit 100%iger Zweigüberdeckung kann man 100%ige Anweisungsüberdeckung garantieren.

- Beim Durchlauf aller Zweige werden alle Anweisungen überdeckt.

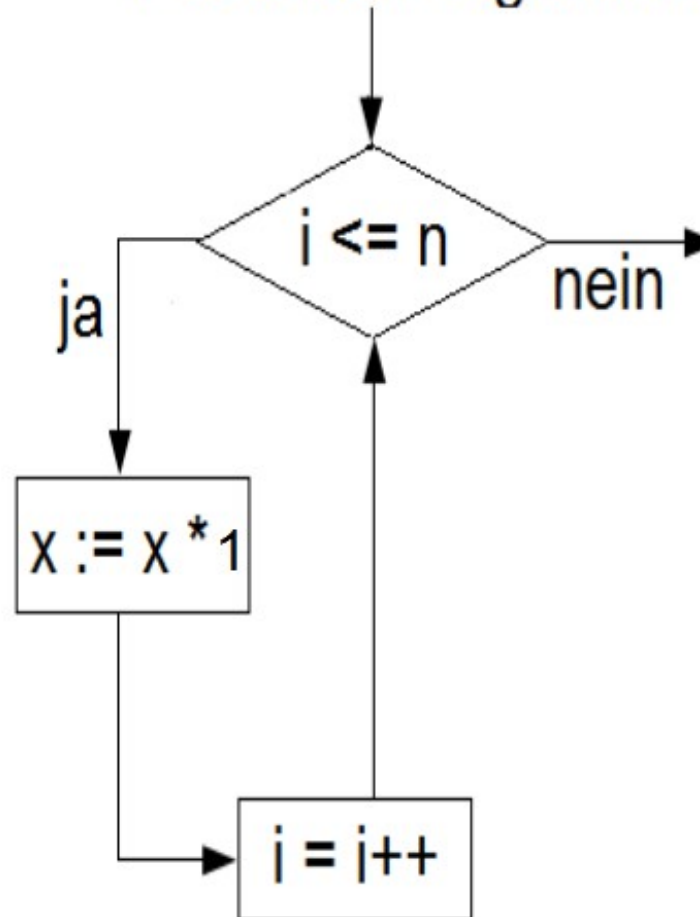
Zweigüberdeckung

Aussagekraft (Beispiel)

Richtiges Programm



Falsches Programm



Testdatum:

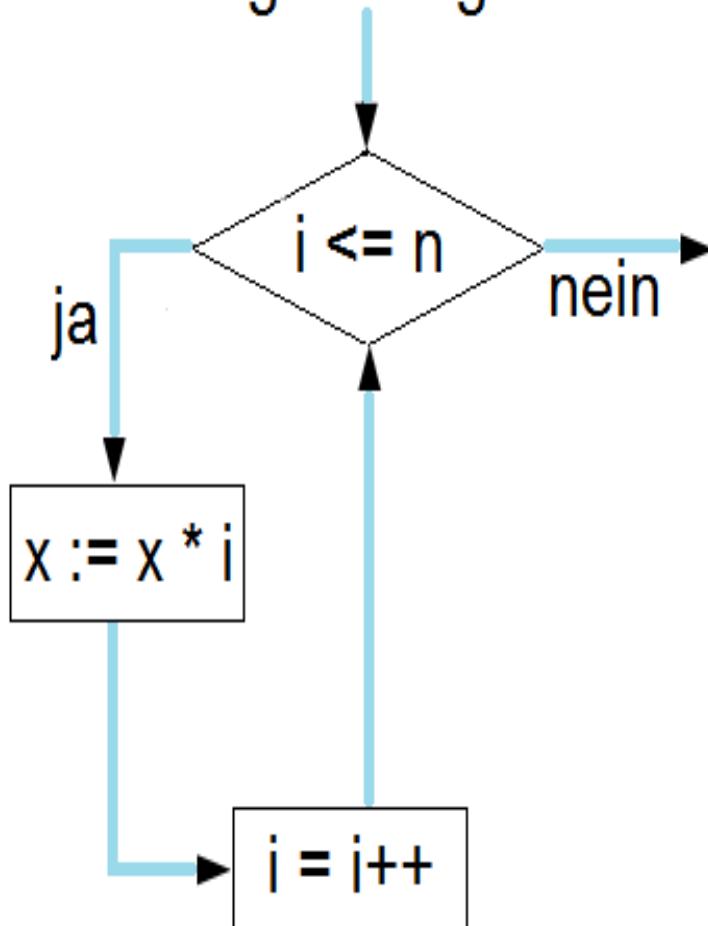
Wie könnte ein einzelner **Testfall** aussehen, der **C₁-Überdeckung** erfüllt ?

Kann er das richtige vom falschen Programm unterscheiden ?

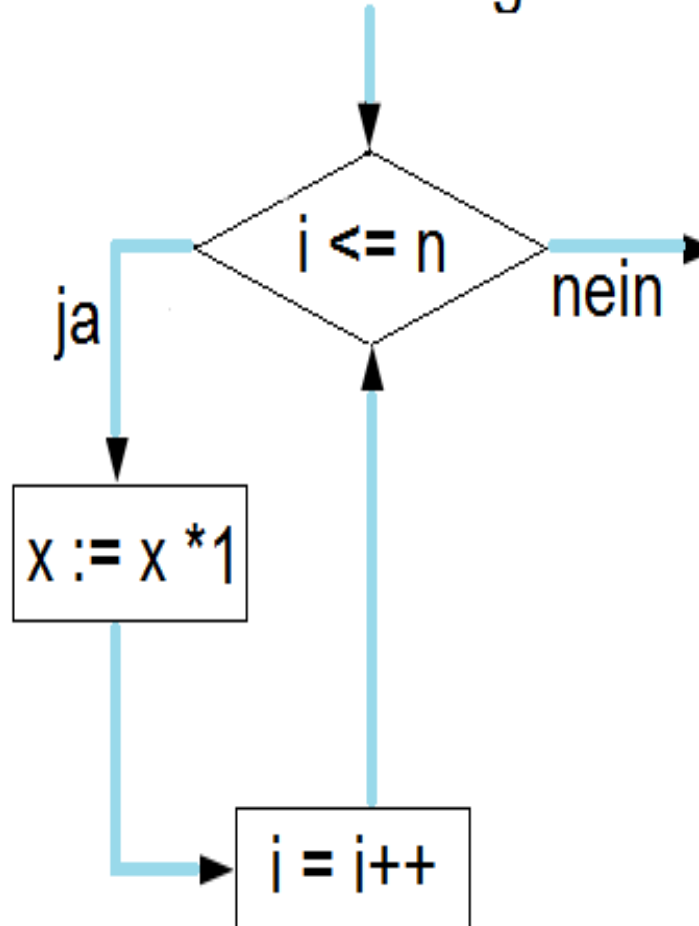
Zweigüberdeckung

Aussagekraft (Beispiel)

Richtiges Programm



Falsches Programm



Testfall T1:

Eingabedaten:

$i=1, x=1, n=1$

Solldaten: $i=2, x=1$

erfüllt zwar C_1 -

Überdeckung,

deckt aber nicht Fehler im falschen Programm auf, da die Schleife nur einmal durchlaufen wird.

→ Stärkeres Kriterium notwendig: **Grenze-Inneres-Überdeckung !**

Grenze-Inneres-Überdeckung (gi): Dynamisches, kontrollfluss-basiertes Testentwurfsverfahren. Fordert, dass **jede Schleife**:

- **genau einmal** ausgeführt wird und
- **mehr als einmal** ausgeführt wird.

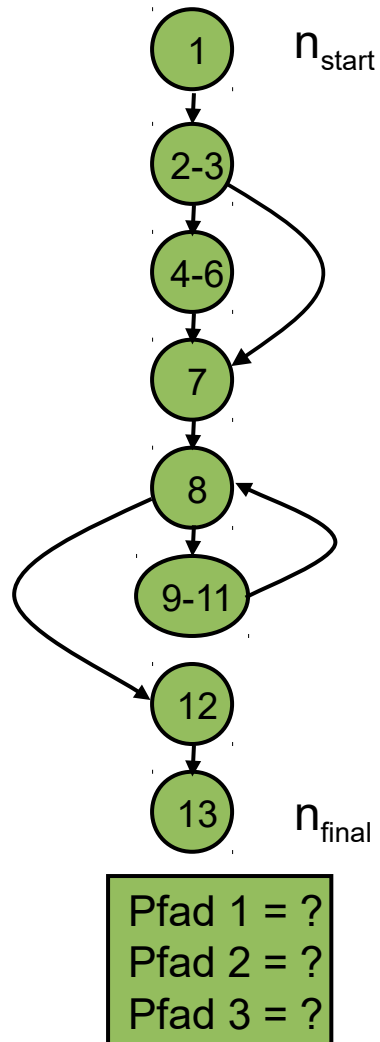
Außerdem soll **jede abweisende Schleife** in min. einem Testfall **gar nicht** ausgeführt werden. Dies sind:

- **While, for**

(nur wenn Abbruchbedingung bei erstmaliger Auswertung wahr).

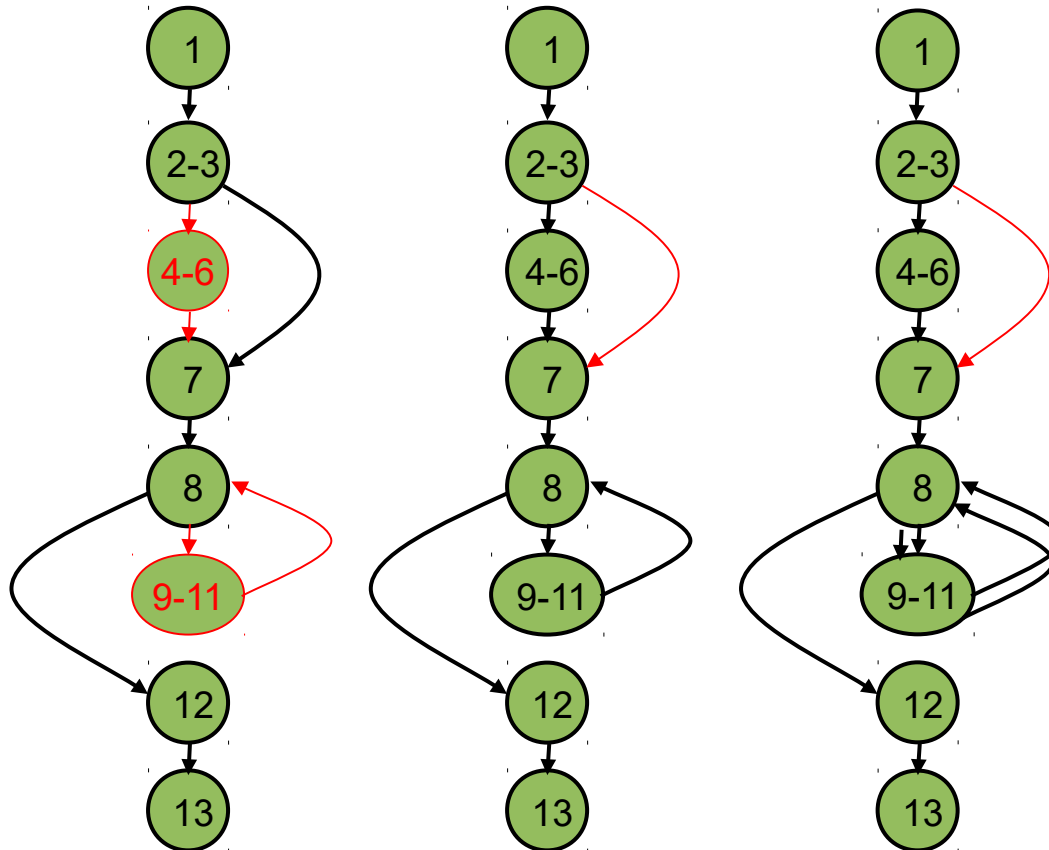
Zugehöriger Überdeckungsgrad:

$$\text{Grenze-Inneres-Überdeckungsgrad} = \frac{\text{Anzahl (gi) getestete Schleifen}}{\text{Gesamtzahl Schleifen}}$$



```
1. public int ggT (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
       }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
       }
12.    return n;
13. }
```

Beispiel für Pfadmenge mit Grenze-Inneres-Überdeckung ?



Pfad 1 = (1, 2-3, 7, 8, 12, 13)
Pfad 2 = (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13)
Pfad 3 = (1, 2-3, 4-6, 7, 8, 9-11, 8, 9-11, 8, 12, 13)

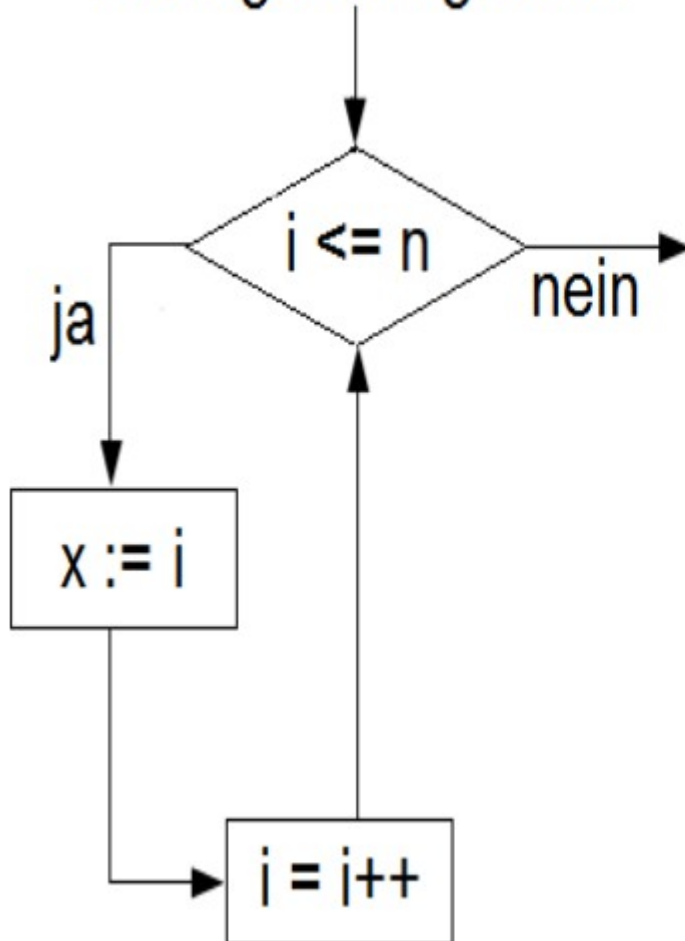
```
1. public int ggt (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
       }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
       }
12.    return n;
13. }
```

- In Aussage auf **Schleifen** beschränktes Kriterium.
 - Verlangt nur, Schleifen auf bestimmte Art zu testen.
 - Berücksichtigt keine Programmteile **außerhalb** von Schleifen.
 - Erkennt z.B. fehlende Anweisungen in „**leeren**“ **Zweigen** nicht !
 - „Mehr als einmal“ ausführen findet ggf. nicht Fehler, die erst ab bestimmter Durchlaufzahl manifest (s. nächste Folie).
- Daher in Praxis höchstens als **ergänzendes** Kriterium verwenden !
- Einzelne Schleifen **unabhängig voneinander** betrachtet (vgl. Zweige in Zweigüberdeckung).
- Für **verschachtelte Schleifen** gibt es spezialisierte, in ihrer Stärke abgestufte Überdeckungsmaße.

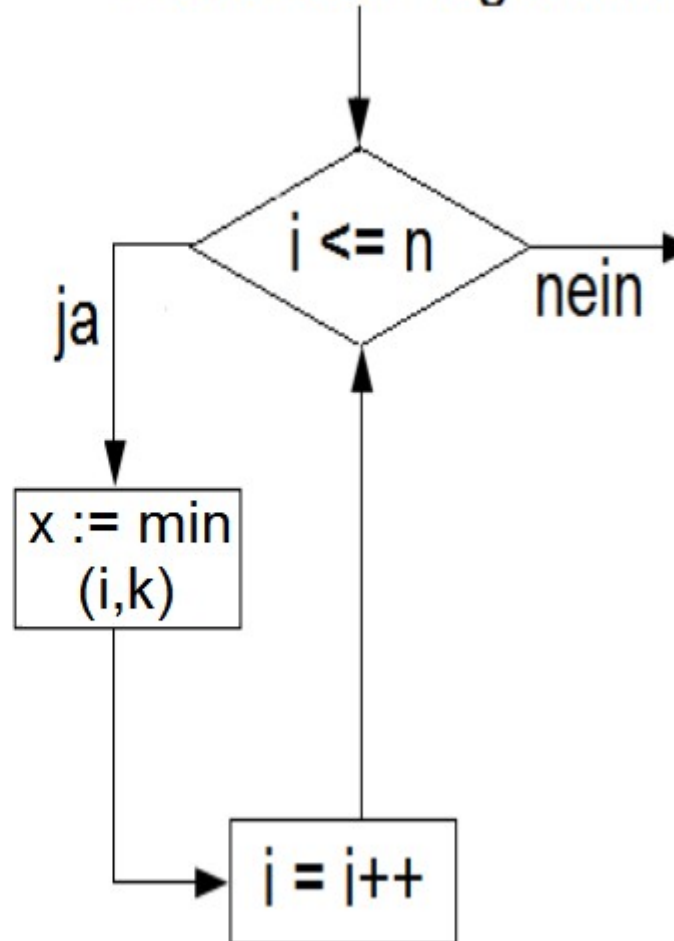
Grenze-Inneres-Überdeckung

Aussagekraft (Beispiel)

Richtiges Programm



Falsches Programm

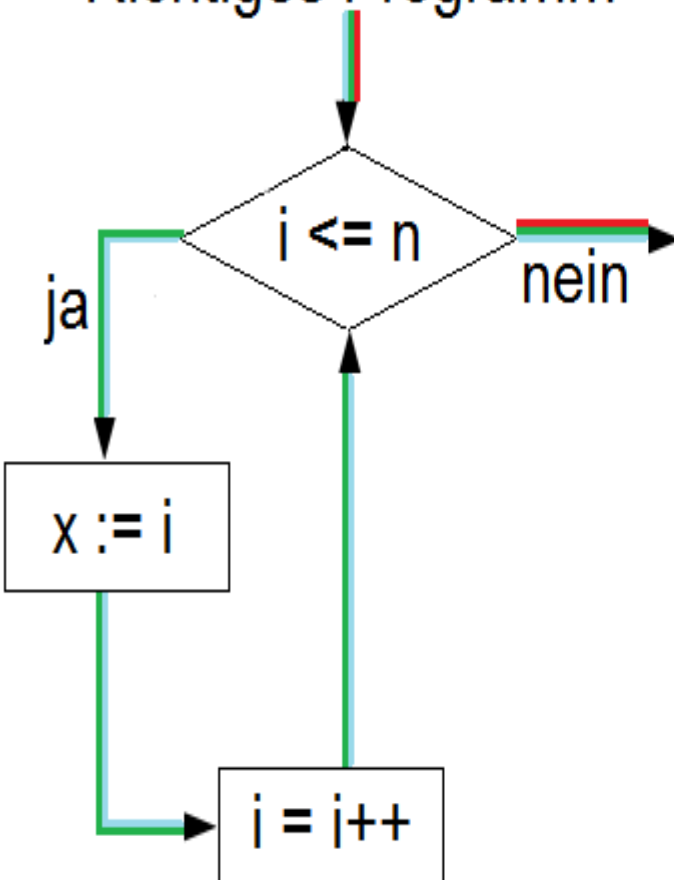


Testdatum:

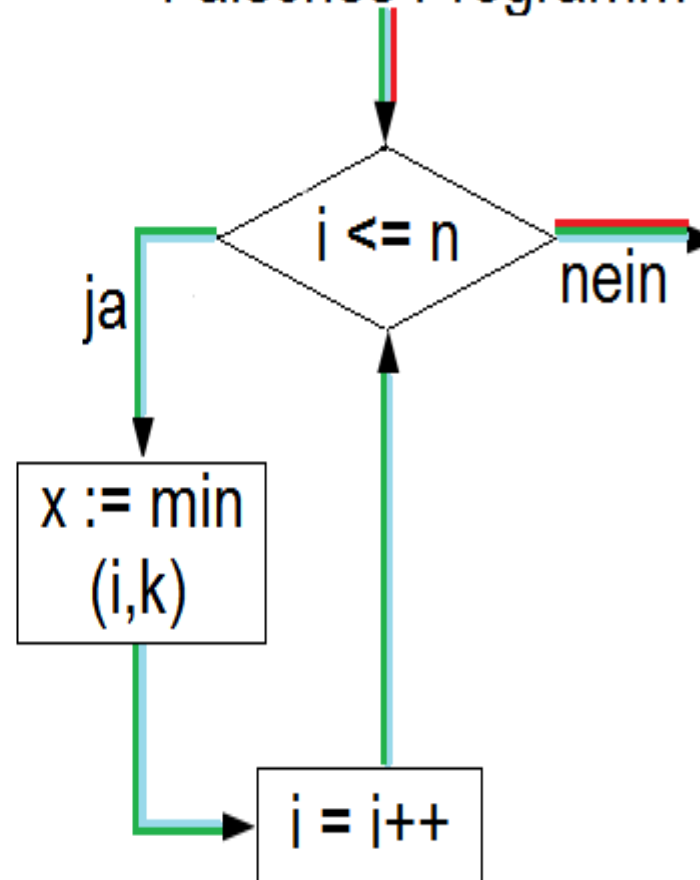
Wie könnte eine **Testfallmenge** aussehen, die **C_1 -Überdeckung** und **Grenze-Inneres-Überdeckung** erfüllt ?

Kann sie das richtige vom falschen Programm unterscheiden ?

Richtiges Programm



Falsches Programm



Testfall T0:

Eingabe: $i=x=1, n=k=0$;

Soll: $i=x=1, n=k=0$

Testfall T1:

Eingabe: $i=x=n=k=1$;

Soll: $i=2, x=n=k=1$

Testfall T2:

Eingabe: $i=x=1, n=k=3$;

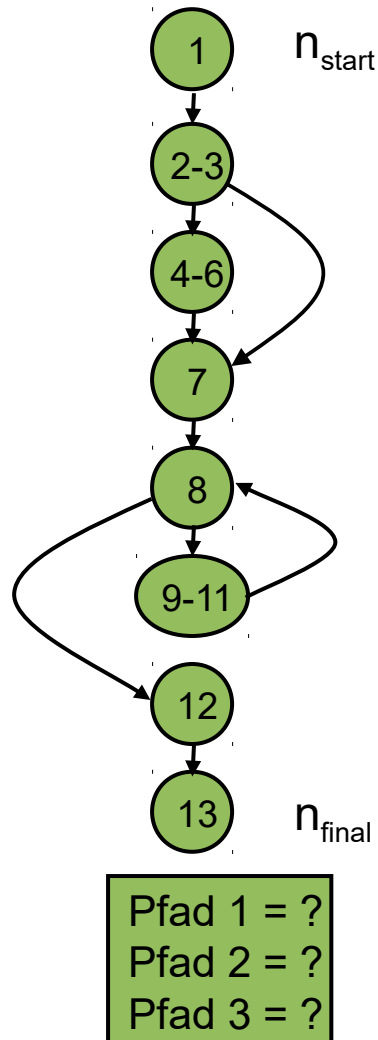
Soll: $i=x=3$

erfüllen zwar **C₁**- und **Grenze-Inneres-Überdeckung**, decken aber nicht Fehler im falschen Programm auf, da $k < n$ nicht getestet wird.

→ Stärkeres Kriterium notwendig: **Pfadüberdeckung** !

- Dynamisches, kontrollflussbasiertes Testentwurfsverfahren.
- Fordert **min. einmalige Ausführung jedes Pfads** im Kontrollflussgraphen.
- Auch als **C_{∞} -Maß** bezeichnet.

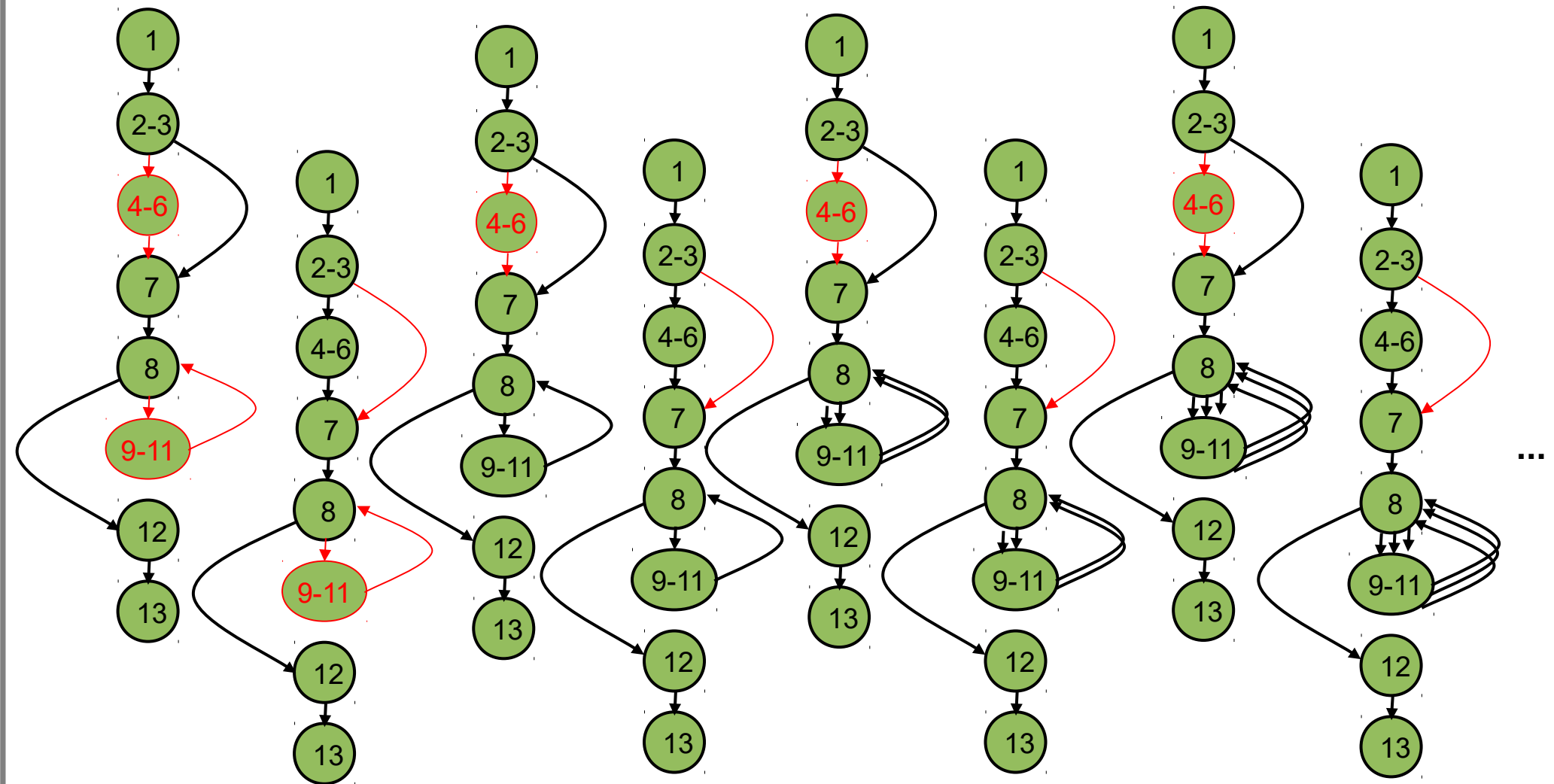
$$\text{Pfadüberdeckungsgrad} = \frac{\text{Anzahl durchlaufene Pfade}}{\text{Gesamtzahl Pfade}}$$



```
1. public int ggt (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
       }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
       }
12.    return n;
13. }
```

Beispiel für Pfadmenge mit Pfadüberdeckung ?

Beispiel: Pfadüberdeckung für `ggt()`



Frage: Welche obere Grenze lässt sich hier angeben ?

Pfadüberdeckung für `ggt()` : Obere Grenze

Grobe obere Grenze für maximale Anzahl Schleifendurchläufe in Abhängigkeit von Eingaben n, m :

- **Zeile 3-6:** ggf. m, n vertauschen
→ betrachte $\max(n, m)$ als obere Grenze für m, n
- **Zeile 8-11:** Es gilt: $\text{mod}(x, y) < \min(x, y)$.
→ r in jedem Schleifendurchlauf strikt kleiner als im Durchlauf davor.
→ höchstens $\max(n, m)$ Durchläufe.
→ höchstens $\maxint$ Durchläufe.

```
1. public int ggt (int m, int n)
   {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
       }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
       }
12.    return n;
13. }
```

Frage:

- Gibt es Programm, zu dem keine obere Grenze für Schleifenabbruch (und damit Pfadüberdeckung) angegeben werden kann ?

Frage: Programm ohne obere Grenze für Schleifenabbruch ?

Antwort:

Beispiel: Schleife, deren Abbruchwert von (echtem) Zufallszahlengenerator erzeugt wird (Benutzereingaben, radioaktiver Zerfall, ...):

Kann beliebig lange Wert false generieren, bevor true generiert wird.

→ **keine Grenze für Pfadüberdeckung**, wenn Schleifenabbruch abhängig von **externem** Nicht-Determinismus.

Warum werden durch **100%-ige Pfadüberdeckung**, die ja 100% der Pfade testet, **nicht 100% der Fehler gefunden** ?

Antwort:

-
-

Warum werden durch **100%-ige Pfadüberdeckung**, die ja 100% der Pfade testet, **nicht 100% der Fehler gefunden** ?

Antwort:

- Es werden zwar alle Pfade getestet, aber **nicht mit allen möglichen Variablenbelegungen**.
- Es wird nur **vorhandene Funktionalität getestet** und nicht, welche Funktionalität evtl. fehlt.

Bei zyklischen Kontrollflussgraphen **potenziell unendlich viele (beliebig lange) Pfade**.

- Aber: **Obere Grenzen** für mögliche Länge (und damit Anzahl) ggf. aus Spezifikation oder aus technischen Einschränkungen ableitbar.

In Praxis 100%ige Pfadüberdeckung oft nicht erreichbar (vgl. Beispiel T2.0 F12-13).

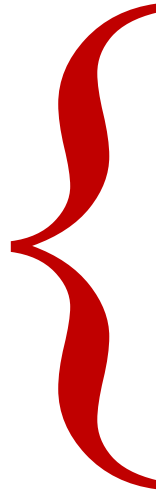
→ **Als theoretisches Vergleichsmaß wichtig.**

Nichtsdestotrotz: auch Pfadüberdeckung kann nicht alle Fehler finden (nicht alle Eingabewerte und Variablenbelegungen getestet).

→ Brauche alternatives Testparadigma als Ergänzung.

→ **Test der Bedingungen, Datenflussbasiertes Testen**
(vgl. folgende Abschnitte)

2.4 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

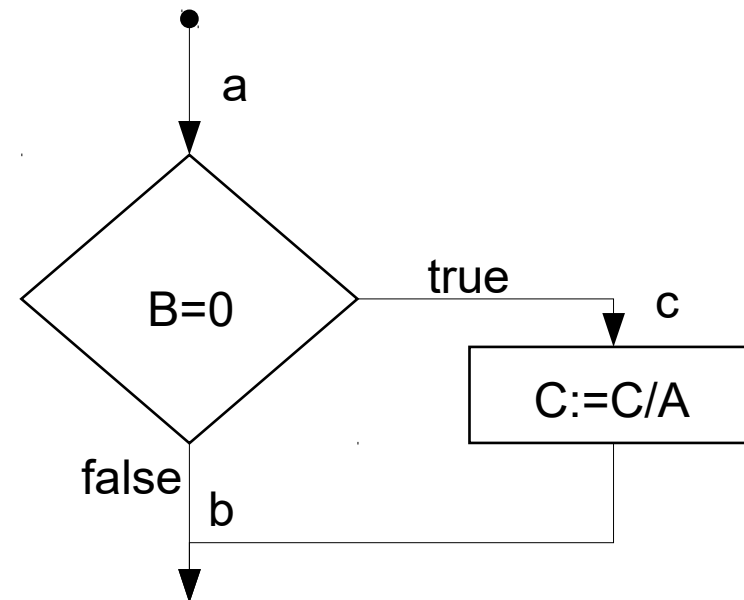
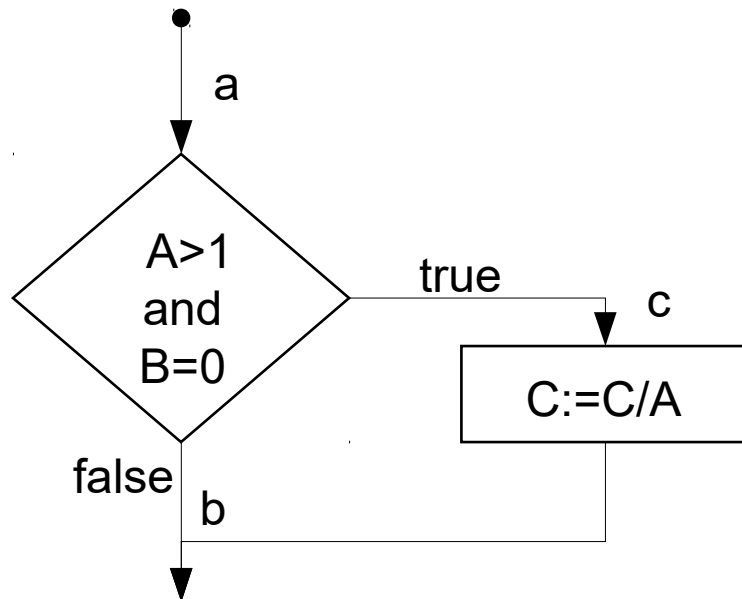
Test der Bedingungen

Datenflussbasierter Test

Statische Analyse

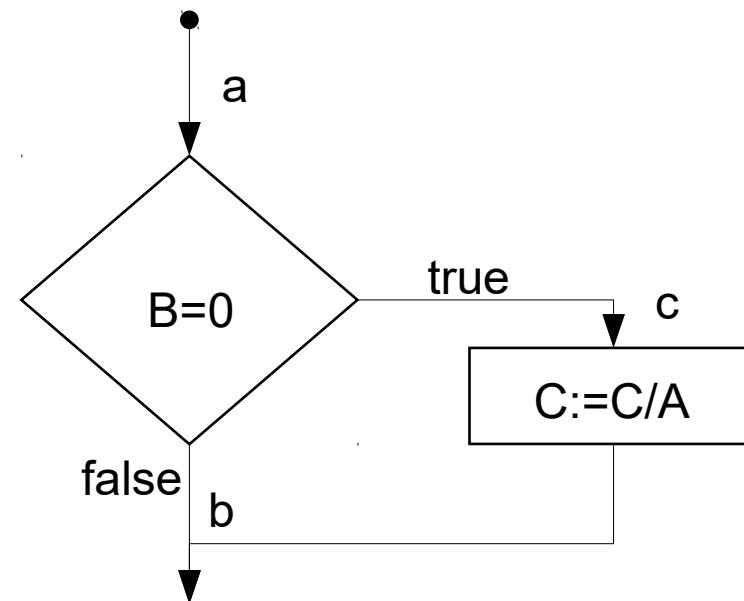
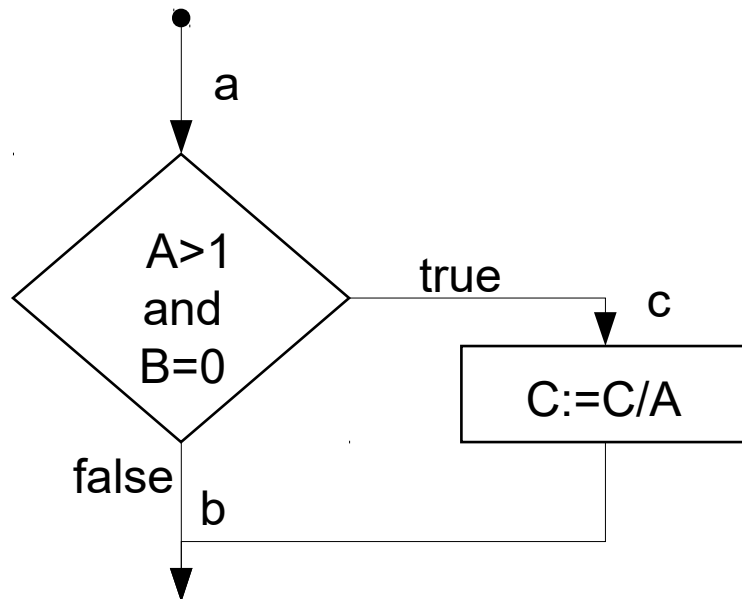
Kann es Testfälle mit Zweigüberdeckung geben, die richtiges und falsches Programm nicht unterscheiden ?

Antwort: ?



Kann es Testfälle mit Zweigüberdeckung geben, die richtiges und falsches Programm nicht unterscheiden ?

Antwort: Testfallmenge $\{(A=2, B=C=0), (A=2, B=1, C=0)\}$.



- Brauchen stärkeres Kriterium, das Teilbedingungen berücksichtigt
- Bedingungsüberdeckung !

- **Wahrheitswerte:** `false`, `true` (oft auch `0`, `1` oder »falsch«, »wahr«).
- **Atomare (Teil-)Bedingung (*condition*):**
 - Variablen vom Typ `boolean`.
 - Operationen mit Rückgabewert vom Typ `boolean`.
 - Vergleichsoperationen.
 - Z.B. `flag; isEmpty(); size > 0`
- **Zusammengesetzte Bedingung (*compound condition*):**
 - Verknüpfung von (Teil-)Bedingungen mit booleschen Operatoren.
 - Basis-Operatoren sind **und** ($\wedge$, $\cap$), **oder** ($\vee$, $\cup$), **nicht** ($\neg$).
- **Entscheidung:** (zusammengesetzte) Bedingung, die Programmablauf steuert.
- **In Java:**
 - `&`, `|`, `^` Bitweise *und*, *oder* und *exklusiv-oder*-Verknüpfung.
 - `&&`, `||` Wie oben, aber lazy evaluation, z.B. `a && b = a ? b : false`.
 - **if** `((size > 0) && (inObject != null)) { ... } else { ... }`.

Bedingungsüberdeckungen:

- Berücksichtigen **Bedingungen in Schleifen und Auswahlkonstrukten** zur Definition von Tests.

Idee: Zusammengesetzte Bedingungen nur einmal zu True und False auswerten ist nicht ausreichend.

→ **Teilbedingungen** (atomare Prädikatterme) **variieren**.

Beispiel:

- if $A > 1$ and $(B = 2 \text{ or } C > 1)$ and D then ... else ...
- Entscheidungsprädikat: $A > 1$ and $(B = 2 \text{ or } C > 1)$ and D
- Atomare Prädikatterme: $A > 1$, $B = 2$, $C > 1$, D

Anderes Beispiel: Vergleich der Zweigüberdeckung von if (a AND b AND c) {...} einer äquivalenten Konstruktion if a {if b {if c {...}}}

Einfache Bedingungsüberdeckung

Einfache (oder auch atomare) Bedingungsüberdeckung (C_2 -Test):

- Testdatenmenge T erfüllt C_2 -Überdeckung g.d.w. es:
 - für jede Verzweigung im Programm mit zwei Ausgängen und
 - für jeden (atomaren) Prädikatterm p des zur Verzweigung gehörenden Booleschen Ausdrucks
 - zu jedem Wahrheitswert von p ein Testdatum t aus T gibt, bei dessen Ausführung p diesen Wahrheitswert annimmt.
- **Hinweis:** Verzweigungen mit mehr als zwei Ausgängen in äquivalentes Konstrukt mit zwei Ausgängen umformbar.
- Garantiert keine Zweigüberdeckung.
→ Testen beider Wahrheitswerte kompletter (zusammengesetzter) Boolescher Ausdrücke nicht garantiert.

Einfache Bedingungsüberdeckung: Testwirksamkeitsmaß C_2

$$C_2 =_{\text{def}} \frac{\text{Anzahl Prädikate}_{\text{TRUE}} + \text{Anzahl Prädikate}_{\text{FALSE}}}{2 * \text{Anzahl der Prädikate}}$$

Wie üblich: 100%ige Überdeckung nicht erreichbar, wenn Belegung von Teilbedingungen mit true und false nicht erreicht werden kann (weil durch Programmkontext ausgeschlossen).

Beispiele zur einfachen Bedingungsüberdeckung

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	C	$A \wedge B \wedge C$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

2 Ausdrücke, 2 Testfälle

2 Ausdrücke, 2 Testfälle

3 Ausdrücke, 2 Testfälle

Einfache Bedingungsüberdeckung: Anzahl Testfälle

Wieviele Testfälle benötigt für einfache Bedingungsüberdeckung für
Bedingung aus **2 Ausdrücken** ?

-

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Wieviele bei **n>2 Ausdrücken** ?

-

A	B	C	$A \wedge B \wedge C$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Wieviele Testfälle benötigt für einfache Bedingungsüberdeckung für Bedingung aus **2 Ausdrücken** ?

- Falls Überdeckung erreicht werden kann, reichen 2: Entweder $T1=((0,0), (1,1))$ oder $T2=((0,1),(1,0))$. Falls beide nicht erreicht werden können (weil je mind. eine Kombination nicht erreicht werden kann), bleiben ohnehin höchstens zwei Testfälle übrig.

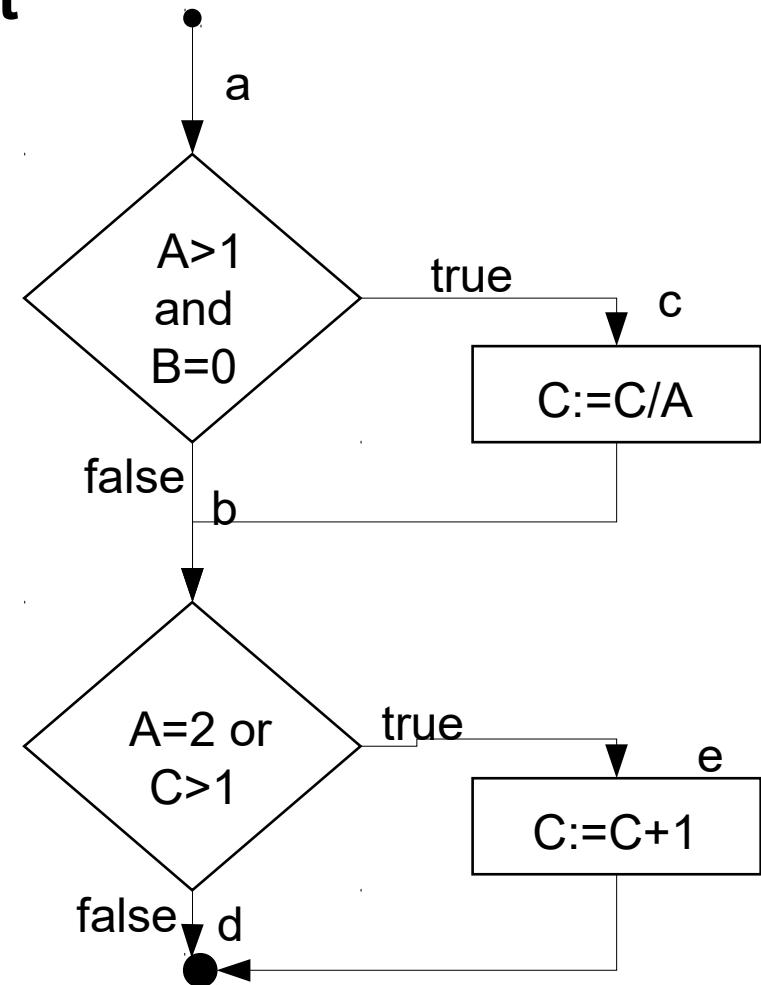
Wieviele bei **$n > 2$ Ausdrücken** ?

- Falls Überdeckung erreicht werden kann, reichen n .

Beweis per Induktion: Induktionsanfang s.o.

Induktionsschritt: Gilt für $n+1$ auf Basis von n : Die n Testfälle decken mind. einen Wert (true oder false) für den $n+1$ sten Ausdruck ab; für den anderen reicht ein weiterer Testfall.

Kann es einen **C₂-Testfall** geben, der **nicht alle Zweige** testet ?



Beispiel

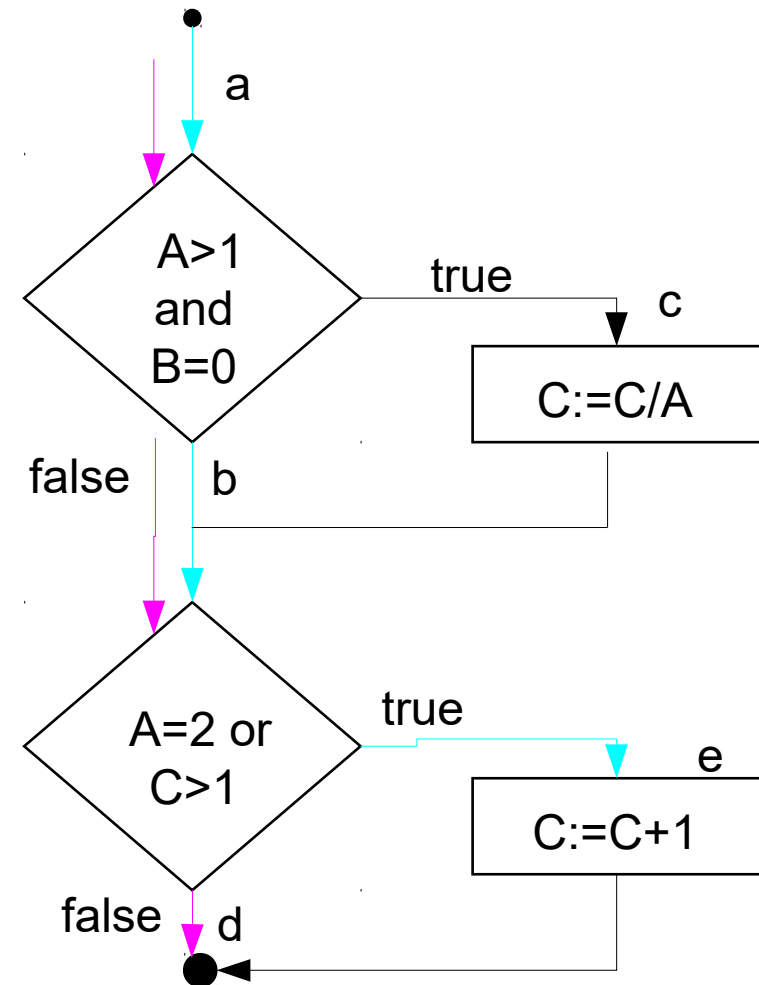
- Testdatum A=2,B=1,C=4

- A>1 true
- B=0 false
- A=2 true
- C>1 true
- Weg: abe

- Testdatum A=1,B=0,C=1

- A>1 false
- B=0 true
- A=2 false
- C>1 false
- Weg: abd

→ Zweig c wird nicht überdeckt!

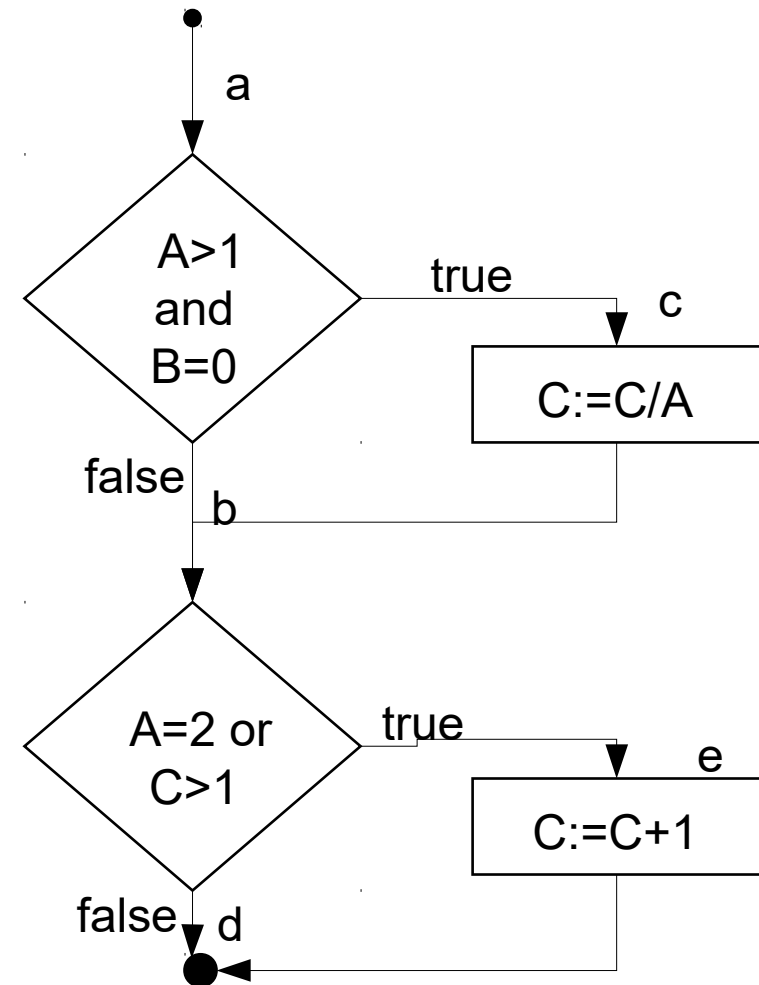


Zweig-/Bedingungsüberdeckung:

Testdatenmenge erfüllt Zweig-/ Bedingungsüberdeckung
gdw. sie **C₂-Überdeckung** (atomare Bedingungsüberdeckung) **und**
C₁-Überdeckung (Zweigüberdeckung) **erfüllt.**

Zweig-/Bedingungsüberdeckung: Ausdrucksstärke

Kann es einen **Zweig- / Bedingungs-
überdeckungs-Testfall** geben, der
nicht alle **Zweigkombinationen**
testet ?



Beispiel:

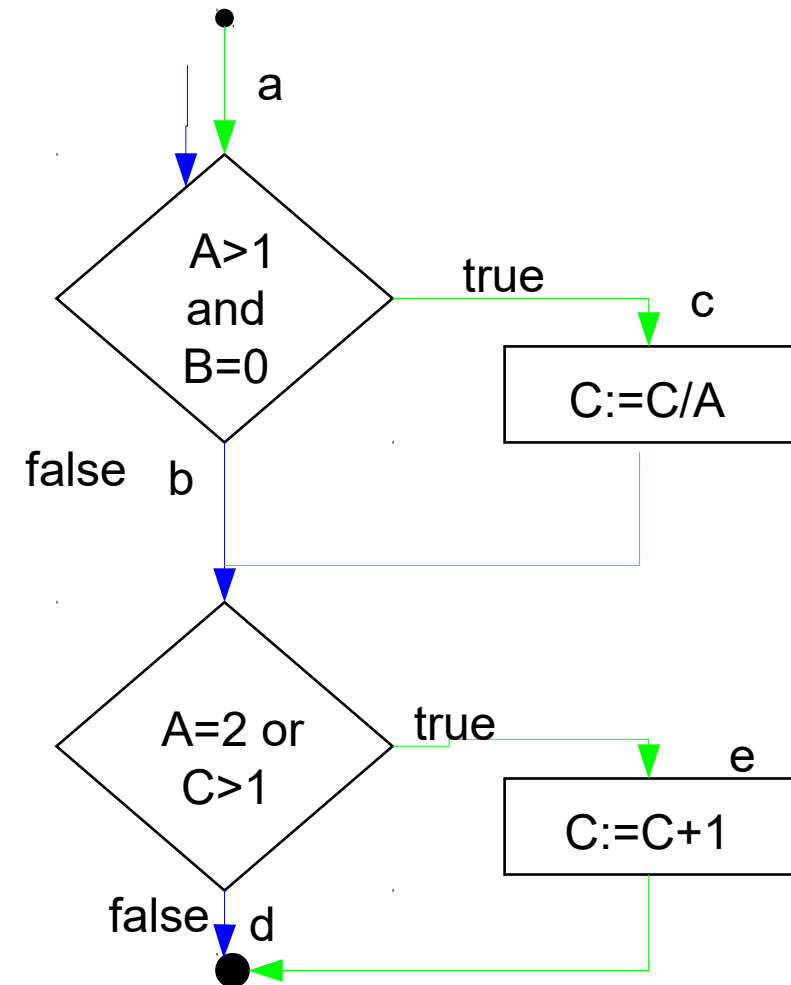
- **Testdatum A=2,B=0,C=4:**

- A>1 true
- B=0 true
- A=2 true
- C>1 true
- Weg: ace

- **Testdatum A=1,B=1,C=1:**

- A>1 false
- B=0 false
- A=2 false
- C>1 false
- Weg: abd

→ Alle Zweige nun überdeckt, aber
nicht alle **Zweigkombinationen!**

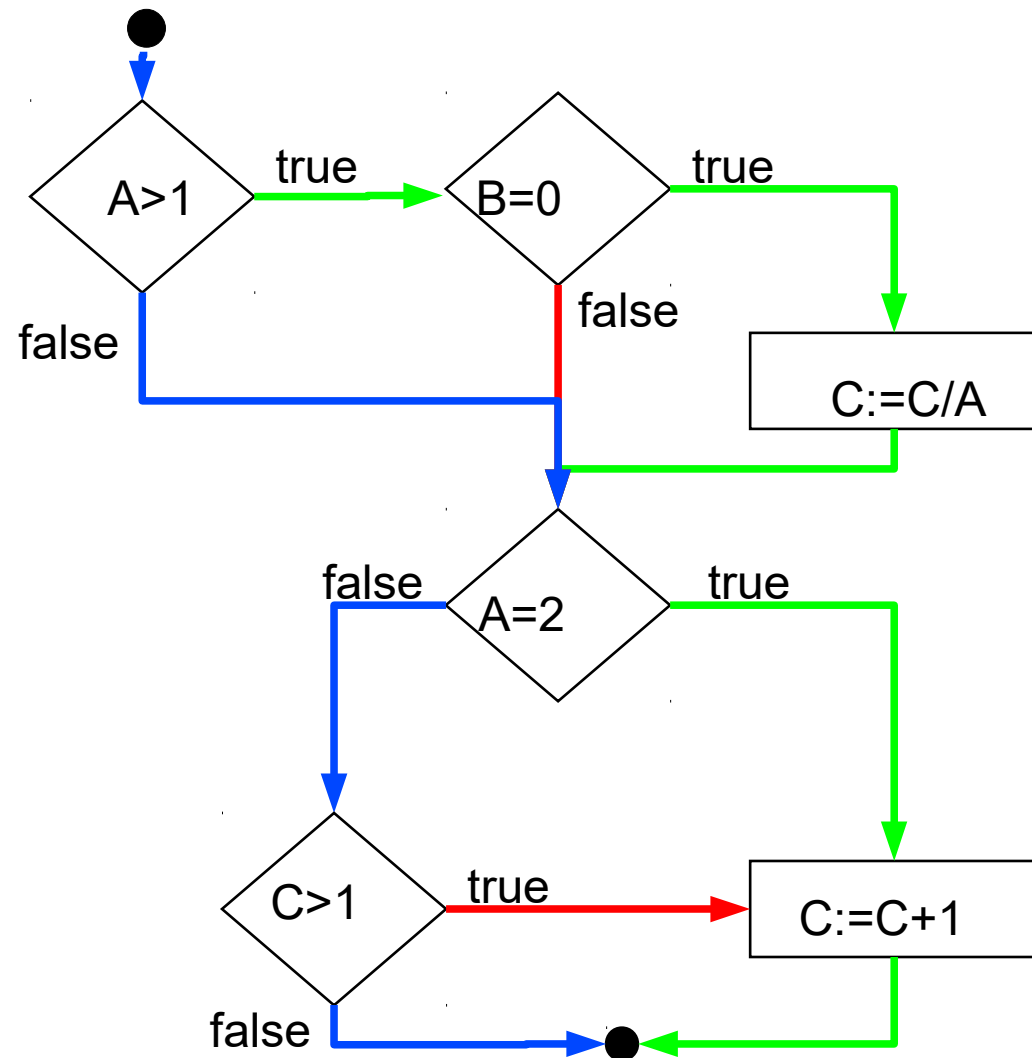


Zweig-/Bedingungsüberdeckung: Ausdrucksstärke

Zur Veranschaulichung: Umformung
des Kontrollflussgraphen
(normalerweise nicht vorgesehen,
hier nur zu didaktischen Zwecken):

- Testdatum $A=2, B=0, C=4$
- Testdatum $A=1, B=1, C=1$
- 2 Zweige fehlen !

Lösung:
Mehrfachbedingungsüberdeckung !





Mehrfachbedingungsüberdeckung (C_3 - oder $C_2(M)$ -Überdeckung):

Testdatenmenge erfüllt Mehrfachbedingungsüberdeckung g.d.w.

- **für jede Verzweigung** im Programm mit zwei Ausgängen und
- für zur Verzweigung gehörenden **Booleschen Ausdruck**
- **jede mögliche Kombination** der Wahrheitswerte atomarer Prädikatterme ausgeführt wird.

Falls Programm keine Tautologien enthält: Ergibt true/false-Belegung der zusammengesetzten Booleschen Ausdrücke $\Rightarrow$ Impliziert Zweigüberdeckung.

Wie üblich: 100%ige Überdeckung nicht erreichbar, wenn Kombination von Wahrheitswerten nicht erreicht werden kann.

- Z.B. bei $(x > 0) \ \&\& \ (x < 5)$ ist (falsch, falsch) nicht realisierbar

Beispiel: Mehrfachbedingungsüberdeckung

Teste jede Kombination der Wahrheitswerte aller atomarer Ausdrücke !

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	C	$A \wedge B \wedge C$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

2 Ausdrücke, 4 Testfälle

2 Ausdrücke, 4 Testfälle

3 Ausdrücke, 8 Testfälle

Frage: Wieviele Testfälle werden benötigt (für n atomare Prädikatterme) ?

Testfall-Anzahl bei n atomaren Ausdrücken: 2^n .

$$\text{Mehrfachbedingungsüberdeckungsgrad} = \frac{\text{Anzahl getesteter Kombinationen atomarer Ausdr.}}{2^{\text{Gesamtzahl atomarer Ausdrücke}}}$$

→ Aufwändig zu realisieren.

Alle Kombinationen durch Testdaten **nicht immer realisierbar.**

→ Weist nicht auf Fehler der Programmlogik hin!

→ Testbeurteilung wird erschwert!

Besser: Minimale Mehrfachbedingungsüberdeckung.

Testdatenmenge erfüllt **Minimale Mehrfachbedingungsüberdeckung** ($C_2(mM)$ -Überdeckung, *minimal bestimmende Mehrfachbedingungsüberdeckung*) g.d.w.:

- für jede Verzweigung im Programm mit **zwei Ausgängen** und
- für zur Verzweigung gehörenden **Booleschen Ausdruck**
- folgende Kombinationen der Wahrheitswerte der enthaltenen atomaren Prädikatterme ausgeführt werden:
 - **Jede Kombination** von Wahrheitswerten, für die Änderung des Wahrheitswertes **eines Terms** den **gesamten Wahrheitswert ändert**.

[Bemerkung: Da diese Eigenschaft jede Kombination von Wahrheitswerten für sich betrachtet, gibt es immer eine **eindeutige** kleinste Kombinationsmenge, die mM erfüllt (im Gegensatz z.B. zur einfachen Bedingungsüberdeckung).]

Minimale Mehrfachbedingungsüberdeckung

$$\text{Minimale Mehrfachbedingungs-} \quad = \quad \frac{\text{Anzahl getesteter MM Kombinationen}}{\text{Gesamtzahl MM Kombinationen}} \\ \text{überdeckungsgrad}$$

100% **minimale Mehrfachbedingungsüberdeckung** impliziert 100% **Entscheidungsüberdeckung**.

Anzahl Testfälle **im Normalfall** geringer als bei Mehrfachbedingungsüberdeckung.

Gesamtzahl **Kombinationen** für $C_2(mM)$ -Überdeckung **bei reinen AND bzw. OR-Bedingungen** mit n atomaren Prädikaten: $n+1$.

Algorithmus für **Überprüfung** eines Testfalls auf MM-Bedingung: Direkte Überprüfung der Eigenschaft „Änderung des Wahrheitswertes **eines Terms** ändert den **gesamten** Wahrheitswert“.

Algorithmus für die **Generierung** einer MM-Testfallmenge: Sukzessive Überprüfung der MM-Bedingung für jeden möglichen Testfall.

Frage: Minimale Mehrfachbedingungsüberdeckung

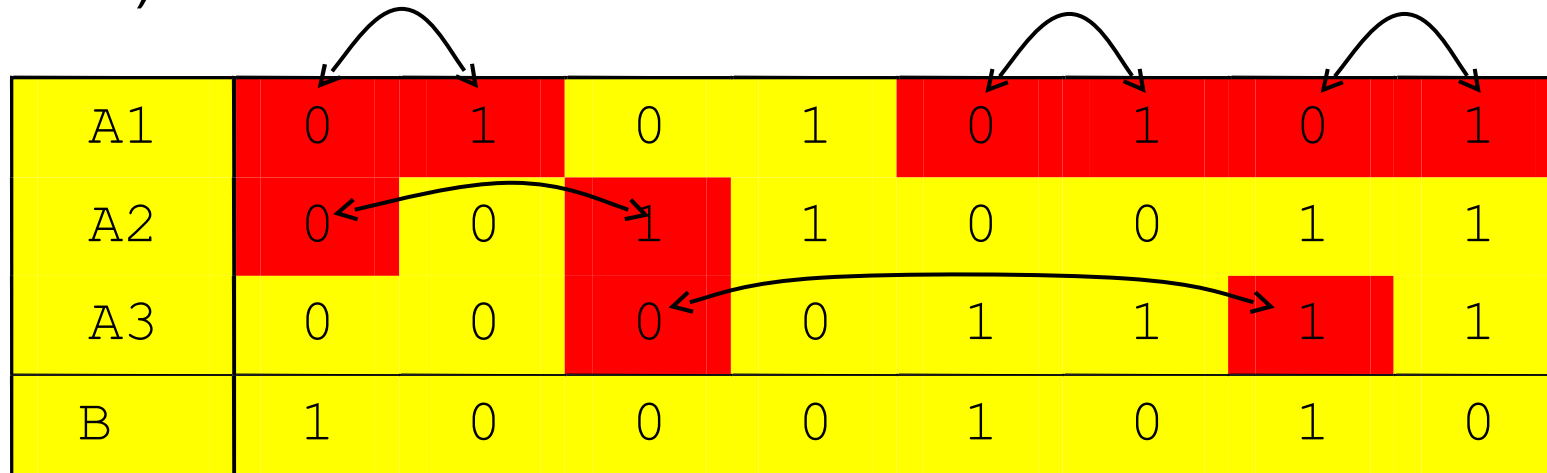
Seien A_1 , A_2 , A_3 atomare Ausdrücke und $B = \text{NOT}(A_1) \text{ AND NOT}(A_2 \text{ AND NOT}(A_3))$ der durch unten abgebildete Wahrheitswerttabelle definierte Gesamtausdruck.

Frage: Welche Werte-Kombinationen von A_1 , A_2 und A_3 sind für **minimale Mehrfachbedingungsüberdeckung** mit Testfällen zu bewirken ?

A_1	0	1	0	1	0	1	0	1
A_2	0	0	1	1	0	0	1	1
A_3	0	0	0	0	1	1	1	1
B	1	0	0	0	1	0	1	0

Antwort: **Mit Testfällen alle Kombinationen (Spalten) bewirken**, in denen mindestens eine rote Markierung ist.

Da hier MM-Kombinationen existieren, ist sowohl **B=0** als auch **B=1** **abgedeckt** und daher kein gesonderter Testfall notwendig, der (Gesamt-)Ausdruck **einmal zu wahr und einmal zu falsch** auswertet.



A1	0	1	0	1	0	1	0	1
A2	0	0	1	1	0	0	1	1
A3	0	0	0	0	1	1	1	1
B	1	0	0	0	1	0	1	0

Bemerkung: Für minimale Mehrfachbedingungsüberdeckung reicht **nicht**, alle Belegungen der Teilbedingungen und der Gesamtbedingung mit 0 und 1 zu testen (sonst würden hier die Testfälle 1,2,3,7 reichen).

Beispiele: Minimale Mehrfachbedingungsüberdeckung

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	C	$A \wedge B \wedge C$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

2 Ausdrücke, 3 Testfälle

2 Ausdrücke, 3 Testfälle

3 Ausdrücke, 4 Testfälle

Minimale Mehrfachbedingungs- überdeckung: Vgl. früheres Beispiel

Testdatum A=3,B=0,C=3:

- A>1 true
- B=0 true
- A=2 false
- C>1 false (da
Auswertung C:=C/A)

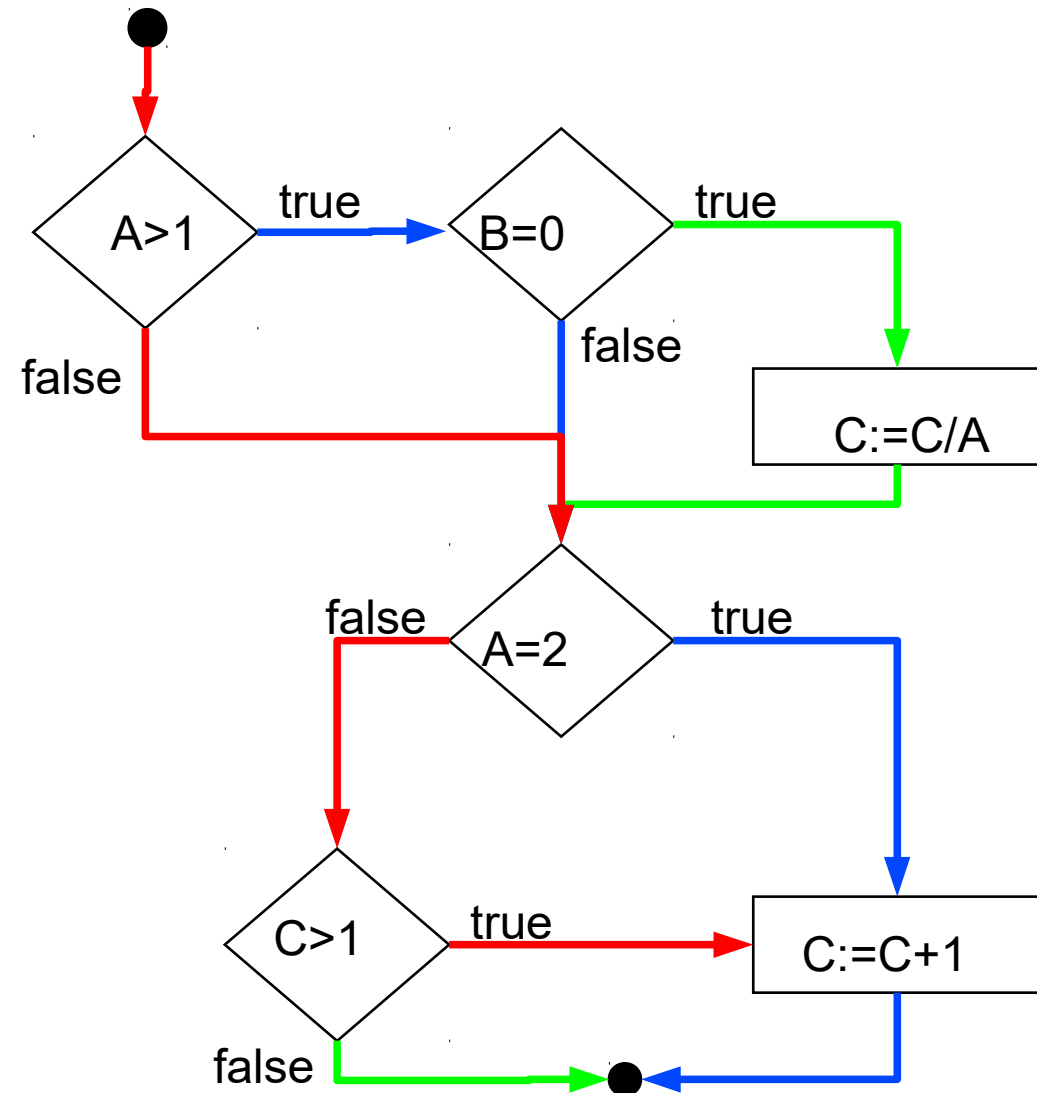
Testdatum A=2,B=1,C=0:

- A>1 true
- B=0 false
- A=2 true
- C>1 false

Testdatum A=1,B=0,C=2:

- A>1 false
- B=0 true
- A=2 false
- C>1 true

→ Alle Zweige geprüft.



In DO-178B (Software Considerations in Airborne Systems and Equipment Certification) geforderte »*Modified Condition/Decision Coverage*« (MC/DC) ist ähnlich der **Minimalen Mehrfachbedingungsüberdeckung**.

- Fordert für jeden atomaren Ausdruck in min. einem Fall zu zeigen, dass er **alleine Gesamtentscheidung beeinflussen** kann.
- **Zwei MM-Kombinationen pro atomaren Ausdruck** notwendig.
- Bei n atomaren und/oder/nicht-verknüpften Ausdrücken: **Testfall-Anzahl** bei MC/DC mindestens $n+1$ und höchstens $2n$ (also nur linear wachsend !).

A1	0	1	0	1	0	1	0	1
A2	0	0	1	1	0	0	1	1
A3	0	0	0	0	1	1	1	1
B	1	0	0	0	1	0	1	0

Diagram illustrating Modified Condition/Decision Coverage (MC/DC) with four atomic expressions (A1, A2, A3, B) and their values (0 or 1) across eight test cases. Red numbers indicate the values that change the overall decision outcome. Arrows show the relationship between the values of the atomic expressions and the overall decision outcome.

Problem: Indirekte Verwendung von zusammengesetzten Booleschen Ausdrücken, z.B.:

- Beispiel: `flag = a || (b && c); if (flag) { ... }`

Problem: Messung der Überdeckung der Teilbedingungen.

- Programmiersprachen und Compiler **verkürzen Auswertung von booleschen Ausdrücken**, sobald Ergebnis feststeht.
- **Beispiel:** AND-Verknüpfung: eine Teilbedingung »false«
→ Gesamtbedingung: »false« (egal welcher zweite Wert).
- Compiler **ändern Reihenfolge der Auswertung** in Abhängigkeit von booleschen Operatoren („lazy evaluation“, s. nächste Folie) aus Effizienzgründen.
- Wegen Verkürzung der Auswertung Überdeckung oft **nicht nachweisbar**.

»**lazy evaluation**«: Bedingungen so lange prüfen, bis Wahrheitswert feststeht.

- Im Programm: `if (A && B) then op1(); op2() ...`
- Im Objectcode: `if (A) then if (B) then op1(); op2() ...`

→ Einfache Bedingungsüberdeckung: **drei Fälle**, sonst B=1 nicht Programm berücksichtigt (Abbruch, sobald A=0 erkannt ist).

- **Gesamtausdruck zu 0 und 1** ausgewertet.
- **Mehrfach-BÜ und MM-BÜ nicht erreichbar.**

Abbruch der
Auswertung

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Einfache (oder atomare) Bedingungsüberdeckung (C_2 -Test, *condition coverage*):

- Atomare Prädikate einmal True und einmal False auswerten.
- Garantiert keine Zweigüberdeckung.

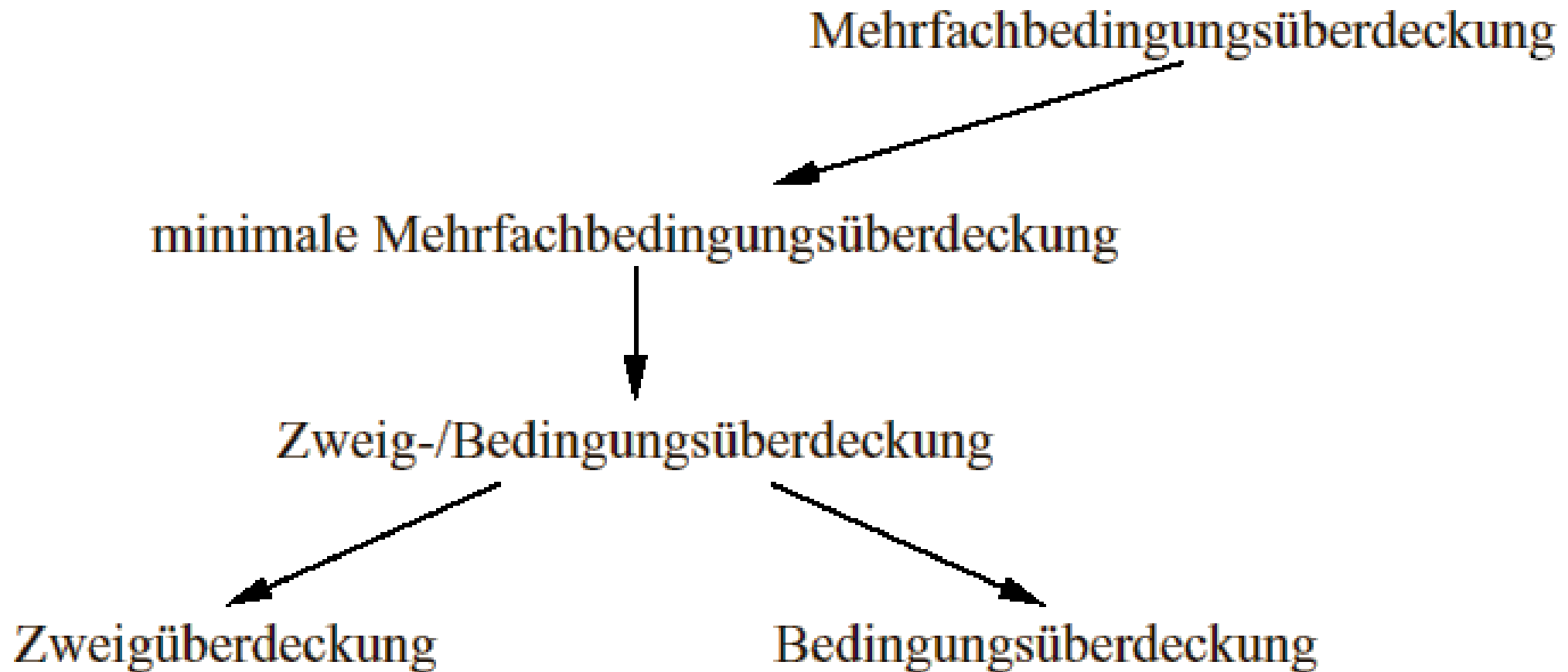
Mehrfach-Bedingungsüberdeckung (C_3 -Test, *multiple condition coverage*):

- Kombinationen atomarer Prädikate betrachten.
- Bei einigermaßen komplizierten Prädikaten nicht mehr handhabbar.

Minimale Mehrfach-Bedingungsüberdeckung ($C_{\text{minMehrfach}}$, *condition determination coverage*):

- Kompromiss zwischen C_2 und C_3 ,
- jedes Prädikat (egal ob atomar oder zusammengesetzt) zu True und False ausgewertet,
- weniger als kombinatorische Abdeckung,
- mehr als beidseitige Auswertung der zusammengesetzten Prädikate.
- impliziert Zweigüberdeckung

K1 → **K2**: jede Testdatenmenge, die K1 erfüllt, erfüllt auch K2.



Anzahl Tests pro Testkriterium (einzelne Entscheidung mit n Termen):

Kriterium	Zahl der erforderlichen Tests pro Entscheidung mit n Termen
Bedingungsüberdeckung	n
Minimale Mehrfachbedingungsüberdeckung (reine AND- bzw. OR-Bedingungen)	$\leq n+1$
Mehrfachbedingungsüberdeckung	2^n

Falls **Anzahl ausgehender Kanten** pro Entscheidungsknoten und **Anzahl Variablen pro Segment** durch Konstante **begrenzt**, dann für Programm mit n Segmenten im schlechtesten Fall:

- **Pfadüberdeckung:** Keine a-priori-Grenze für Anzahl von Tests (nur vorhanden bei vorhandener Grenze an Anzahl Schleifendurchläufe).
- **Strukturierte Pfadüberdeckung:** $O(2^n)$ Tests.
- **Zweig- und Anweisungsüberdeckung:** $O(n)$ Tests.

Aufgedeckte Fehler pro Testkriterium (vgl. [Rie97]):

Kriterium	Gefundene Fehler
Anweisungsüberdeckung	18% bis 41%
Zweigüberdeckung	16% bis 92%
Strukturierte Pfadüberdeckung	zweifache der Zweigüberdeckung (43% statt 21%*)
Pfadüberdeckung	dreifache der Zweigüberdeckung (62% bis 64% statt 21%*)
[minimale (Mehrfach-)] Bedingungsüberdeckung	keine Studien vorhanden

Unterschiedliche Quoten beruhen auf unterschiedlichen Untersuchungsansätzen.

* Angabe 21% beruht auf Messung nach [How78].

Anweisungs- und Entscheidungsüberdeckung für objektorientierte Software nicht ausreichend:

- Komplexität objektorientierter Systemen in **Beziehungen** zwischen **Klassen** bzw. Interaktionen zwischen Objekten verborgen.
 - **Methoden** in Klassen oft von **geringer Komplexität**.
- Anweisungs- und/oder Zweigüberdeckung leicht erreichbar, aber wenig aussagekräftig (trotzdem notwendig, aber nicht hinreichend).
- Datenflussbasiertes Testen (nächster Abschnitt), insbesondere Interprocedural Data Flow Testing

Auswahl eines Kontrollfluss-Testverfahrens

ja					Liegt eine Prozedur, Funktion oder Operation einschließlich Quellcode vor?					nein	
... nur Anweisungen		... nur Anweisungen und atomare, nichtzusammengesetzte Bedingungen		... nur Anweisungen, atomare, nichtzusammengesetzte Bedingungen und Schleifen		... nur Anweisungen und zusammengesetzte Bedingungen		... nur Anweisungen, zusammengesetzte Bedingungen und Schleifen		Black-Box-Testen ^s	
Anwendungsüberdeckungstest		Zweigüberdeckungstest		Genügt es, Schleifen einmal zu wiederholen?		Reicht Überprüfung aller atomaren Bedingungen aus?		Genügt es, Schleifen einmal zu wiederholen?			
				ja nein		ja nein		ja nein			
		boundary interior-Pfadtest		strukturierter Pfadtest		Zweigüberdeckungstest und einfacher Bedingungsüberdeckungstest		minimaler Mehrfach-Bedingungsüberdeckungstest			
								boundary interior-Pfadtest			
								Reicht die Überprüfung aller atomaren Bedingungen aus?			
								ja nein			
								einfacher Bedingungsüberdeckungstest		minimaler Mehrfach-Bedingungsüberdeckungstest	

Werkzeugunterstützung für Kontrollflusstesten (weitere s. Anhang)

Softwarekonstruktion
WS 2016/17



LEHRSTUHL 14
SOFTWARE ENGINEERING

Cobertura (Freeware): <http://cobertura.sourceforge.net>

Features: Prozentuale Überdeckung, zyklomatische Komplexität (McCabe), Zweigüberdeckung, Zeilenüberdeckung.

CodeCover (Freeware): <http://www.codecover.org>

Features: Entscheidungs-, Zweig-, Anweisungs-, Grenze-Inneres-Überdeckung.

Jcoverage (Freeware): <http://jcoverage.sourceforge.net/>

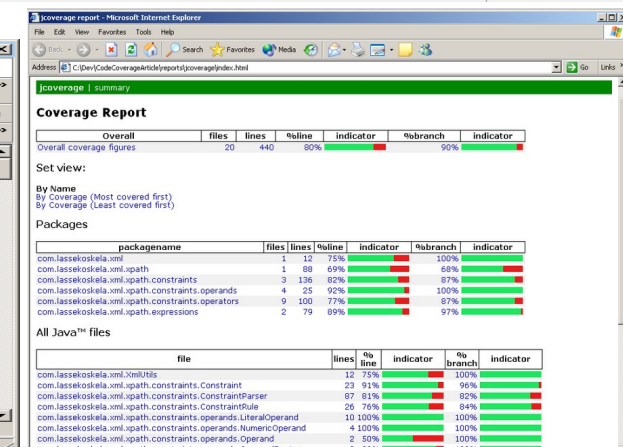
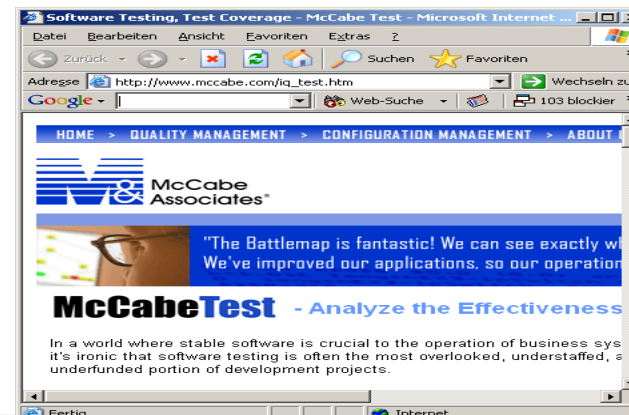
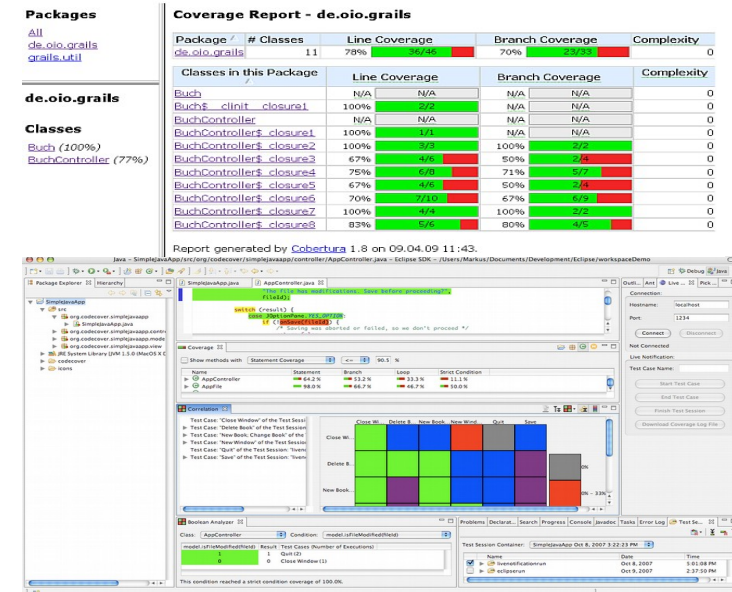
Features: Zeigt wie oft jede Code-Zeile ausgeführt wurde.

McCabe Test coverage:

<http://www.mccabe.com/ip-test.htm>

Features: Prozentuale Überdeckung der Pfade.

Bericht nach allen vorhandenen Tests



96

Beispiel-Programm: Ziffernstringauswertung (1)

Softwarekonstruktion
WS 2016/17

Auswertung eines Ziffernstrings und Ermittlung des dargestellten Wertes:

```
PROCEDURE WerteZiffernfolgeAus (inZiffernString: Ttext) : REAL;
TYPE TWoBinIch = (VordemKomma, NachdemKomma);
  Tziffern = SET OF [0, ..., 9]
CONST Cfehlercode = -1.0;
  Cziffern = Tziffern {0, ..., 9}
VAR Zchn : CHAR
  Wert, Genauigkeit : REAL;
  Position: CARDINAL ;
  WoBinIch: TWoBinIch ;
  Fehlerfrei : Boolean;
BEGIN
  Wert:=0.0;
  Genauigkeit:=1.0;
  WoBinIch:= VordemKomma;
  Fehlerfrei:=TRUE;
  Position:=1;
  ...
  WHILE (Position <=TextVw.Length(inZiffernString))
    AND Fehlerfrei DO
      Zchn:=TextVW.LiesZeichen (inZiffernString,Position);
      IF Zchn IN Cziffern THEN
        IF WoBinIch=NachDemKomma THEN
          Genauigkeit:= Genauigkeit / 10.0;
        END; (* IF *)
        Wert:= 10.0 *Wert + Konvertiere (Zchn)
      ELSIF (Zchn='.') AND (WoBinIch=VorDemKomma) THEN
        WoBinIch:=NachDemKomma
      ELSE
        Fehlerfrei:=FALSE;
      END; (* IF *)
      INC (Position);
    END; (*WHILE*)
  ...
```

...

Beispiel-Programm: Ziffernstringauswertung (2)

Softwarekonstruktion
WS 2016/17

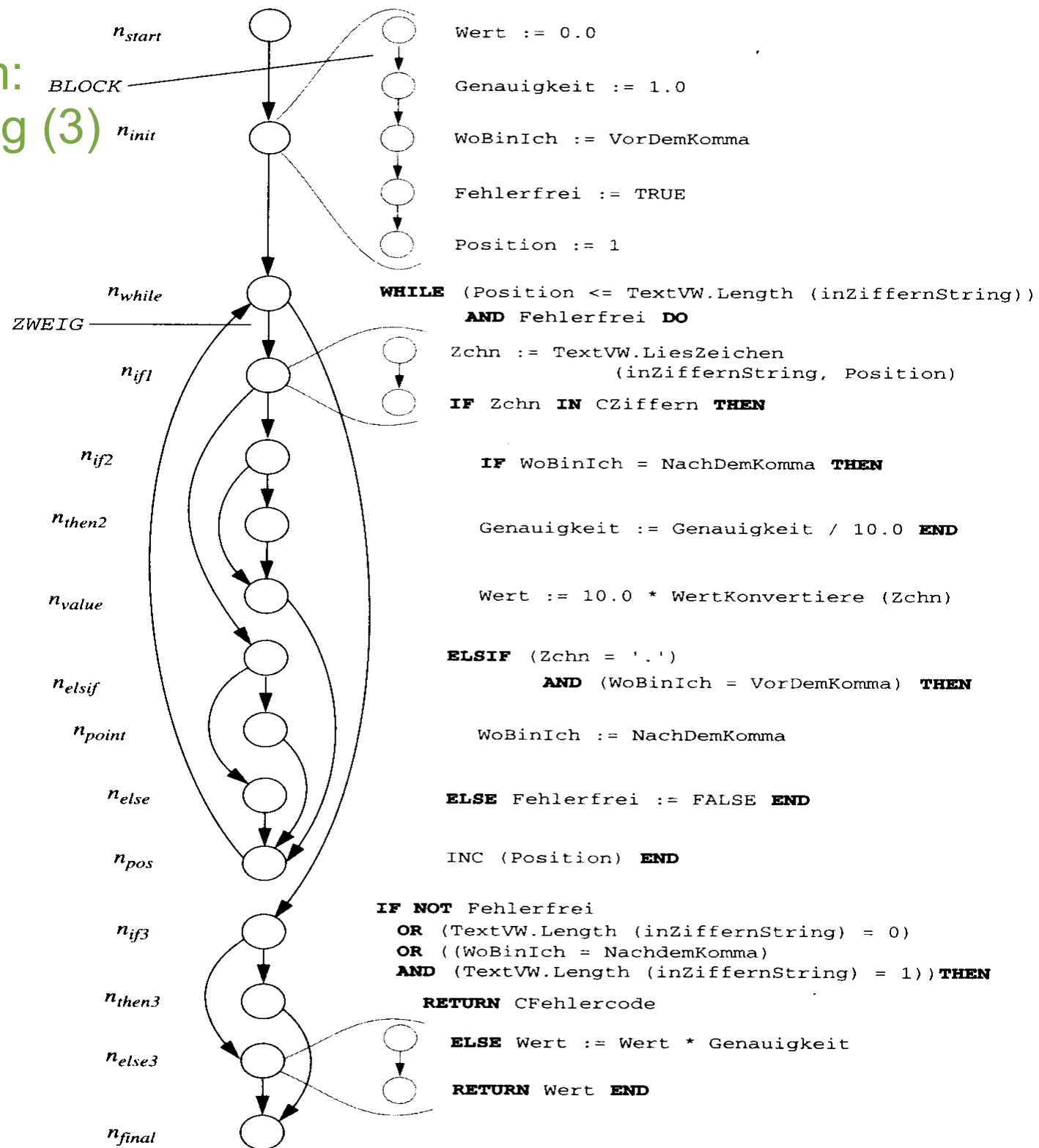
...

```
IF NOT Fehlerfrei
OR (TextVW.Length (inZiffernString) =0);
OR (WoBinIch = NachDemKomma)
AND (TextVwLenght (inZiffernString) = 1)) THEN
    RETURN Cfehlercode
    (* -- ILLEGALE ZEICHENFOLGE -- *)
ELSE
    Wert:=Wert * Genauigkeit;
    RETURN Wert
END (* IF *)
END WerteZiffernFolgeAus;
```

Beispiel-Programm: Ziffernstringauswertung (3)

Zugehöriger Kontrollfluss- graph

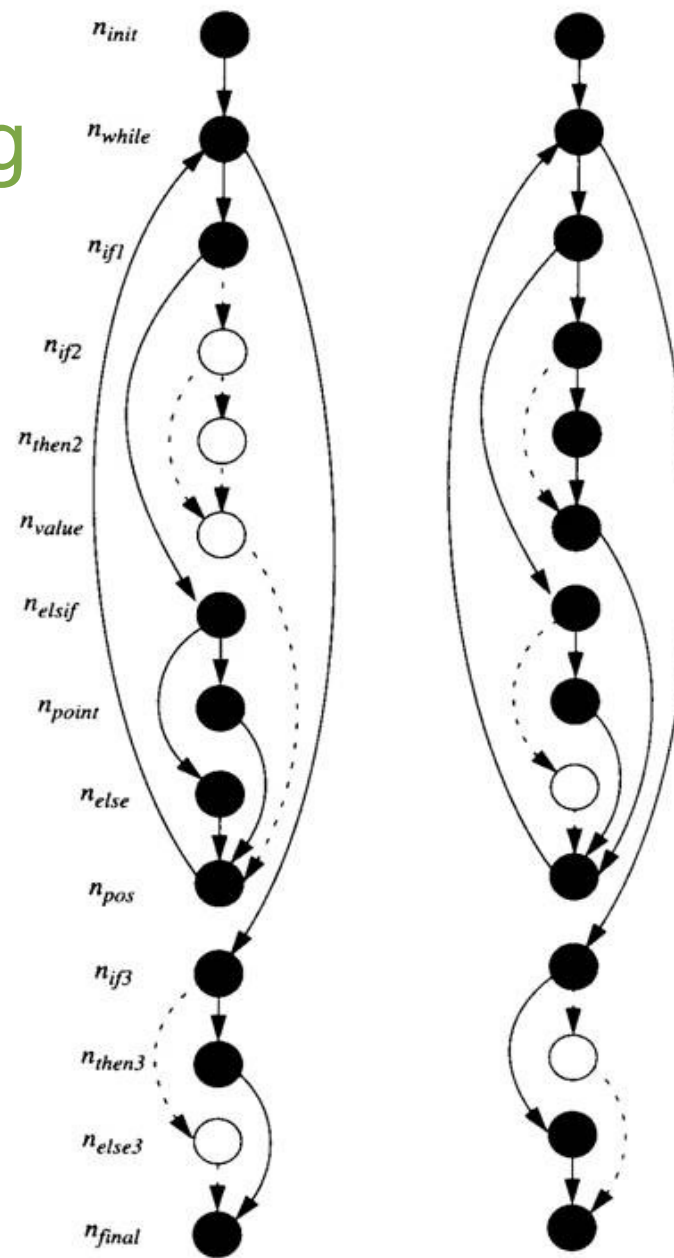
Testfälle für Anweisungs- überdeckung ?



Ziffernauswertung: Anweisungsüberdeckung

Testfälle für
Anweisungs-
überdeckung.

Erfüllt dies
**Zweigüber-
deckung** ?



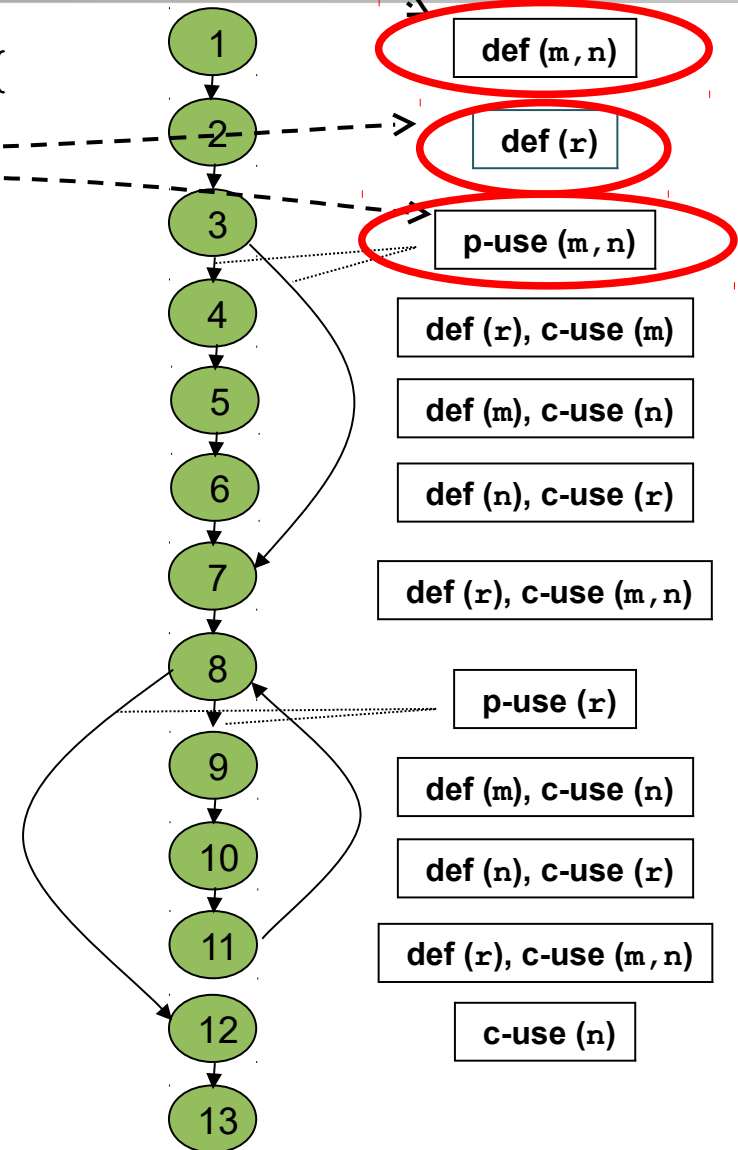
(a) Testfall 1
(Eingabedatum ' . . ')

(b) Testfall 2
(Eingabedatum ' . 9 ')

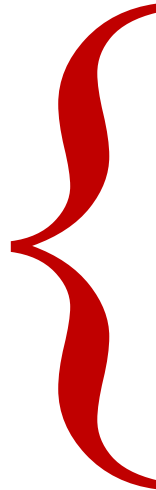


Beispiel ggt: Datenflussgraph

```
1. public int ggt (int m, int n) {  
2.   int r;  
3.   if (n > m) {  
4.     r = m;  
5.     m = n;  
6.     n = r;  
7.   }  
8.   r = m % n;  
9.   while (r != 0) {  
10.    m = n;  
11.    n = r;  
12.    r = m % n;  
13.  }  
14. }
```



2.4 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Test der Bedingungen

Datenflussbasierter Test

Statische Analyse

Welche **Ursachen** kann es haben, wenn **Anweisung falsches Ergebnis liefert**?

Antwort:

-
-

Welche **Ursachen** kann es haben, wenn **Anweisung falsches Ergebnis liefert**?

Antwort:

- Anweisung ist **falsch**.
- Anweisung ist **korrekt**, aber referenzierte Werte werden vorher falsch berechnet.

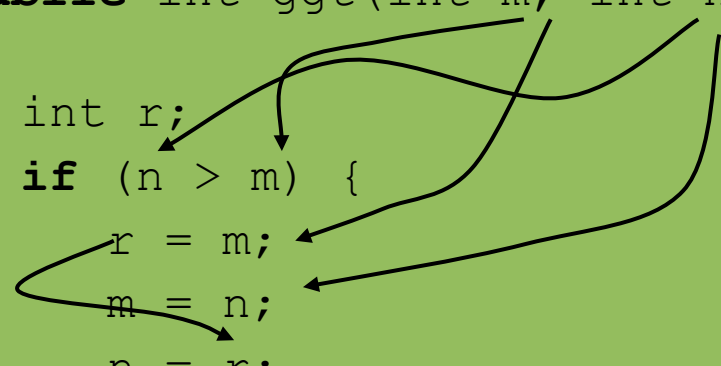
Beispiel:

Anweisung $A=B+C*D$ liefert falsches Ergebnis, wobei Anweisung korrekt ist, aber berechnete Werte vorher falsch berechnet wurden.

Datenflussbasierter Test:

- **White-box-Verfahren.** → Nutzt **Programmstruktur** aus.
- **Dynamisches Verfahren:** Basiert auf Ausführung des Programms.
- **Hypothese: fehlerhafte Datenverwendung !**
- **Idee:** Testen der **Interaktion zwischen Anweisungen**, die Wert einer Variablen berechnen (**definieren**), und Anweisungen, die diesen Variablenwert benutzen (**referenzieren**).
- Testfälle unter Berücksichtigung der Datenverwendung **herleiten**.
- **Vollständigkeit** anhand Datenverwendung beurteilen.

```
1. public int ggt(int m, int n) {  
2.     int r;  
3.     if (n > m) {  
4.         r = m;  
5.         m = n;  
6.         n = r;     ...  
}
```



Ziel (wie beim Kontrollflusstesten): **Möglichst viele Fehler** finden, ohne **vollständige Pfadüberdeckung** erreichen zu müssen (zu aufwendig).

- **Unterscheidung** datenflussorientierter Verfahren:
Alle Interaktionen oder nur Teil davon testen.
(→ Datenflussbasierte Überdeckungsmaße).
- **Definition der Überdeckungsmaße** orientiert sich am Kontrollflussgraphen, erweitert um zusätzliche Informationen.
→ **Datenflussgraph.**

Kontrollflussgraph, bei dem zusätzlich zu jedem Knoten k Mengen $DEF(k)$, $UNDEF(k)$, und $REF(k)$ gehören.

- **$DEF(k)$** : Menge der Variablen x , für welche Anweisungsfolge f , die zum Knoten k gehört, der Variablen x Wert zuweist, der nicht anschließend in f undefiniert wird.
- **$UNDEF(k)$** : Menge der Variablen x , für welche Anweisungsfolge f , die zum Knoten k gehört, die Variablen x in undefinierten Zustand überführt, ohne x anschließend in f neu zu definieren.
 - Beispiele: lokale Iterationsvariablen in Schleifen, `free` (C), `undef` (Perl)
- **$REF(k)$** : Menge der Variablen x , für welche Anweisungsfolge f , die zum Knoten k gehört, die Variable x referenziert, ohne dass x vorher in f undefiniert wird.

- **$X := X + 1$**

sei Anweisung des Knotens k

- $DEF(k) = \{?\}$; $REF(k) = \{?\}$.
- $UNDEF(k) = \{?\}$.

- **$Buffer(bufpos) := c$**

sei Anweisung des Knotens k

- $DEF(k) = \{?\}$.
- $REF(k) = \{?\}$.
- $UNDEF(k) = \{?\}$.

- **$X := A + B$; $Y := C * D$; $B := Z$; $FREE(A)$; $Y := A + Z$**

sei Anweisungsfolge
des Knotens k

- $DEF(k) = \{?\}$.
- $REF(k) = \{?\}$.
- $UNDEF(k) = \{?\}$.

- **$X := X + 1$**

sei Anweisung des Knotens k

- $\text{DEF}(k) = \text{REF}(k) = \{X\}$.
- $\text{UNDEF}(k) = \{\}$.

- **$\text{Buffer}(\text{bufpos}) := c$**

sei Anweisung des Knotens k

- $\text{DEF}(k) = \{\text{Buffer}\}$.
- $\text{REF}(k) = \{\text{bufpos}, c\}$.
- $\text{UNDEF}(k) = \{\}$.

- **$X := A + B; Y := C * D; B := Z; \text{FREE}(A); Y := A + Z$**

sei Anweisungsfolge
des Knotens k

- $\text{DEF}(k) = \{X, Y, B\}$.
- $\text{REF}(k) = \{A, B, C, D, Z\}$.
- $\text{UNDEF}(k) = \{A\}$.

Berücksichtigen:

- Rein **lokale Datenflüsse** vermeiden (wenn intern Referenz auf Definition folgt).
→ Keine Berücksichtigung bei datenflussbezogenen Testkriterien.

Zwei Arten von lokalem Datenfluss:

- Innerhalb eines Blocks von sequentiell aufeinanderfolgenden Anweisungen.
- Bei Zuweisung innerhalb einer Bedingung (z.B. in C).

$X:=A+B; Y:=C*D; B:=Y, \text{FREE}(A); Y:=A+Z$ sei Anweisungsfolge des Knotens

Enthält **rein lokalen Datenfluss** von $Y:=C*D$ nach $B:=Y$, also auf zwei Knoten aufteilen:

- **$X:=A+B; Y:=C*D;$** sei Anweisungsfolge des Knotens k
 - $\text{DEF}(k) = \{X, Y\}.$
 - $\text{REF}(k) = \{A, B, C, D\}.$
 - $\text{UNDEF}(k) = \{\}.$
- **$B:=Y, \text{FREE}(A); Y:=A+Z$** sei Anweisungsfolge des Knotens l
 - $\text{DEF}(l) = \{Y, B\}.$
 - $\text{REF}(l) = \{Y, Z\}.$
 - $\text{UNDEF}(l) = \{A\}.$

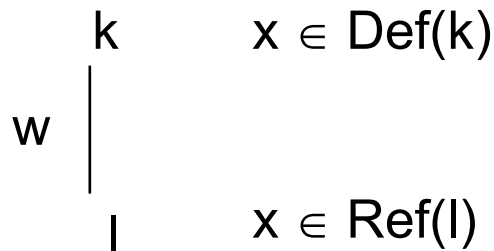
Zuordnung von bedingten Anweisungen zu Knoten so wählen, dass diese Knoten („Entscheidungsknoten“) nur Referenzen von Variablen enthalten (d.h. $\text{DEF}(K) = \text{UNDEF}(K) = \{\}$).

- Beispiel: *If* $((B=C+D))$ aufsplitten in $B=C+D$ und *if* B .

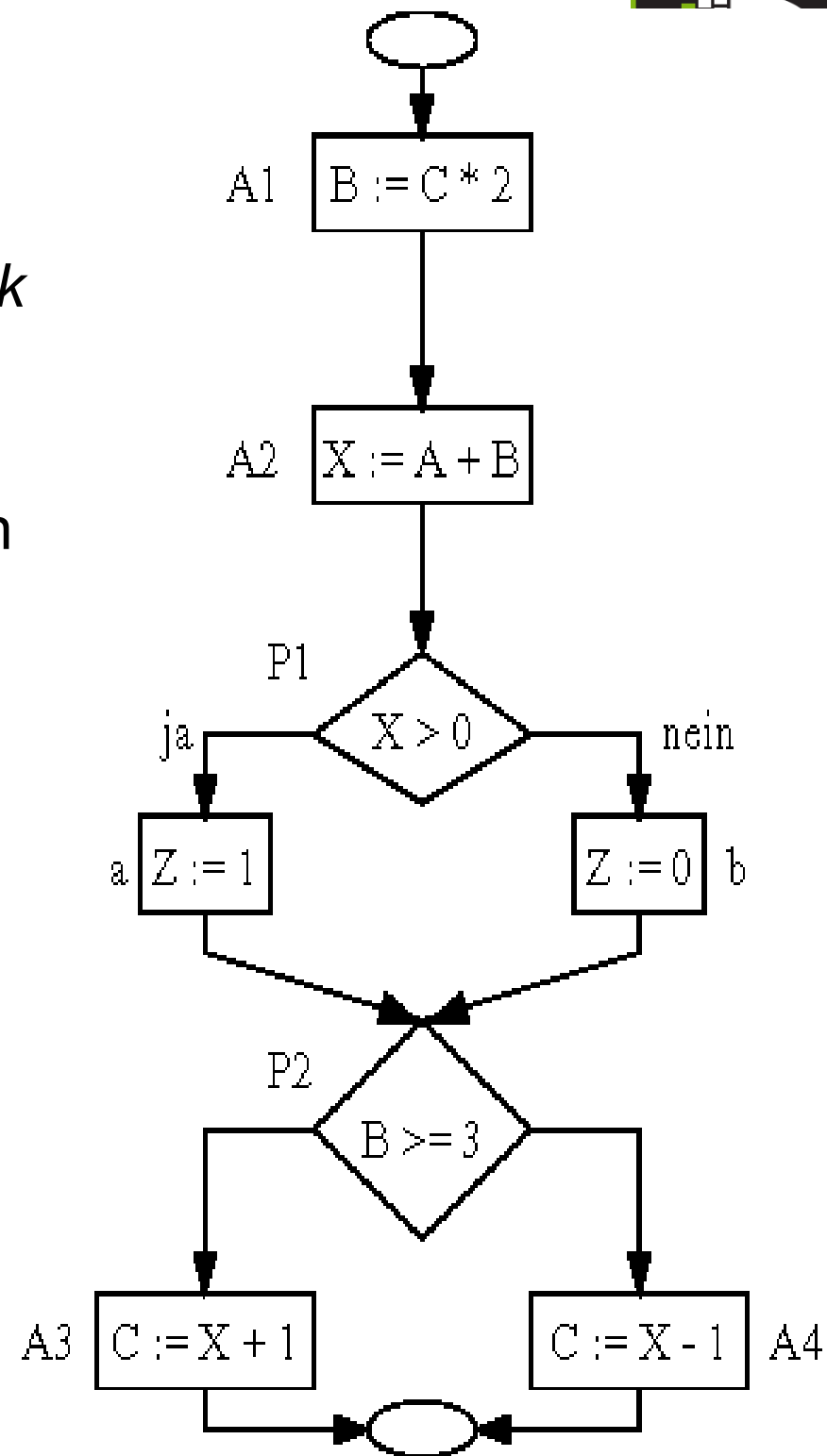
DR-Weg

DR-Weg: „Die Definition von x in Knoten k erreicht eine Referenz von x im Knoten l über Weg w “:

- Gdw. Weg w im Kontrollflussgraph von k nach l führt und Variable x auf Weg **nicht neu definiert** oder **undefiniert** wird.



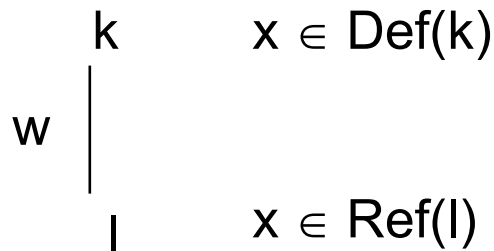
Beispiel für einen solchen Weg im nebenstehenden DFG ?



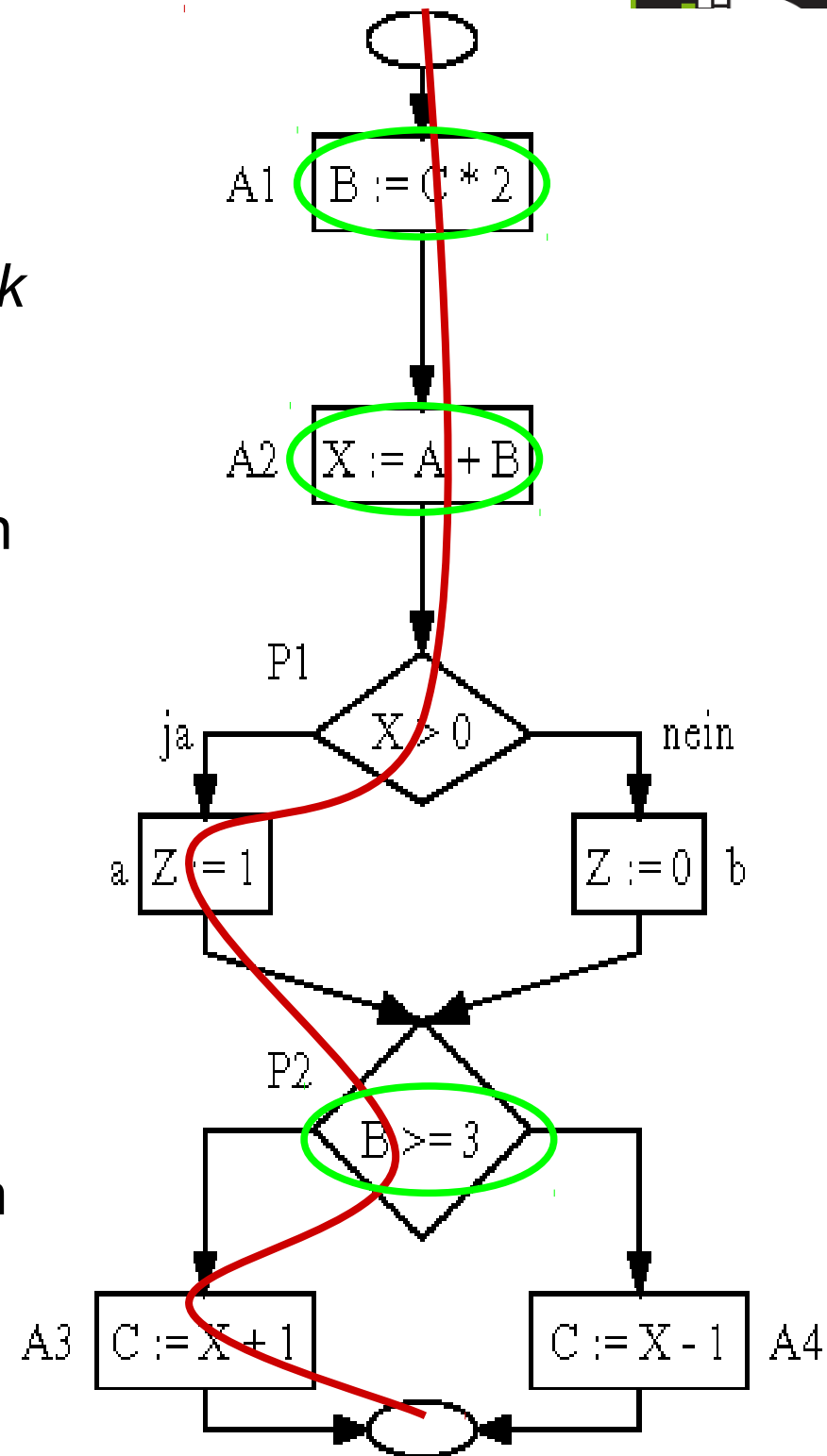
DR-Weg

DR-Weg: „Die Definition von x in Knoten k erreicht eine Referenz von x im Knoten l über Weg w “:

- Gdw. Weg w im Kontrollflussgraph von k nach l führt und Variable x auf Weg **nicht neu definiert** oder **undefiniert** wird.



Beispiel für einen solchen Weg im nebenstehenden DFG: z.B. Definition von B in $A1$, Referenz in $P2$.



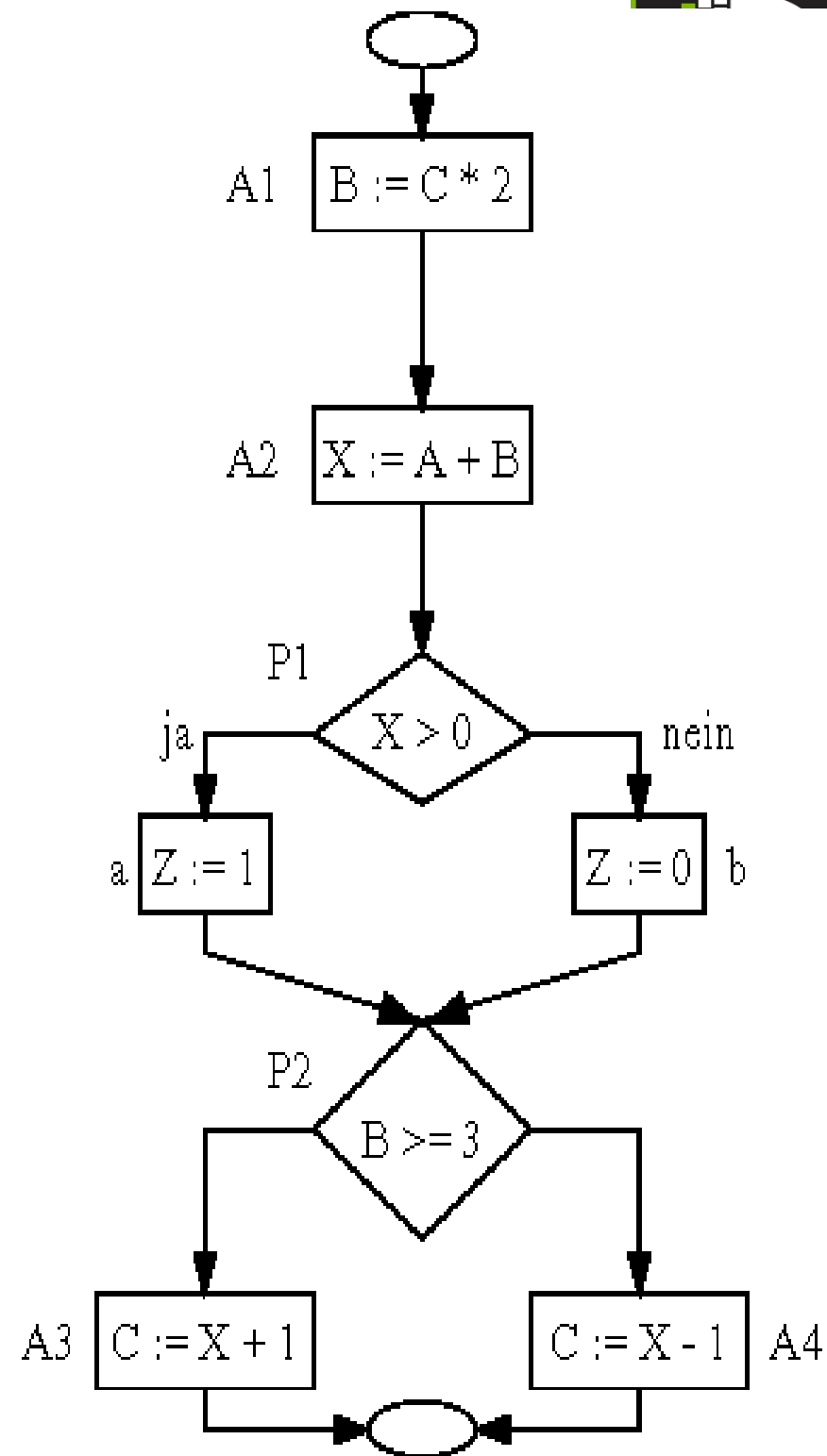
Analog zu den kontrollflussbasierten Kriterien: Kriterien an die Menge der Testpfade definieren.

Kriterium „alle Definitionen“ (all-Defs):

- Resultat jeder Zuweisung (**Definition**) wenigstens einmal **benutzen** („referenzieren“).
- Testfallmenge T erfüllt Kriterium „**alle Definitionen**“ g.d.w.
 - Für jede Variable x und jede Definition von x existiert mindestens ein Weg in $Wege(T)$, auf dem die Definition eine Referenz von x erreicht.

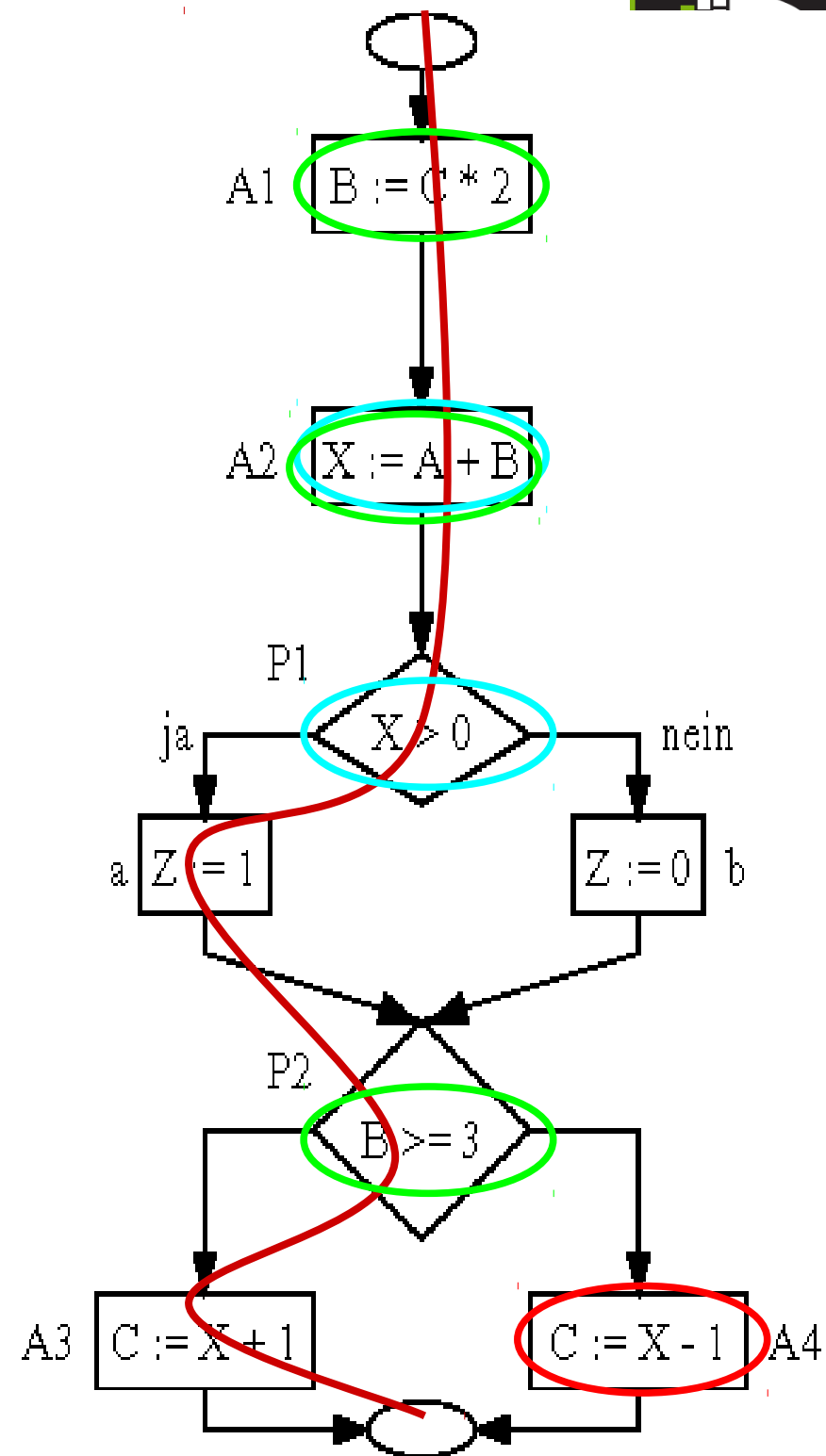
Alle Definitionen: Beispiel

- Welcher Testpfad erfüllt Kriterium „**alle Definitionen**“ (Resultat jeder **Definition** wenigstens einmal **benutzen**) z.B. für Variablen X und B ?
- Werden dabei **alle** (Definition, Referenz)-Paare dieser Variablen getestet ?



Alle Definitionen: Beispiel

- Testpfad $T=(A1, A2, P1, a, P2, A3)$ erfüllt Kriterium „**alle Definitionen**“ für Variablen X und B.
- Aber: nicht **alle** (Definition, Referenz)-Paare einer Variablen werden getestet (z.B. Definition von X in A2, Referenz von X in A4).



Alle DR-Interaktionen

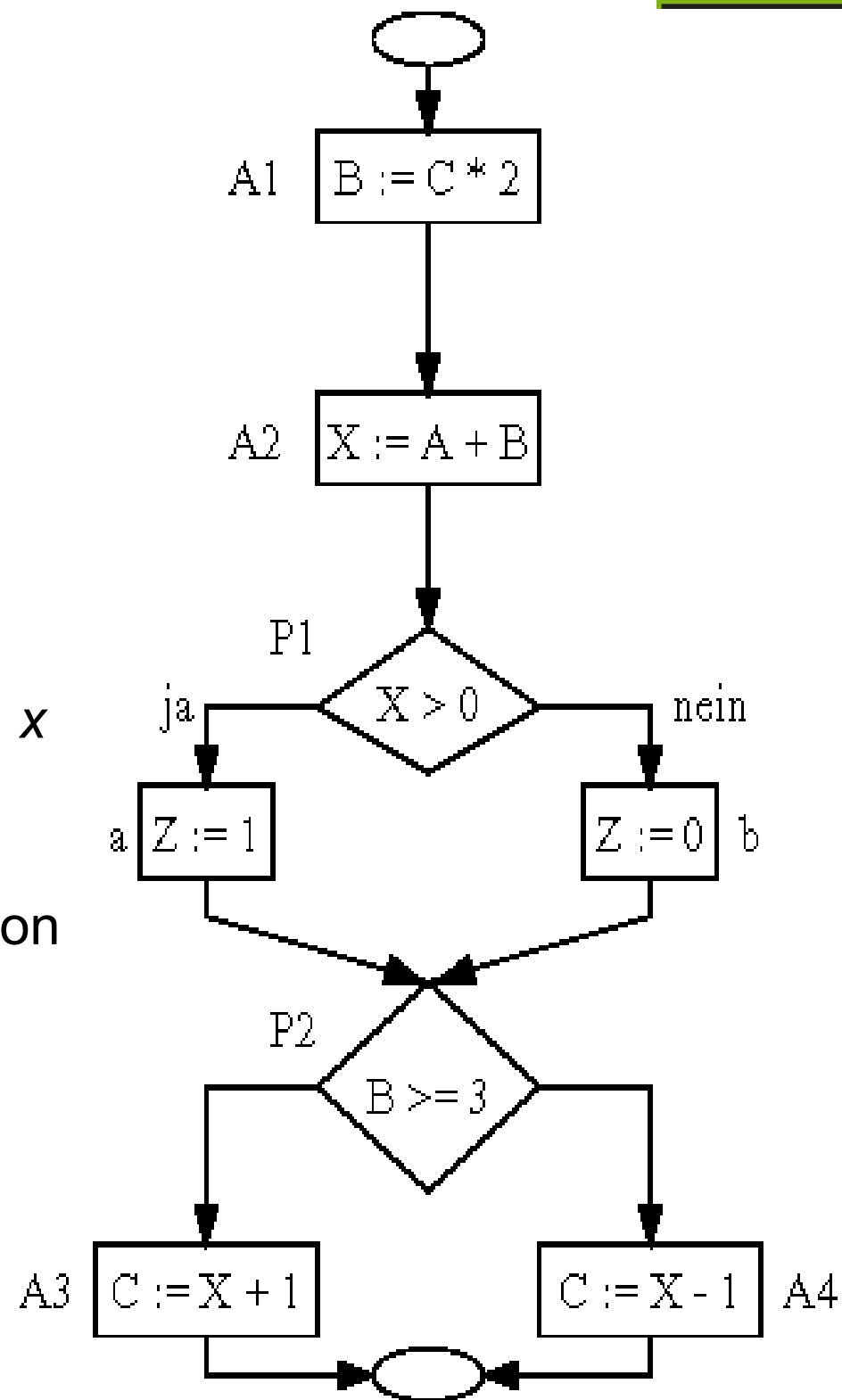
Kriterium „alle DR-Interaktionen“:

Ziel: Alle Paare von Definitionen / Referenzen einer Variablen testen

Testdatenmenge T erfüllt Kriterium „alle DR-Interaktionen“ g.d.w.

- für jede Variable x , jede Definition von x und jede Referenz von x , die davon erreicht wird, **mindestens ein** Weg in $Wege(T)$ existiert, auf dem die Definition eine Referenz von x erreicht.

Beispiel: Welche Testpfadmenge erfüllt dies für X und B ? Testet sie die Konsequenzen der Referenzen?

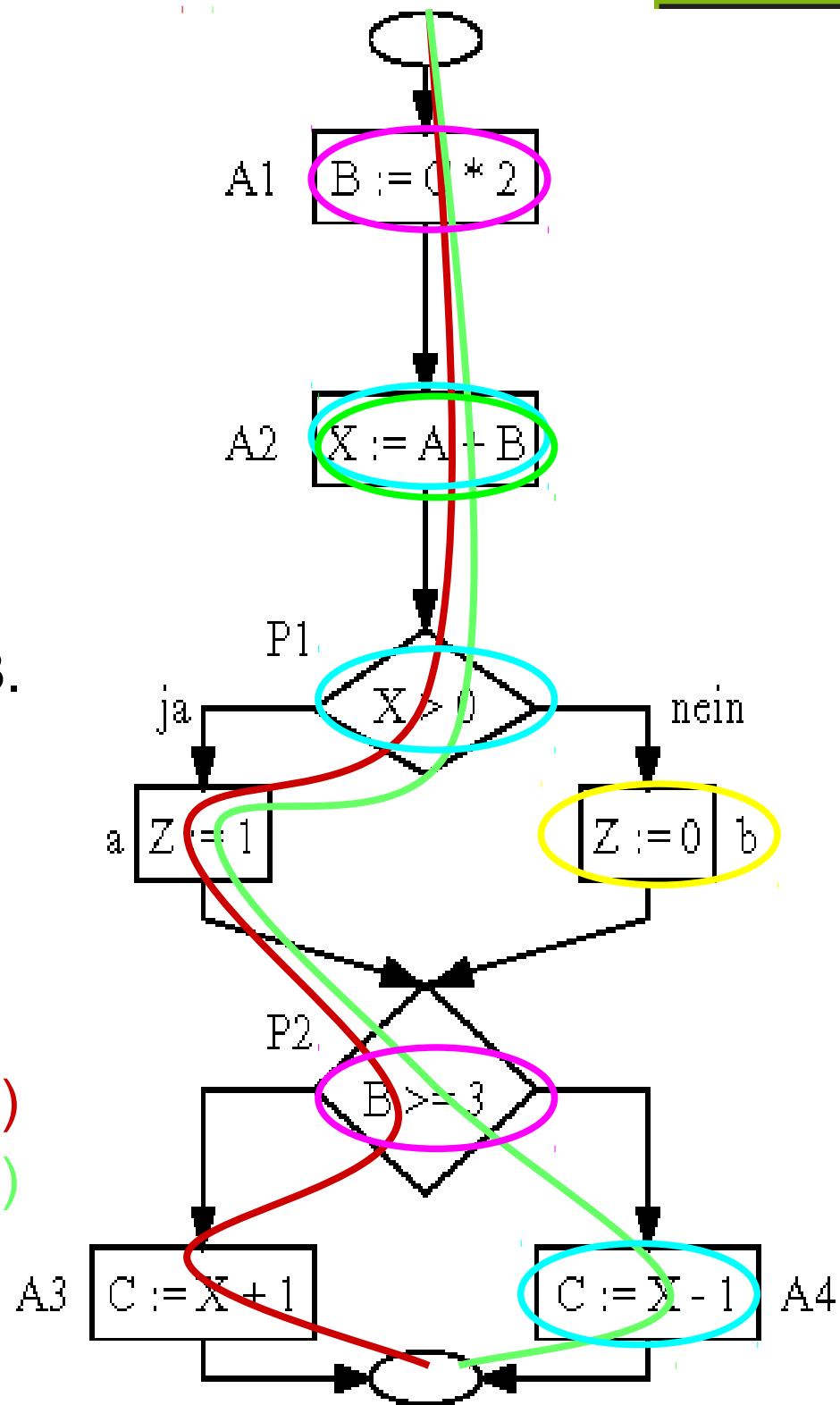


Alle DR-Interaktionen

- T sei Testdatenmenge, die Wege (A1, A2, P1, a, P2, A3) und (A1, A2, P1, a, P2, A4) ausführt.
- T erfüllt Kriterium „**alle DR-Interaktionen**“ für Variablen X und B.
- Aber: Entscheidungskante b (= Konsequenz aus Referenz von X) nicht ausgeführt.

Weg (A1, A2, P1, a, P2, A3)

Weg (A1, A2, P1, a, P2, A4)



Motivation: Für Referenz einer Variablen im Entscheidungsknoten:
Ausgang der Entscheidung wichtig.

Kriterium „alle Referenzen“:

- Ziel: **Alle ausgehenden Kanten** eines **Entscheidungsknotens** berücksichtigen.

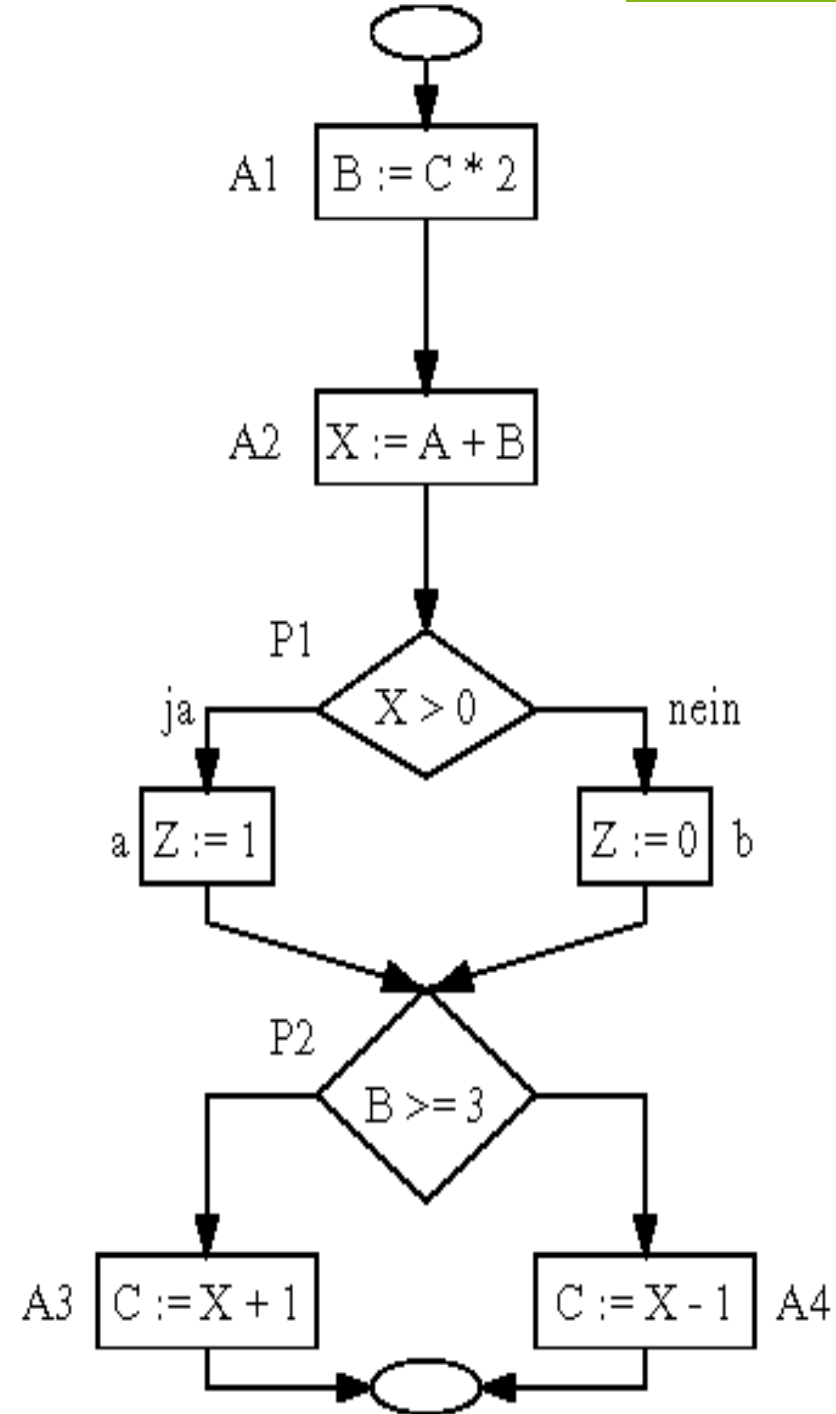
Testdatenmenge T erfüllt Kriterium „**alle Referenzen**“ g.d.w.:

- Für jede Variable x , jede Definition von x im Knoten k , jede Referenz von x im Knoten l , die von Definition in k erreicht wird, und für jeden Nachfolgeknoten m von l die Wegemenge $Wege(T)$ **mindestens ein** Wegstück $u^*m=\{k,...,l,m\}$ enthält, wobei die Definition von x in k die Referenz von x in l über Weg u erreicht.

Hierbei: $u^*m=\{k,...,l,m\}$ ist die Knotenfolge mit $u=\{k,...,l\}$ gefolgt von Knoten m .

Alle Referenzen: Beispiel

- **Beispiel:** Was sind Nachfolgeknoten von Referenzen von X und B ? Welche Testpfadmenge erfüllt „**alle Referenzen**“ (alle ausgehenden Kanten eines Entscheidungsknotens berücksichtigen) für X und B ?
- Testet sie alle Wege ?



Alle Referenzen: Beispiel

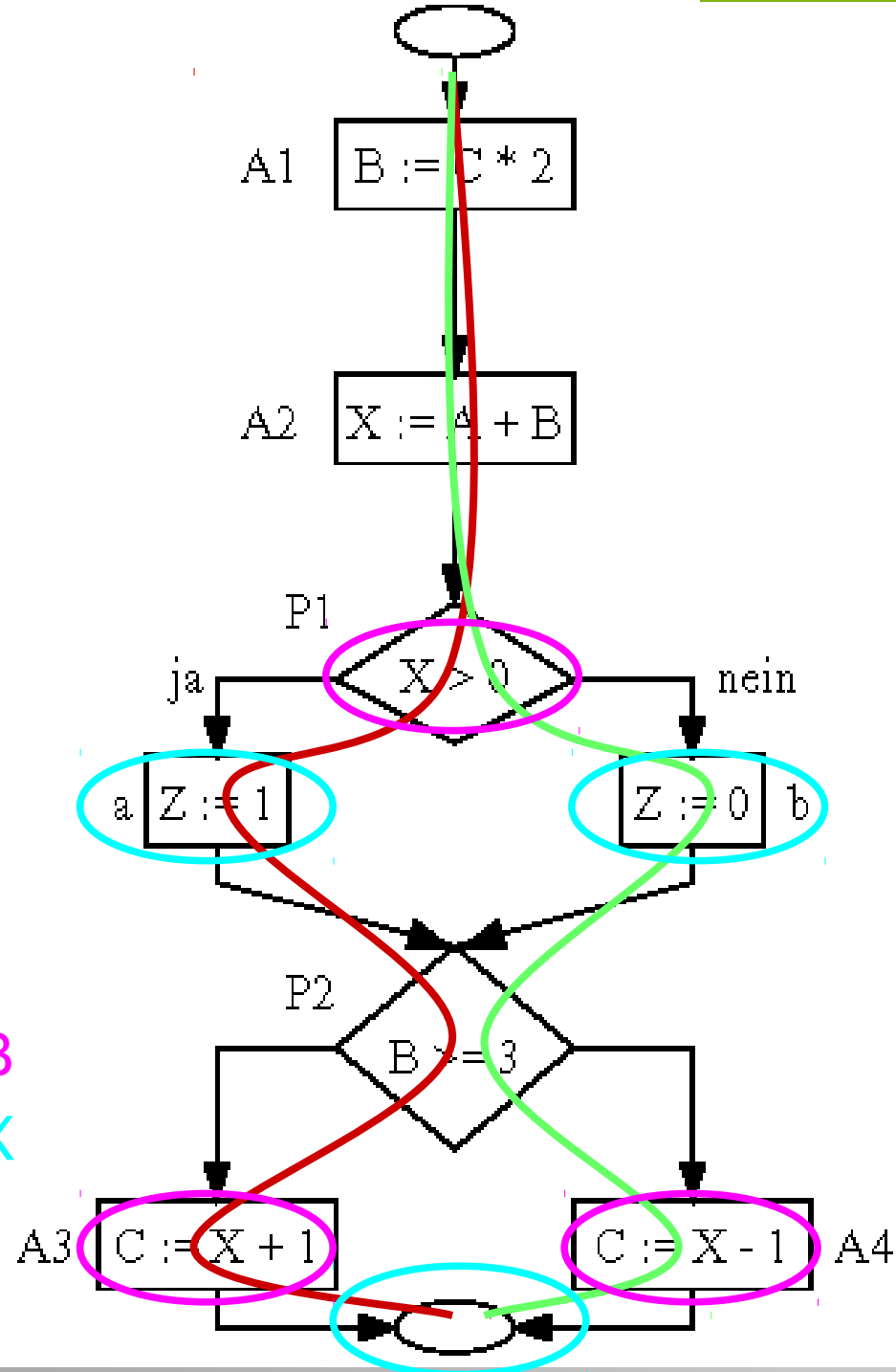
- T sei Testdatenmenge, die Wege (A1, A2, P1, a, P2, A3) und (A1, A2, P1, b, P2, A4) ausführt.
- T erfüllt Kriterium „**alle Referenzen**“ für Variablen X und B.
- **Aber:** Wege (A1, A2, P1, a, P2, A4) und (A1, A2, P1, b, P2, A3) nicht ausgeführt.

Nachfolgeknoten von Referenzen von B

Nachfolgeknoten von Referenzen von X

Weg (A1, A2, P1, a, P2, A3)

Weg (A1, A2, P1, b, P2, A4)



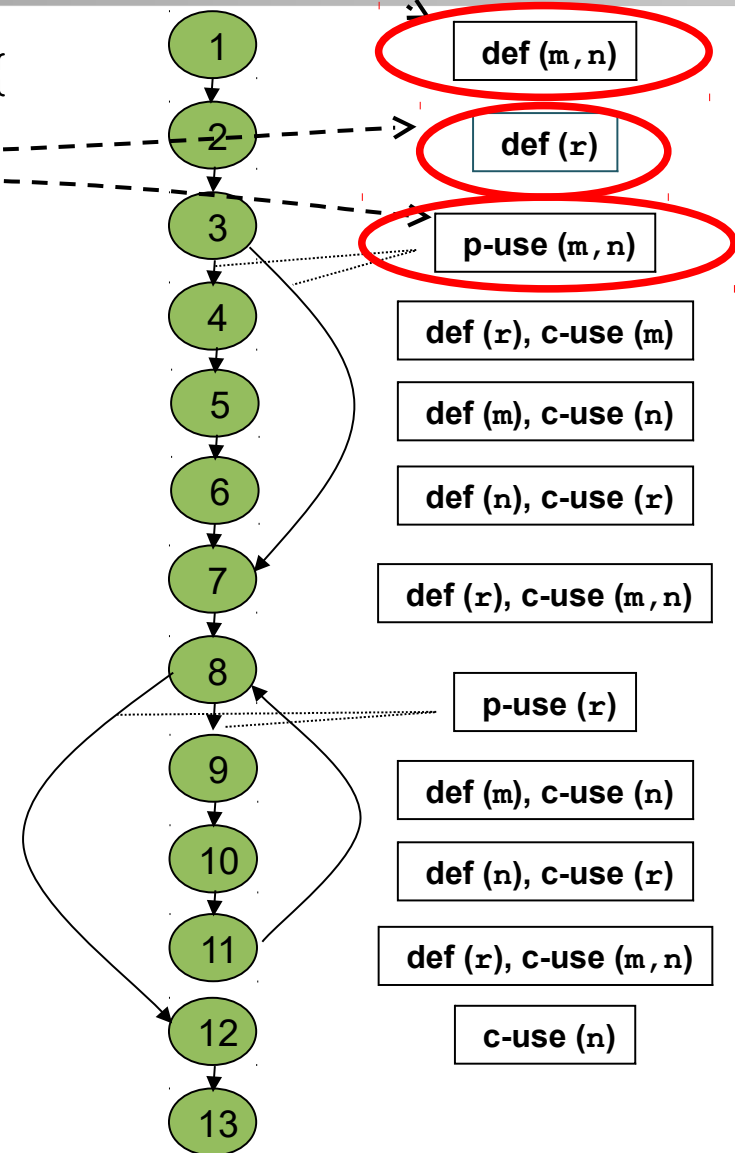
- **Einfache Datenflusskriterien wie „Alle DR-Interaktionen“ und „alle Referenzen“: Kriterien zum Testen aller Paare von Definitionen und Referenzen.**
 - Jeweils **auf einem Weg** von Definition zur Referenz.
- Ausreichend unter Gesichtspunkt des Datenflusses.
 - Zwischen Definition und Referenz **keine Änderung der Variablen.**
- Etwas feinkörniger: zwischen **Entscheidungs-** und **Berechnungs-Referenzen** unterscheiden (nächste Folie).

Mögliche Variablen- / Objektverwendung:

- Wertzuweisung, zustandsverändernd (**Definition / Definitional use**):
 - z.B. `r = m` oder `r = 5 : def(r)`
- Benutzung in Ausdrücken, zustandserhaltend: (**Berechnungs-Referenz / Computational use**):
 - z.B. `r = m mod n` oder `r = op1(m, n) : c-use(m, n)` und `def(r)`.
- Benutzung in Bedingungen, zustandserhaltend (**Entscheidungs-Referenz / Predicative use**):
 - z.B. `while (r != 0)` oder `if (r != 0) : p-use(r)`.

Beispiel ggt: Datenflussgraph

```
1. public int ggt (int m, int n) {  
2.   int r;  
3.   if (n > m) {  
4.       r = m;  
5.       m = n;  
6.       n = r;  
7.   }  
8.   while (r != 0) {  
9.       m = n;  
10.      n = r;  
11.      r = m % n;  
12.   }  
13. return n;  
14. }
```



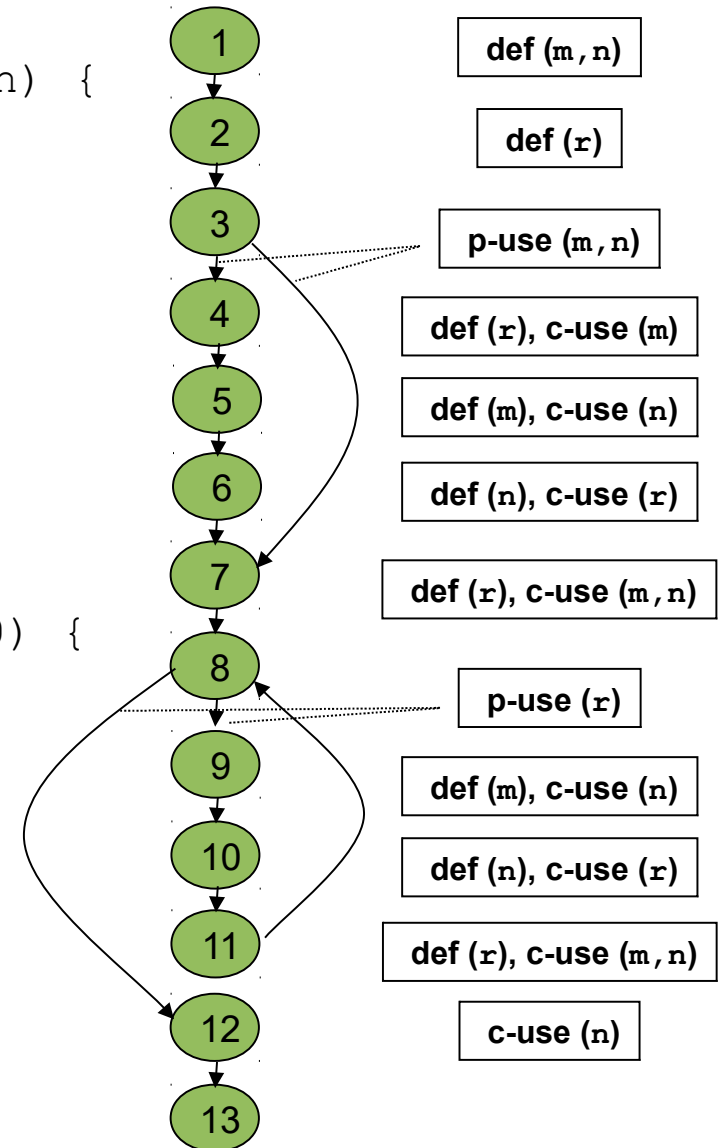
Beispiel ggt: Alle Definitionen

All-defs: Jede Definition min. einmal ohne dazwischen liegendes erneutes def in Referenz (c-use oder p-use) verwenden.

[Bem.: Egal ob c-use oder p-use, also konsistent mit Def. auf F. 114.]

Gibt es eine Testmenge, die All-defs erfüllt?

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
   }
7.   r = m % n;
8.   while (r != 0) {
9.     m = n;
10.    n = r;
11.    r = m % n;
   }
12.  return n;
13. }
```



Pfad (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13) erfüllt **All-defs** mit Ausnahme von „2: def(r)“ (nicht ohne dazwischen liegendes erneutes def verwendet):

1, def m: p-use 3-4

1, def n: p-use 3-4

4, def r: c-use 6

5, def m: c-use 7

6, def n: c-use 7

7, def r: p-use 8-9

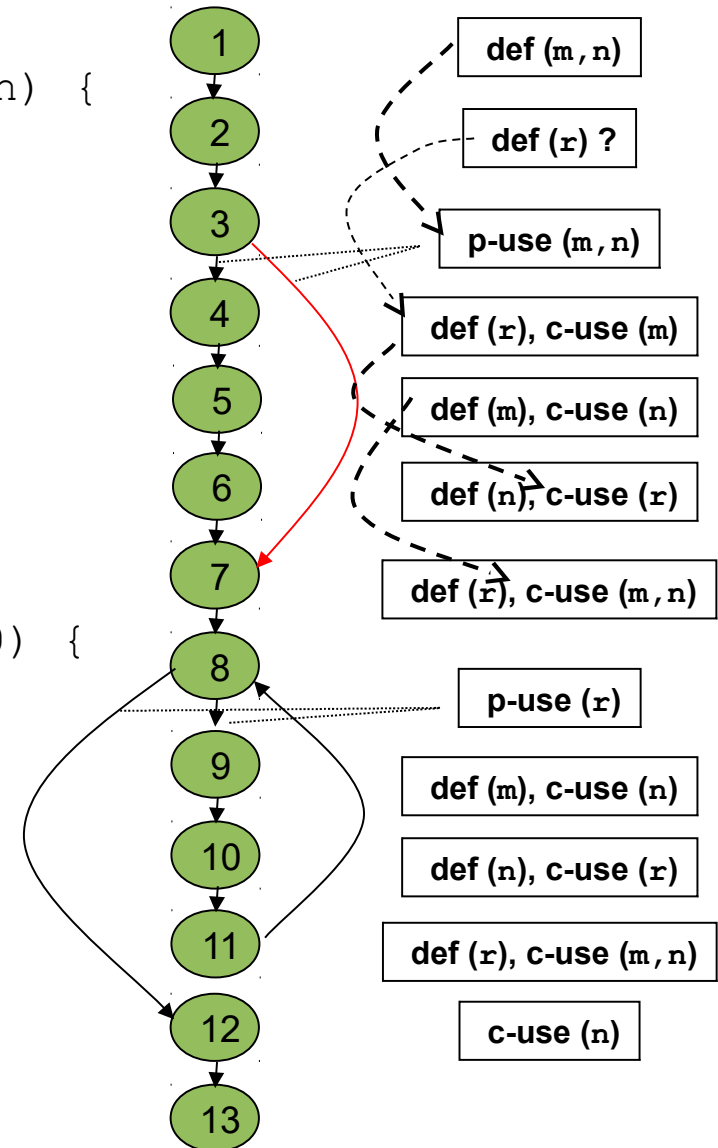
9, def m: c-use 11

10, def n: c-use 11

11, def r: p-use 8-9

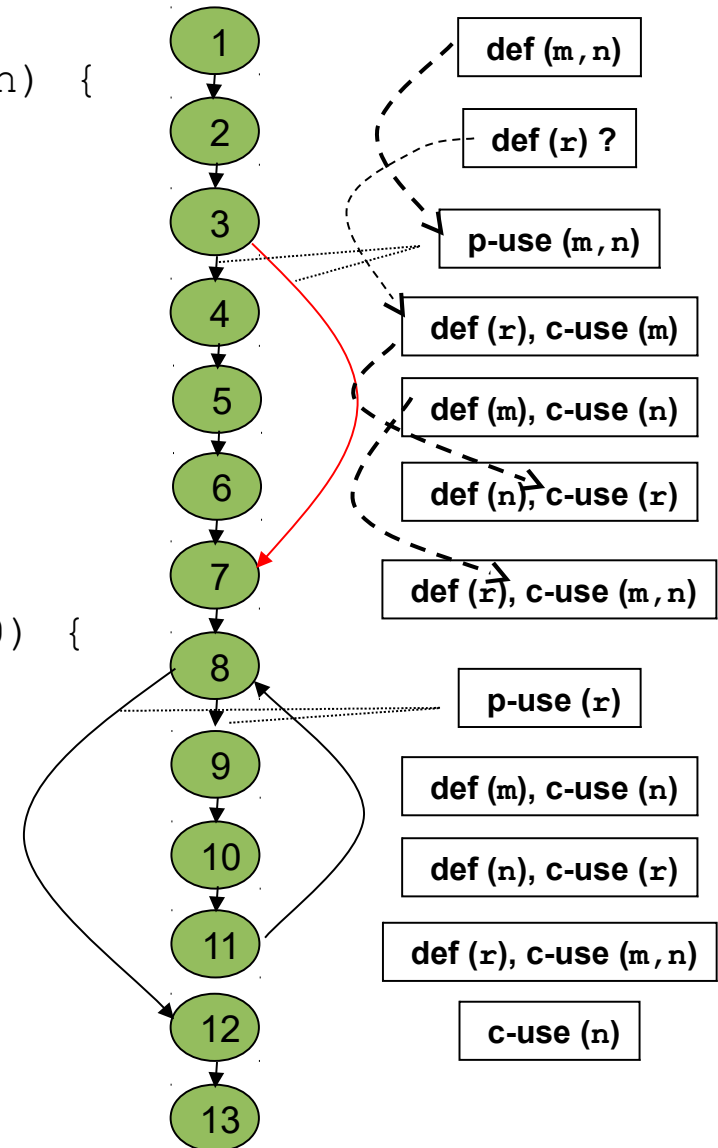
NB: „3-4“ zeigt an, welcher Zweig ausgehen von 3 in diesem Pfad genommen wurde (für All-defs aber nicht relevant.)

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
   }
7.   r = m % n;
8.   while (r != 0) {
9.     m = n;
10.    n = r;
11.    r = m % n;
   }
12.  return n;
13. }
```



Erfüllt Pfad (1, 2-3, 4-6, 7, 8, 9-11, 8, 12, 13)
auch **Alle DR-Interaktionen**
[= jedes Paar def/ref (ohne dazwischen liegendes erneutes def) ausführen] ?

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
7.   }
8.   r = m % n;
9.   while (r != 0) {
10.    m = n;
11.    n = r;
12.    r = m % n;
13.  }
14.  return n;
15. }
```



Beispiel ggt: alle DR-Interaktionen

Alle DR-Interaktionen: jedes Paar def/ref (ohne dazwischenliegendes erneutes def) ausführen:

1, def m: p-use 3-4

1, def m: c-use 4

1, def m: c-use 7

→ **brauche else-Zweig 3-7 !**

1, def n: p-use 3-4

1, def n: c-use 5

1, def n: c-use 7 (**braucht 3-7 !**)

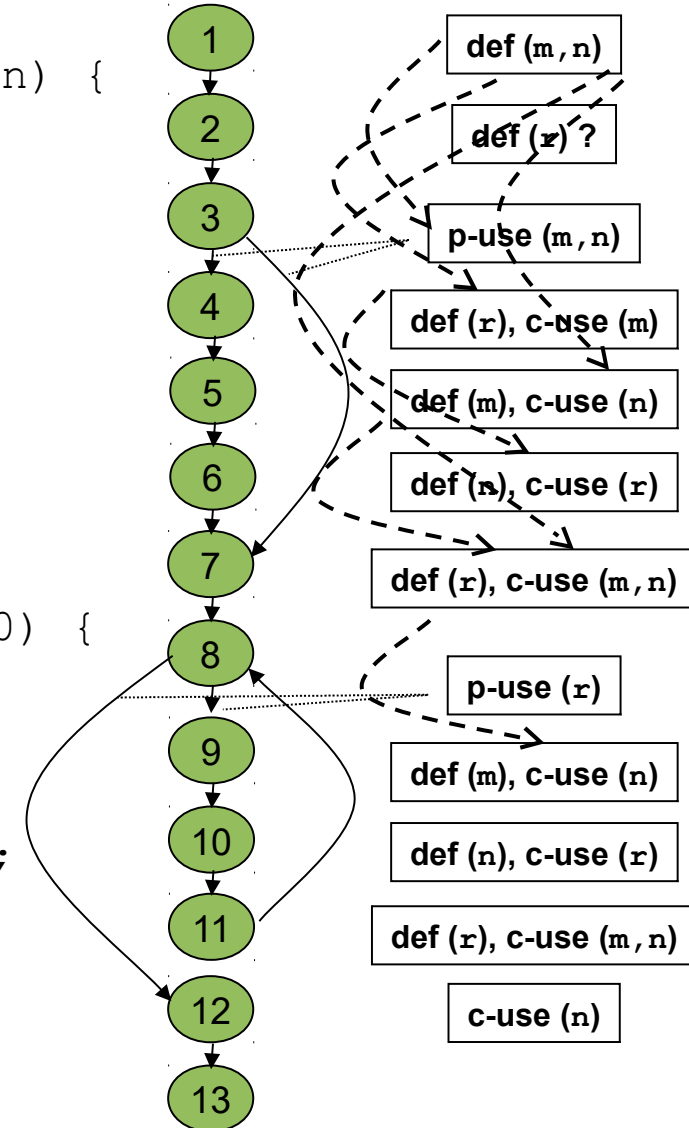
4, def r: c-use 6

.....

→ Pfad reicht nicht, brauche weiteren Pfad für else-Zweig 3-7.

NB: Brauche keinen Pfad, der die Schleife 8-11 null mal ausführt, weil für „12: c-use(n)“ das vorhergehende „1/6: def(n)“ dann bereits durch „7: c-use(n)“ abgedeckt wird

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
7.   }
8.   r = m % n;
9.   while (r != 0) {
10.    m = n;
11.    n = r;
12.    r = m % n;
13.   }
14.   return n;
15. }
```

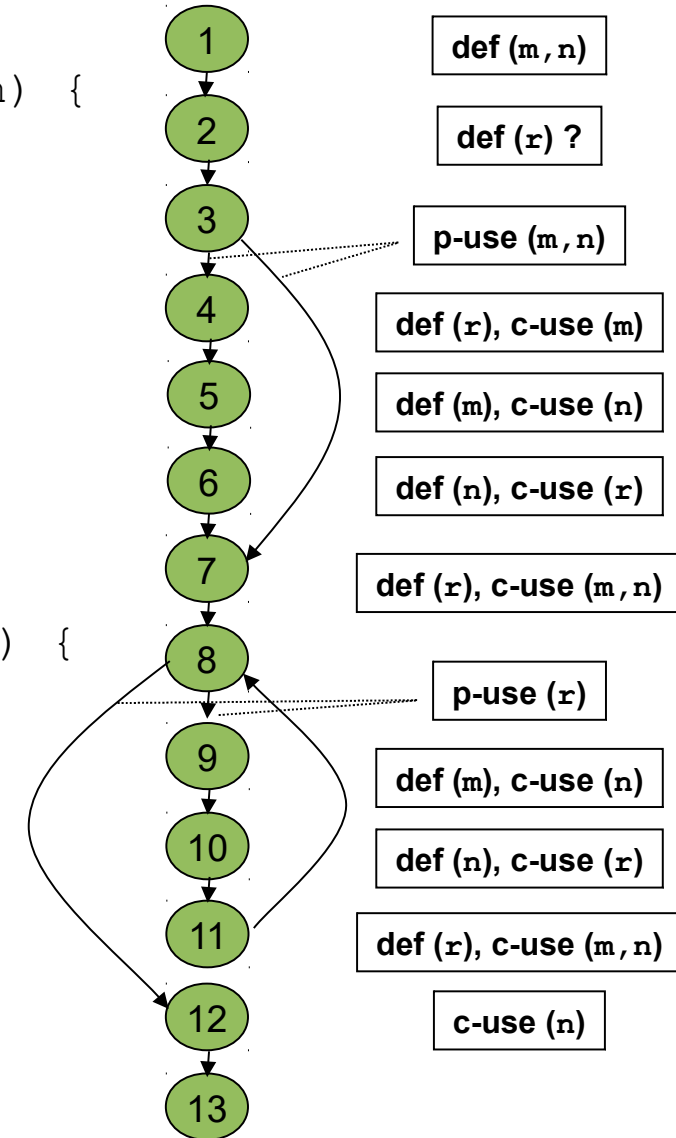


Erfüllt die Pfadmenge
von der vorherigen Folie
Alle-Referenzen ?

Reicht schon der
einzelne Testfall von
davor ?

Was sind jeweils die
Nachfolgeknoten ?

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
   }
7.   r = m % n;
8.   while (r != 0) {
9.     m = n;
10.    n = r;
11.    r = m % n;
   }
12.  return n;
13. }
```



Alle B-Referenzen (c-uses):

1, def m: c-use 4-5

1, def m: c-use 7-8

→ brauche else-Zweig 3-7 !

1, def n: c-use 5-6

1, def n: c-use 7-8 (else!)

4, def r: c-use 6-7

.....

Alle E-Referenzen (p-uses):

1, def m p-use 3-4

1, def m p-use 3-7

→ brauche else-Zweig 3-7 !

.....

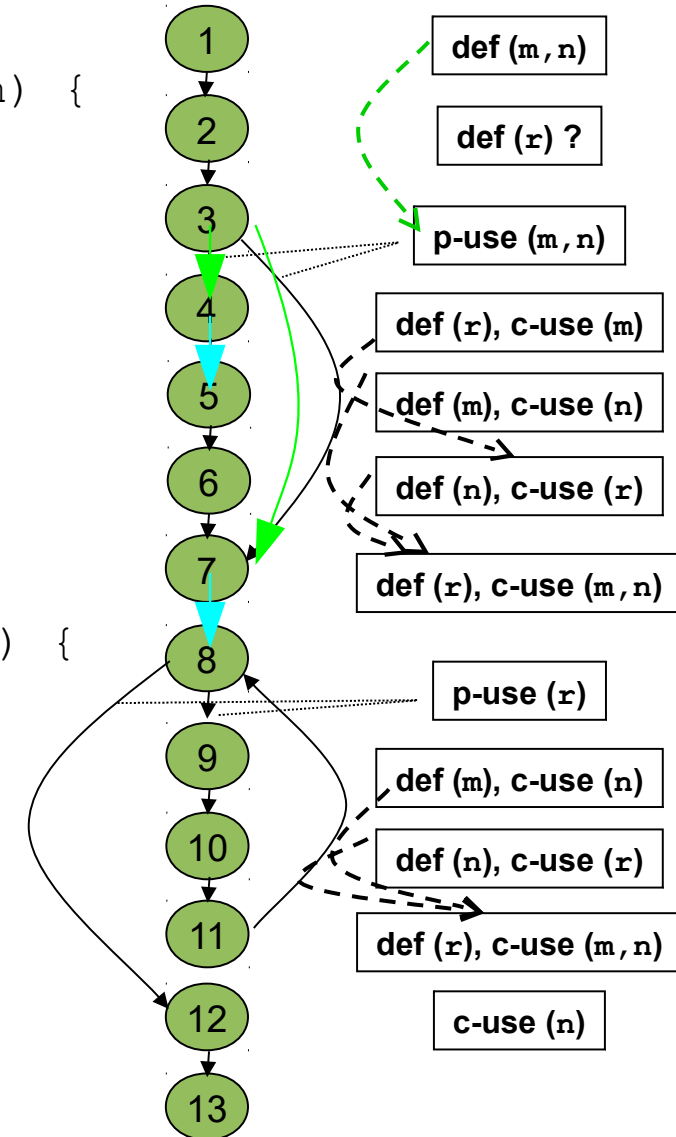
→ Brauche beide Testfälle.

Nachfolgeknoten von p-uses von m.

Nachfolgeknoten von c-uses von m.

```

1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
   }
7.   r = m % n;
8.   while (r != 0) {
9.     m = n;
10.    n = r;
11.    r = m % n;
   }
12.  return n;
13. }
    
```



Beispiel ggt: Alle k-DR-Interaktionen

Alle k-DR-Interaktionen:
Verkettung von DR-Paaren.
Beispiel für k=3:

(1, m, 4, r, 6):

- m wird in 1 definiert, in 4 referenziert,
- r wird in 4 definiert, in 6 referenziert

→ damit hängt 6 von 1 ab!

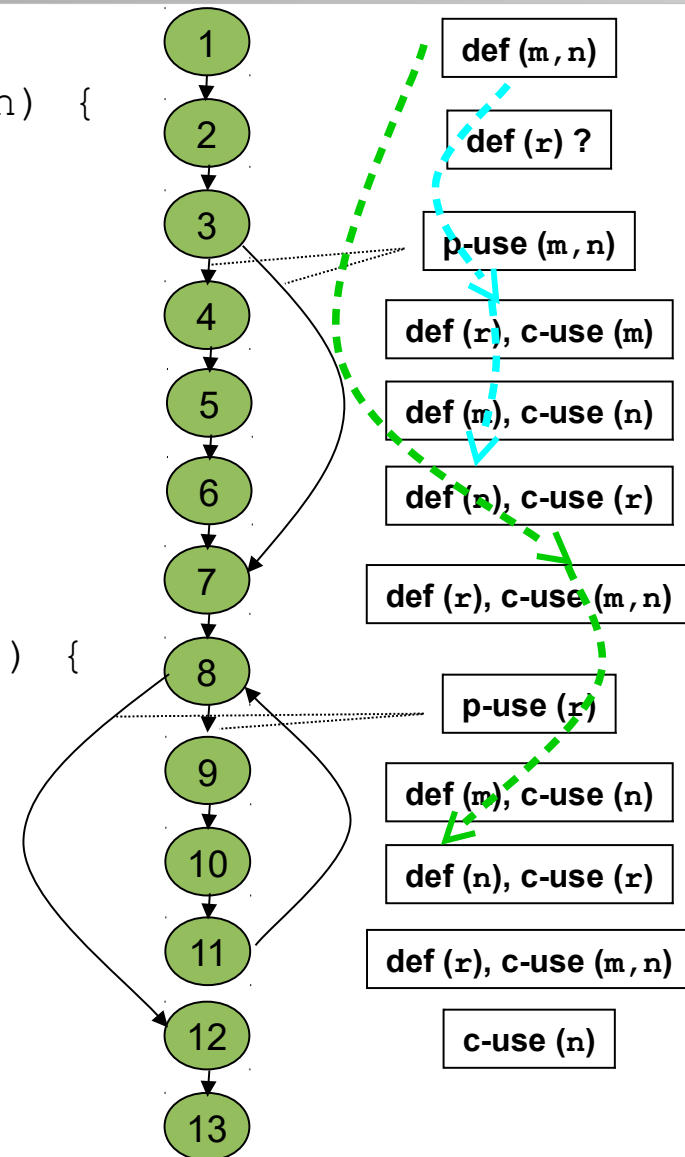
Analog für (1, m, 7, r, 10).

→ Teste Wegstücke

(1,2,3,4,5,6) und

(1,2,3,7,8,9, 10).

```
1. public int ggt
   (int m, int n) {
2.   int r;
3.   if (n > m) {
4.     r = m;
5.     m = n;
6.     n = r;
   }
7.   r = m % n;
8.   while (r != 0) {
9.     m = n;
10.    n = r;
11.    r = m % n;
   }
12.  return n;
13. }
```



Beispiel ggt: Kontextüberdeckung

Beispiel für Kontextüberdeckung:

In 7 wird r definiert durch referenzierte Variablen m und n.

„Definitionskontext“:

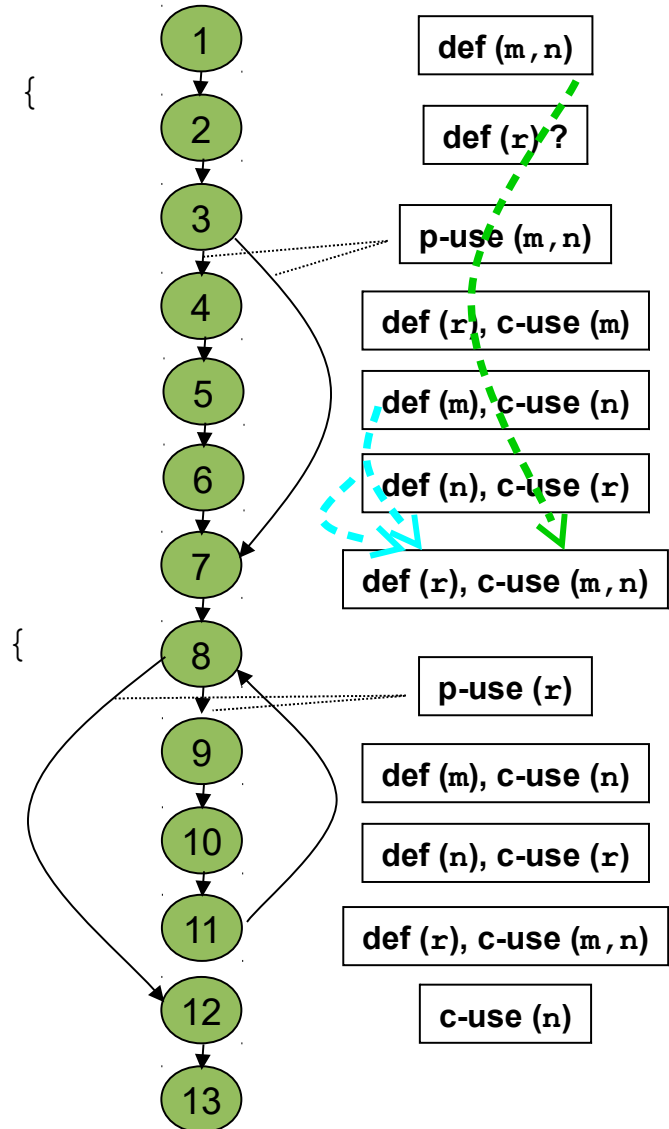
Knoten, in denen m und n vorher definiert. Hier zwei Fälle:

DK1= {(1,m),(1,n)}

DK2= {(5,m),(6,n)}

→ teste Wegstücke
(1,2,7) und (5,6,7).

```
1. public int ggt
    (int m, int n) {
2.     int r;
3.     if (n > m) {
4.         r = m;
5.         m = n;
6.         n = r;
    }
7.     r = m % n;
8.     while (r != 0) {
9.         m = n;
10.        n = r;
11.        r = m % n;
    }
12.    return n;
13. }
```



- **„alle Definitionen“ (all-defs):**
 - Jede Definition min. einmal (ohne dazwischenliegendes erneutes def) im c-use oder p-use verwenden.
- **„alle DR-Interaktionen“:**
 - Jedes Paar def/ref (ohne dazwischenliegendes erneutes def) auf irgendeinem Weg ausführen.
- Variation: **k-DR-Interaktionen**
- **Kontextüberdeckung**

- **Alle Referenzen:**
 - Alle ausgehenden Kanten eines Entscheidungsknotens berücksichtigen.
- **Alle Entscheidungs-/einige Berechnungs-Referenzen:**
 - Schwächere Überdeckung des Datenflusses, trotzdem Zweigüberdeckung und Testen „aller Definitionen“.
- **Alle Berechnungs-/einige Entscheidungs-Referenzen:**
 - Keine Zweigüberdeckung, aber Testen „aller Definitionen“ und „aller Referenzen“ in Berechnungsknoten.
- **Alle DR-Wege:**
 - Stärkere Überdeckung unter Kontrollflussaspekten durch Annäherung an Pfadtesten, allerdings ohne Schleifeniterationen.

Anzahl Testdaten pro Testkriterium:

Falls **Anzahl ausgehender Kanten pro Entscheidungsknoten** und **Anzahl Variablen pro Segment** durch Konstante **begrenzt**, dann für Programm mit n Segmenten **im schlechtesten Fall**:

- „**alle Definitionen**“: $O(n)$ Testdaten.
- „**alle E- / einige B-Referenzen**“, „**alle B- / einige E-Referenzen**“, „**alle Referenzen**“, Kontextüberdeckung: $O(n^2)$ Testdaten.
- „**alle DR-Wege**“: $O(2^n)$ Testdaten.

Aufgedeckte Fehler pro Testkriterium (vgl. [Rie97]):

Kriterium	Fehler erkannt
Alle Definitionen	24%
Alle E- / einige B-Referenzen	34%
Alle B- / einige E-Referenzen	48%
Alle DR-Interaktionen	51%

„**Alle B-Referenzen**“: z.B. bis zu 88% aller Berechnungsfehler.

„**Alle E-Referenzen**“: z.B. 100% aller Bereichsfehler.

Für restliche Kriterien keine Studien vorhanden.

Folgende **Fehler schlecht aufdeckbar**:

- Fehlende Pfade.
- Bereichsfehler durch falsch platzierte Anweisungen und falsche arithmetische Operatoren.
- Berechnungsfehler bei speziellen Werten.

Aber: ca. **9% aller Fehler** nur mit **datenflussbezogenen Methoden** **findbar**.

[Quelle: Riedemann: Spezialvorlesung „Software-Testmethoden“]

Kontrollflussbasiert:

- **Anweisungsüberdeckung** (C_0 -Überdeckung)
- **Zweigüberdeckung** (C_1 -Überdeckung).
- **Grenze-Inneres-Überdeckung** (C_{gi}).
- **Pfadüberdeckung** (C_∞ -Überdeckung).

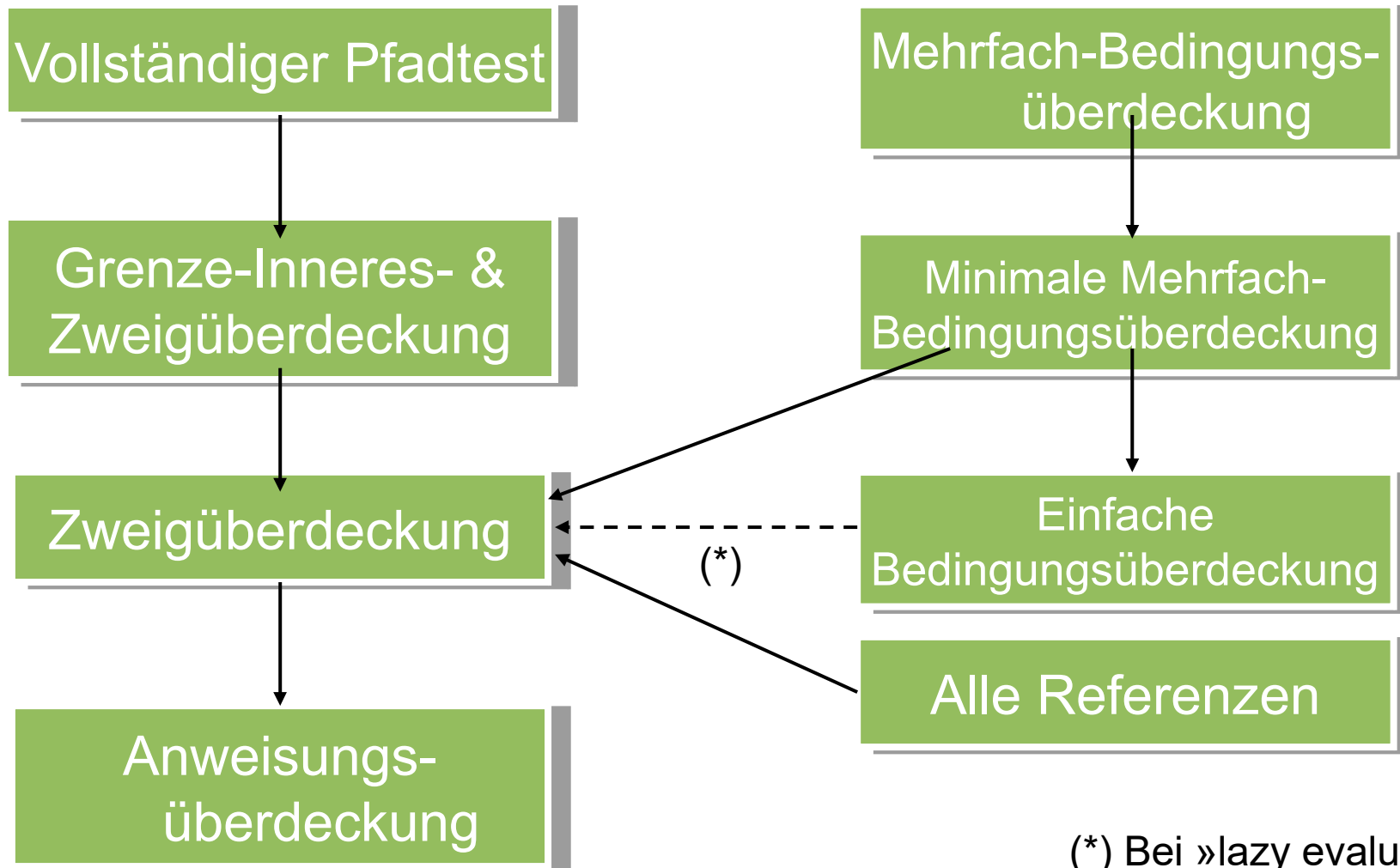
Bedingungsbasiert:

- **Einfache Bedingungsüberdeckung.**
- **Mehrfachbedingungsüberdeckung.**
- **Minimal bestimmende Mehrfachbedingungsüberdeckung.**

Datenflussbasiert:

- **Alle Definitionen** (all defs).
- **Alle Definition-Benutzung-Paare** (all def-uses).
- ...

Mächtigkeit der White-Box-Techniken



(*) Bei »lazy evaluation«

Bewertung der White-Box-Techniken

Überdeckungsmaß	Leistungsfähigkeit	Bewertung
Anweisungs- überdeckung (C_0)	Niedrig Entdeckt knapp ein Fünftel der Fehler	Notwendig, aber nicht hinreichend Entdeckt »dead-code«
Zweigüberdeckung (Entscheidungs- überdeckung, C_1)	Mittel, schwankt aber stark Entdeckt ca. 30% aller Fehler und ca. 80% der Kontrollfluss-Fehler	Entdeckt nicht ausführbare Zweige Zielt auf Verzweigungen
Bedingungsüber- deckung (C_2)	Niedrig	Umfasst i.Allg. nicht die Anweisungs- und Entscheidungsüberdeckung
Grenze-Inneres Test (C_{GI})	Mittel	Ergänzendes Kriterium nur für Schleifen
Mehrfach-Bedin- gungsüberdeckung	Hoch	Zielt auf komplexe Bedingungen Umfasst Entscheidungsüberdeckung Aufwand wächst stark
Datenflusstest	Mittel bis hoch, All c-uses ca. 50%, All p- uses ca. 34%, all defs ca. 25% (keine Berechnungsfehler!)	Zielt auf Variablen-Verwendung c-uses findet viele Berechnungsfehler
Pfadüber- deckung (C_∞)	Sehr hoch Entdeckt über 70% der Fehler	In den meisten Fällen nicht praktikabel

Einige Warnungen zum Thema Testüberdeckungen (1)

<http://www.elektronikpraxis.vogel.de/themen/embeddedsoftwareengineering/testinstallation/articles/247210>

„Acht Irrtümer über Code Coverage“:

- Irrtum 1: Man kann immer 100% Abdeckung erreichen.
 - z.B. nicht-erreichbarer Code
- Irrtum 2: Ein Coverage-Maß hat nur einen Namen.
 - z.B. minimale Mehrfachüberdeckung
- Irrtum 3: Ein Name bezeichnet immer dasselbe Coverage-Maß.
 - z.B. C0, C1, C2 unterschiedlich verwendet
- Irrtum 4: Es ist klar, wie Coverage gemessen wird.
 - Werkzeuge können Definitionen verschieden interpretieren

Einige Warnungen zum Thema Testüberdeckungen (2)

- Irrtum 5: Für die Coverage ist es egal, wie der Code formuliert ist.
 - z.B. zusammengesetzte Bedingungen vs. kaskadierende IF-Ausdrücke
- Irrtum 6: Durch geschickte Programmierung kann man sich das Leben erleichtern.
 - Ziel ist nicht Kennzahlen-Optimierung sondern Fehler-Findung
- Irrtum 7: Reicht, Testfälle zur vollständigen Code-Abdeckung aus Code abzuleiten.
 - z.B. fehlende Code-Abschnitte
- Irrtum 8: Code-Coverage misst die Qualität des Codes.
 - 100% Überdeckung heisst nicht 100% fehlerfrei.

- **Ziel:** Mit wenig Aufwand ausreichend unterschiedliche Testfälle erzeugen.
→ Mit gewisser Wahrscheinlichkeit vorhandene Fehlerzustände zur Wirkung bringen.
- **Ausführung der Testfälle:** Codebasierte Überdeckung messen.
- **Obere Teststufen: Black-Box** Testentwurfsverfahren.
- **Untere Teststufen: White-Box** Testentwurfsverfahren.

Weil vollständiges Testen nicht möglich: Testpriorisierung notwendig.

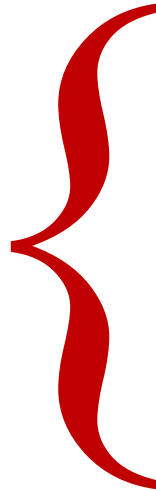
Ein Ansatz: Statische Analyse → vgl. nächster Unterabschnitt !

Wir wünschen allen ein
frohes Weihnachtsfest
und
einen guten Rutsch
in ein erfolgreiches
Jahr 2017!

Oxana, Emily, Roland,
Michael & Boris



2.4 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Test der Bedingungen

Datenflussbasierter Test

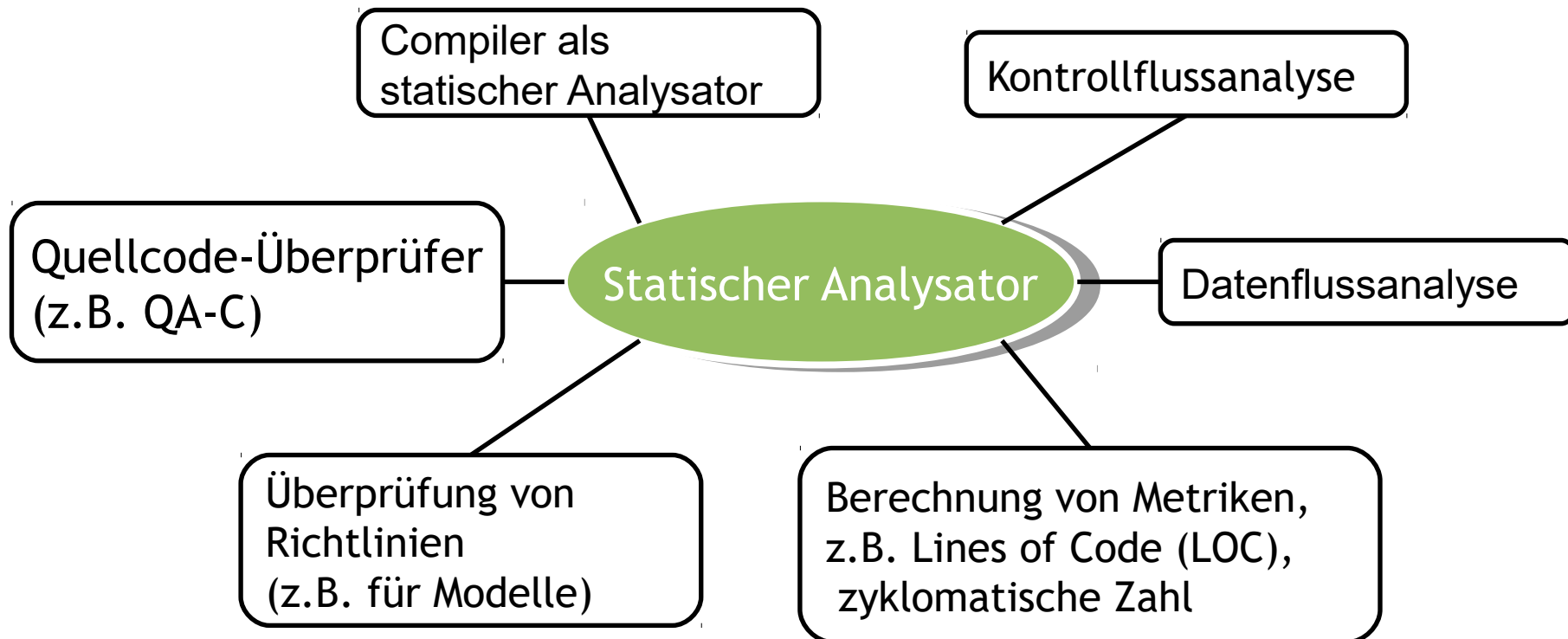
Statische Analyse

Was haben wir bislang über das Testen gelernt ?

- Völliges Austesten nicht möglich (vgl. Kap. 2.0).
- Trade-off zwischen Testaufwand und Anteil gefundener Fehler (niemals 100%).
- Kontrollfluss- und Datenflussbasierte Überdeckungskriterien helfen bei Trade-off, aber garantieren keine 100%-ige Fehlerfreiheit.

Lösung: Verifikationsansätze, die nicht (nur) auf Ausführung des Programms (= dynamisches Testen) beruhen:

- **Statische Analyse:** vollautomatisch, aber nur bestimmte Fehlerklassen
 - z.B. Analyse des Kontrollflussgraphen auf Anomalien
- **Symbolische Ausführung**
- **formale Verifikation:**
 - vollautomatisch (z.B. **Modell-Checking**): leichtere Bedienung, eingeschränkte Mächtigkeit
 - oder teilautomatisiert (z.B. **interaktives Theorembeweisen**): anspruchsvolle Bedienung, prinzipiell (beinahe) uneingeschränkte Mächtigkeit



- Suche nach Anomalien im Programmtext.
- **Anomalie:** Unstimmigkeit, die zur Fehlerwirkung führen kann.
 - Anomal: unregelmäßig, regelwidrig.
 - Kann Fehlerzustand sein, muss aber nicht.
- **Statische Analyse:** Nicht alle Fehlerzustände einfach nachweisbar (**Fehlerzustände** als Fehlerwirkung bei Ausführung).
 - Z.B.: bei Division Wert des Divisors in Variable halten
 - Variable kann **zur Laufzeit** Wert Null annehmen.
 - Fehlerwirkung, statisch nicht einfach erkennbar.

Einfach aber effektiv: **Manuelle** Analyse des **Kontrollflussgraphen** auf **Anschaulichkeit**.

- **Ziel:** Abläufe durch Programmstück leicht manuell erfassen.
- Teile des Graphen unübersichtlich
 - Zusammenhänge und Ablauf kaum nachvollziehbar.
 - Fehlerträchtig (und schlecht wartbar).
 - Überarbeitung des Programmtextes.

- **Kontrollflussanomalie:** Statisch feststellbare Unstimmigkeit beim Ablauf des Testobjekts:
 - Sprünge aus Schleifen heraus
 - Sprünge in Schleifen hinein
 - Programmstücke mit mehreren Ausgängen
- Müssen **keine Fehlerzustände** sein, **widersprechen aber Grundsätzen** strukturierter Programmierung und können fehlerträchtig sein.
- Kontrollflussgraph von Werkzeug generieren (gewährleistet **eins-zu-eins–Abbildung** zwischen Programmtext und Graph).

Vorgänger-Nachfolger-Tabellen: Beziehung der Anweisungen (Ausführungsabfolge).

Kein Vorgänger einer Anweisung. → Anweisung nicht erreichbar.

- **Fehlerzustand** ist erkannt.

Keine Vorgänger- bzw. Nachfolge-Anweisung für:

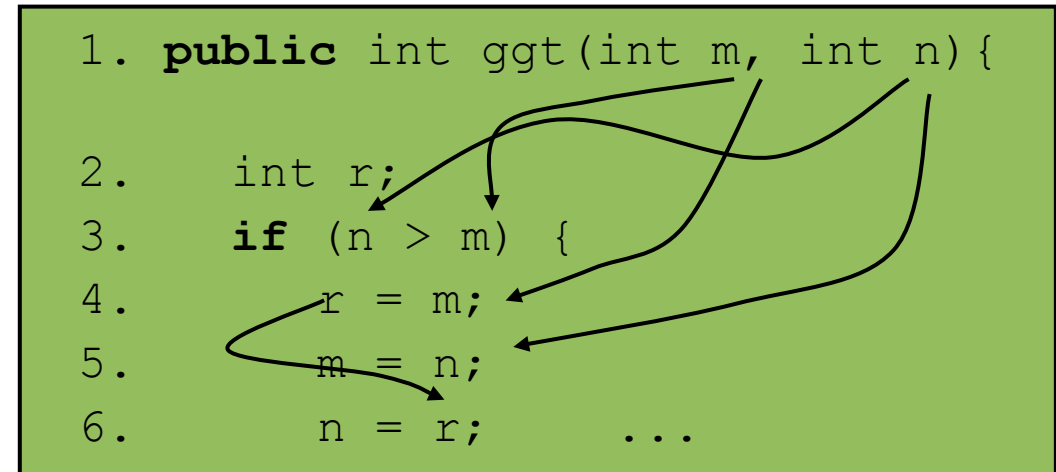
- Erste und letzte Anweisung eines Programm(teil)s.
- Programm(teil)e mit mehreren Eintritts- bzw. Austrittspunkten.

Vorgänger-Nachfolger-Tabelle: Beispiel ggt

```
1. public int ggt(int m, int n) {  
2.     int r;  
3.     if (n > m) {  
4.         r = m;  
5.         m = n;  
6.         n = r;  
7.     }  
8.     r = m % n;  
9.     while (r != 0) {  
10.        m = n;  
11.        n = r;  
12.        r = m % n;  
13.    }  
14. return n;  
15. }
```

Anwei- sung	Nach- folger	Vor- gänger
1	2	-
2	3	1
3	4, 7	2
4	5	3
5	6	4
6	7	5
7	8	3, 6
8	9, 12	7, 11
9	10	8
10	11	9
11	8	10
12	13	8
13	-	12

- Verwendung von Daten auf »**Pfaden**« durch Programm (vgl. datenflussbasiertes Testen)



- Aufdeckung von Datenflussanomalien:
 - **Referenzierende Verwendung (Lesen) einer Variablen** ohne vorherige Initialisierung
 - **Nicht-Verwendung** eines zugewiesenen Wertes einer Variablen

Datenfluss-Zustände von Variablen:

- Undefined (**u**): Variable hat keinen definierten Wert.
- Definiert (**d**): Variablen wird Wert zugewiesen.
- Referenziert (**r**): Wert der Variablen wird verwendet.

Damit Definition der **Datenflussanomalien**:

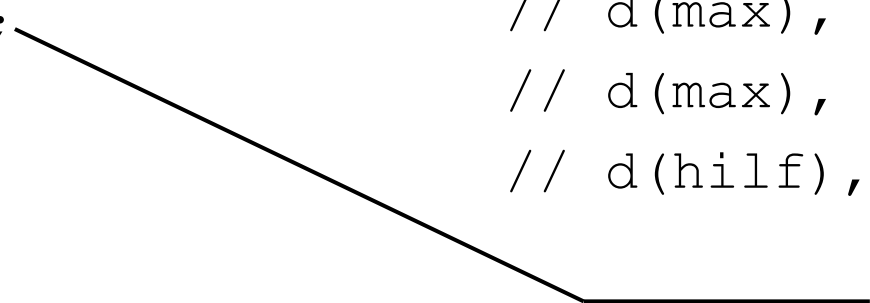
- **ur**-Anomalie: Undefineder Wert (u) einer Variablen wird gelesen (r).
- **du**-Anomalie: Variable erhält Wert (d) und wird ohne Verwendung ungültig (u).
- **dd**-Anomalie: Variable erhält Wert (d) und ohne Verwendung einen zweiten Wert (d).

```
1. void Tausch (int min, int max) { // d(min, max)
2.     int hilf;                    // u(hilf) (Java: hilf=0)
3.     if (min > max) {              // r(min, max)
4.         max = hilf;               // d(max), r(hilf)
5.         max = min;                // d(max), r(min)
6.         hilf = min;               // d(hilf), r(min)
7.     }
8. }                                // u(hilf)
```

Welche Anomalien sehen Sie ?

```
1. void Tausch (int min, int max) { // d(min, max)
2.   int hilf;                      // u(hilf) (Java: hilf=0)
3.   if (min > max) {                // r(min, max)
4.     max = hilf;                   // d(max), r(hilf)
5.     max = min;                   // d(max), r(min)
6.     hilf = min;                  // d(hilf), r(min)
7.   }
8. }
```

// u(hilf) **ur (hilf)**



ur-Anomalie der Variablen `hilf`:

- **Gültigkeitsbereich** der Variablen auf Funktion beschränkt.
- Erste Verwendung der Variablen auf der rechten Seite einer Zuweisung.
- Variable hat hier **undefinierten Wert**, der referenziert wird.
- **Initialisierung** bei Deklaration der Variablen nicht vorgenommen.

Datenfluss-Anomalien: Beispiel dd

```

1. void Tausch (int min, int max) { // d(min, max)
2.     int hilf;                    // u(hilf) (Java: hilf=0)
3.     if (min > max) {              // r(min, max)
4.         max = hilf;              // d(max), r(hilf)
5.         max = min;              // d(max), r(min)
6.         hilf = min;             // d(hilf), r(min)
7.     }
8. }                                // u(hilf)

```

dd-Anomalie der Variablen `max`:

- **Zwei Zuweisungen** ohne zwischenzeitige Verwendung.

Mögliche Ursachen bzw. Korrektur:

- Entweder: erste Zuweisung kann entfallen.
- Oder: Verwendung des ersten Wertes vergessen worden.
- Oder: Variablennamen / Reihenfolgen vertauscht.

dd (max)

Datenfluss-Anomalien: Beispiel du

```
1. void Tausch (int min, int max) { // d(min, max)
2.     int hilf;                    // u(hilf) (Java: hilf=0)
3.     if (min > max) {              // r(min, max)
4.         max = hilf;               // d(max), r(hilf)
5.         max = min;                // d(max), r(min)
6.         hilf = min;               // d(hilf), r(min)
7.     }
8. }
```

du (hilf)

du-Anomalie der Variablen `hilf`

- Variable `hilf` bekommt Wert zugewiesen, wird nirgends verwendet.
- Variable nur **innerhalb der Funktion** gültig.

Nicht jede Anomalie führt direkt zu **fehlerhaftem Verhalten**:

- du-Anomalie: keine direkte Auswirkungen, Programm kann korrekt laufen.
- Genauere Untersuchung der anomalen Programmstellen, weitere Unstimmigkeiten ausfindig machen.

Im Beispiel: Anomalien **offensichtlich**.

Aber: Zwischen Anweisungen, die Anomalie führen, können beliebig viele andere Anweisungen stehen.

- Anomalien nicht mehr offensichtlich.
- Bei manueller Prüfung übersehbar.
- Werkzeug zur Datenflussanalyse deckt Anomalien auf.

„Ausführung“ eines Programms mit symbolischen Werten:

- Verwendung **symbolischer** statt konkreter Werte für Eingabevariablen.
- Mit diesen wird entsprechend ‚gerechnet‘ (z.B. unter Verwendung mit Logik-formalisierter Annahmen an die Werte, algebraische Eigenschaften der verwendeten Operatoren etc.).

Beispiel

Anweisung: $C := A + 2 B$.

Schreibe symbolische Werte als $w()$.

Symbolische Ausführung der obigen Zuweisung ergibt:
 $w(C) = w(A) + 2 w(B)$.

- **Symbolischer “Test”** deckt Vielzahl normaler Testdaten ab.
 - Vollständiges Testen wird prinzipiell möglich.
- Insbesondere werden **Fehler entdeckt**, bei denen für Teilmenge der Eingaben Ergebnisse falsch berechnet werden.
- Im Gegensatz zu formaler Programmverifikation keine **formale Programmspezifikation** erforderlich.

- Benötige **formale Definition** der Programmiersprache.
- Bislang kein vollständiger Ersatz, sondern Ergänzung für funktionsorientierte Tests (insbesondere für sicherheitskritische Systeme).
- Theorembeweiser nötig für logische Berechnungen während „Ausführung“.
- Kein **vollständiger** automatischer Theorembeweiser für mächtige Programmiersprachen vorhanden.
 - Teilweise nur Approximation des Programmergebnisses möglich (aber immer noch umfassender als bei Testen).

- **Referenzierung** einer Variablen mit nicht definiertem Wert.
- **Inkonsistente Schnittstellen** zwischen Modulen und Komponenten.
- Variablen, die nie verwendet werden.
- Unerreichbarer Code (**»dead code«**).
- Verletzung von Programmierkonventionen.
- **Sicherheitsschwachstellen.**
- **Syntax-Verletzungen** von Code und Softwaremodellen.
- ...

Diskussion: Nutzen statischer Analysen

- **Frühe Erkennung** von Fehlerzuständen vor Testdurchführung.
- Frühe Warnung vor verdächtigen Aspekten in Code / Design.
 - Berechnung von Metriken.
- **Identifizierung** von Fehlerzuständen.
 - Durch dynamischen Test nicht effektiv aufzudecken.
- **Aufdecken von Abhängigkeiten** und Inkonsistenzen in Softwaremodellen.
- **Verbesserte Lesbarkeit**, Änderbarkeit und Wartbarkeit von Code und Design.
- **Vorbeugung** von Fehlerzuständen.

In **diesem** Abschnitt:

- White-Box-Testentwurfsverfahren
- Kontrollflussbasierter Test
- Datenflussbasierter Test
- Statische Analyse

Im **nächsten** Abschnitt:

- **Konfigurationsmanagement**

Zusammenfassung: White-Box Testentwurfungsverfahren

Grundlage aller White-Box Testentwurfungsverfahren: **Vorliegender Programmtext.**

Abhängig von Komplexität der Programmstruktur adäquate Testentwurfungsverfahren auswählbar.

- Anhand Programmtextes und ausgewählten Testentwurfungsverfahren Intensität der Tests festlegen (Ausgangskriterium/Testendekriterium).

Problem: »Nicht vorhandener Programmcode« bleibt unberücksichtigt.

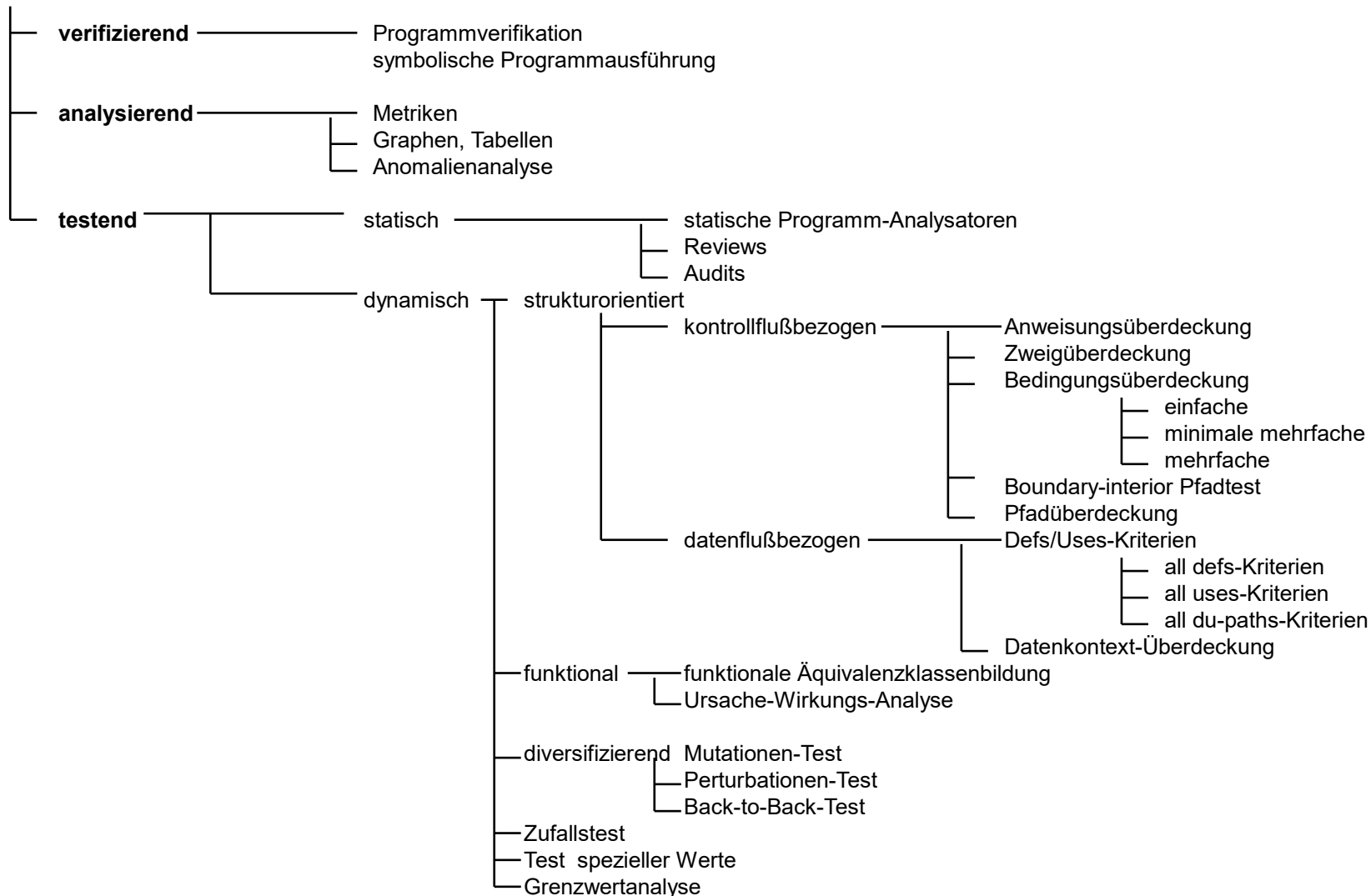
- **Übersehene Anforderungen** durch White-Box Testentwurfungsverfahren **nicht aufdeckbar.**
- Nur im Programm umgesetzte Anforderungen beim White-Box Testentwurfungsverfahren überprüfbar.

Zur **Instrumentierung Werkzeug** verwenden.

Worin unterscheiden sich Testentwurfungsverfahren?

TESTSTART	Welche Art von Tests werden durchgeführt?	Funktionale Anforderungen, Nicht-funktionale Anforderungen
TESTSTUFE	Für welche Testobjekte wird der Test spezifiziert?	Komponententest, Integrationstest, Systemtest, Abnahmetest
TESTER	WER testet?	Entwickler, (erfahrene) Tester, Benutzer, ...
TESTABDECKUNG	WAS wird abgedeckt?	Anforderungen, Anweisung, Entscheidung, ...
POTENZIELLE FEHLER	Welche potenziellen Fehler sollen identifiziert werden?	Behandlung von Grenzwerten Behandlung von Ausnahmen ...
ARTEFAKT	Was ist Grundlage für Auswahl und Herleitung der Testfälle?	Anforderungen -> Black-Box Code -> White-Box ...
DOMÄNE / PARADIGMA	Für welche spezielle Domäne bzw. Entwicklungsparadigma ist die Technik zugeschnitten?	OOP, Web-basiert, DB-basiert, Automotive, Sicherheitskritische SW, ...

Übersicht Prüfverfahren:



Anhang (weitere Einzelheiten und Beispiele)

White-Box-Test: Test, der auf Analyse interner Struktur einer Komponente oder eines Systems basiert.

»**Fehleraufdeckende**« **Stichproben** möglicher Programmabläufe und Datenverwendungen suchen.

White-Box-Testentwurfsv Verfahren: Dokumentiertes Verfahren zur Herleitung und Auswahl von Testfällen, basierend auf interner Struktur einer Komponente oder eines Systems.

Alle **Testentwurfsv Verfahren**, die zur

- Herleitung der Testfälle
- **Bestimmung der Vollständigkeit der Prüfung** (Überdeckungsgrad)

Information über innere Struktur des Testobjekts (z.B. Zweige) heranziehen.

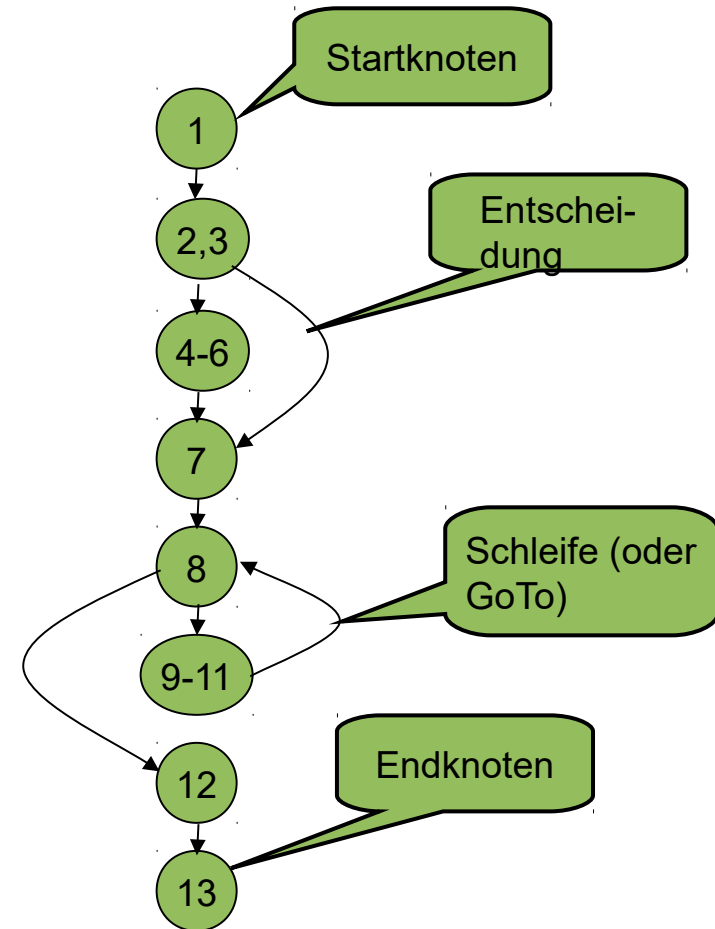
→ Strukturorientierte, strukturbezogene oder strukturelle Testentwurfsv Verfahren.

- **Kontrollfluss:** Abstrakte Repräsentation von Reihenfolgen von Ereignissen während Programmausführung.
 - Abfolge ausgeführter Anweisungen.
 - Muss nicht Reihenfolge der Anweisungen im Programmtext sein.
- **Determinierung** durch »steuernde« Anweisungen:
 - Unbedingte Sprünge
 - Bedingte Verzweigungen
 - Schleifen
- **Steuerung** durch Wahrheitswerte von Bedingungen:
 - Ermittelter Wahrheitswert:
 - »wahr«: → Ausführung des Programms mit THEN-Teil der IF-Anweisung
 - »unwahr«: → Durchlaufung des ELSE-Teil.
- **Schleifen** führen zu vorherigen Anweisungen zurück.
 - Bewirkt **mehrfache Ausführung** oder Schleifenkörper wird umgangen.

- Gerichteter Graph.
- **Knoten:** Anweisungen des Programms
Sequenzen von Anweisungen: Darstellung als einziger Knoten (Block), falls
 - Sequenz nur durch ihren ersten Knoten betretbar.
 - Keine Änderung des Programmablaufes innerhalb Sequenz.→ Genau dann, wenn erste Anweisung der Sequenz ausgeführt wird, werden alle weiteren Anweisungen der Sequenz ausgeführt
- **Kanten:** Ausführungsreihenfolgen die durch Knoten repräsentierten Anweisungen:
 - Knoten mit mehreren ausgehenden Kanten repräsentieren Kontrollfluss steuernde Anweisungen.
 - IF-Anweisung und Schleifensteuerung: Zwei abgehende Kanten.
 - CASE-Anweisung: Mehrere abgehende Kanten.
- Genau ein **Startknoten**.
- Genau ein **Endknoten**.

Bestimmung des größten gemeinsamen Teilers (ggT)
zweier ganzer Zahlen m und n:

```
1. public int ggt(int m, int n) {  
2.     int r;  
3.     if (n > m) {  
4.         r = m;  
5.         m = n;  
6.         n = r;  
7.     }  
8.     while (r != 0) {  
9.         m = n;  
10.        n = r;  
11.        r = m % n;  
12.    }  
12.    return n;  
13. }
```



Minimale Mehrfachbedingungsüberdeckung ($C_2(mM)$ -Überdeckung):

X (A>1)	Y (B=0)	X and Y
false	false	false
false	true	false
true	false	false
true	true	true

X (A=2)	Y (C>1)	X or Y
false	false	false
false	true	true
true	false	true
true	true	true

Welche **Testkombination** muss jeweils **nicht getestet werden** ?

Minimale Mehrfachbedingungsüberdeckung ($C_2(mM)$ -Überdeckung):

X (A>1)	Y (B=0)	X and Y
false	false	false
false	true	false
true	false	false
true	true	true

Alle Kombinationen bis auf erste testen, da bei dieser Änderung des Wahrheitswertes eines Terms **keine Änderung des Wahrheitswertes** der Kombination **bewirkt**.

X (A=2)	Y (C>1)	X or Y
false	false	false
false	true	true
true	false	true
true	true	true

Alle Kombinationen bis auf letzte testen, da bei dieser Änderung des Wahrheitswertes eines Terms **keine Änderung des Wahrheitswertes** der Kombination **bewirkt**.

Minimale Mehrfachbedingungsüberdeckung ($C_2(mM)$ -Überdeckung):

X (A>1)	Y (B=0)	X and Y
false	false	false
false	true	false
true	false	false
true	true	true

Testdatum

A=3,B=0,C=3

Testdatum

A=2,B=1,C=0

Testdatum

A=1,B=0,C=2

X (A=2)	Y (C>1)	X or Y
false	false	false
false	true	true
true	false	true
true	true	true

Werkzeugunterstützung für Kontrollflusstesten (1)

Softwarekonstruktion
WS 2016/17

Bullseye Coverage (Freeware):

<http://bullseyecoverage.software.informer.com>

Features: Funktionsüberdeckung, **Zweigüberdeckung**

Quilt (Freeware):

<http://quilt.sourceforge.net>

Features: Wieviel % des Codes überdeckt.

Emma (Freeware):

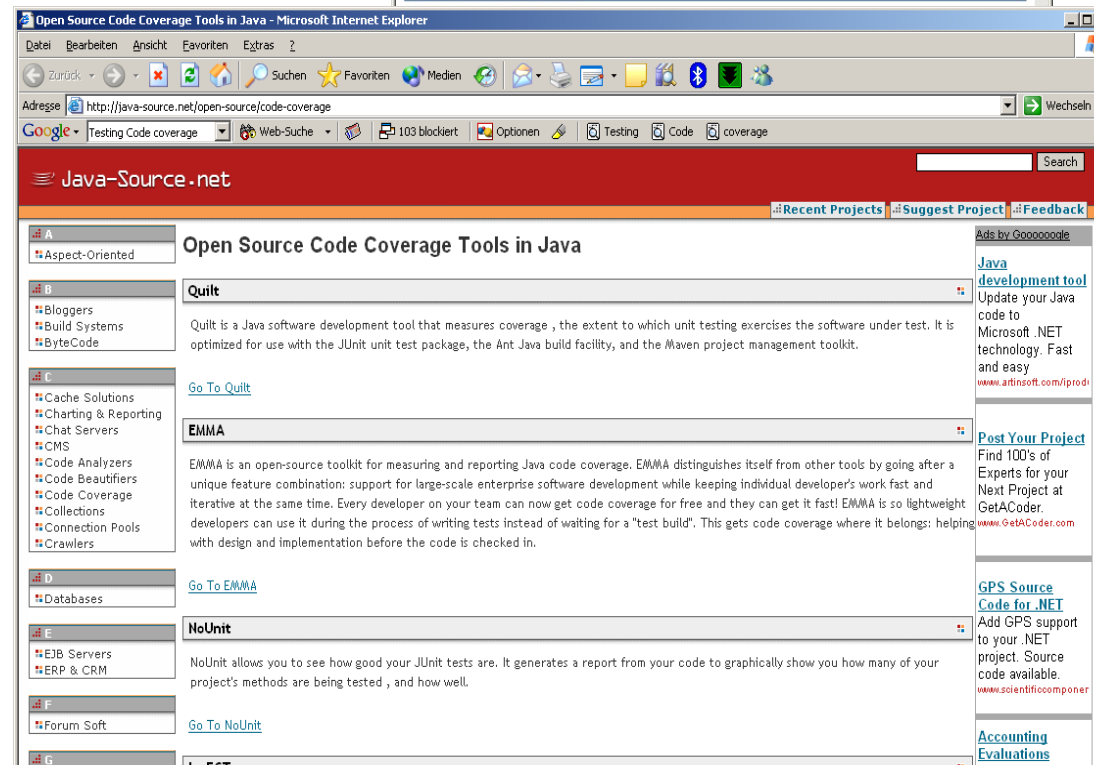
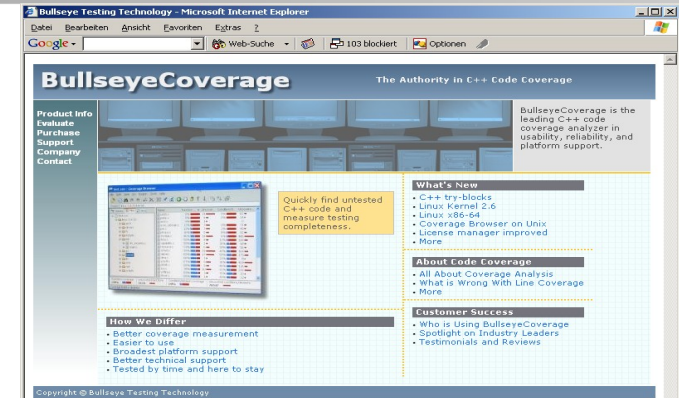
<http://emma.sourceforge.net>

Features: Testabdeckung; Erkennung von **toten Programmteilen**.

NoUnit (Freeware):

<http://nunit.sourceforge.net>

Features: Zeigt wie gut **Junit Tests** sind.



Werkzeugunterstützung für Kontrollflusstesten (2)

Softwarekonstruktion
WS 2016/17

Tessy (Freeware):

<http://www.hitex.com/index.php?id=code-coverage-in-tessy>

Features: Analysiert C_1 - und C_2 -Überdeckung.

Hansel (Freeware): <http://hansel.sourceforge.net>

Features: Gibt an wie viel **Prozent** des angenommenen Tests wirklich getestet wurde.

Testwell CTC++:

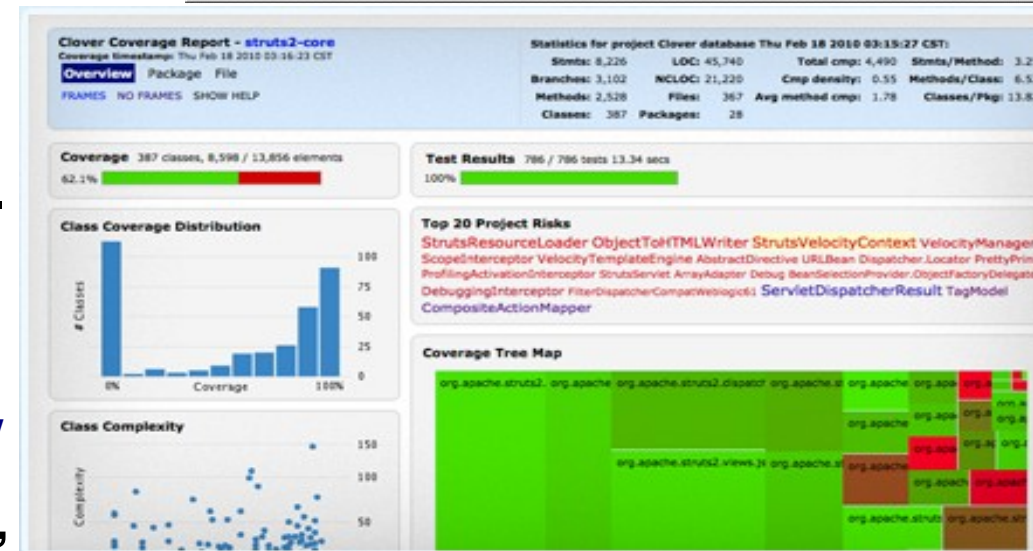
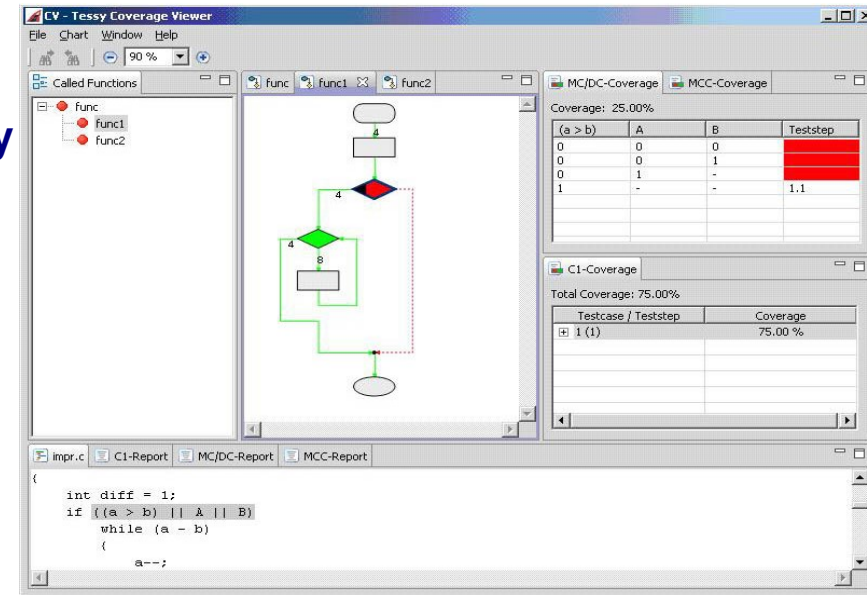
http://www.verifysoft.com/de_ctcpp.html

Features: Analysiert **alle** Testabdeckungsstufen.

Clover:

<http://www.atlassian.com/software/clover/overview>

Features: Analysiert **Anweisungs-, Zweig-, Entscheidungs- & Pfadüberdeckung.**



Ziel: Aufdeckung vorhandener Fehlerzustände im Dokument.

- **»Statische Analyse«:** Beinhaltet keine Ausführung der Prüfobjekte.

Beispiele:

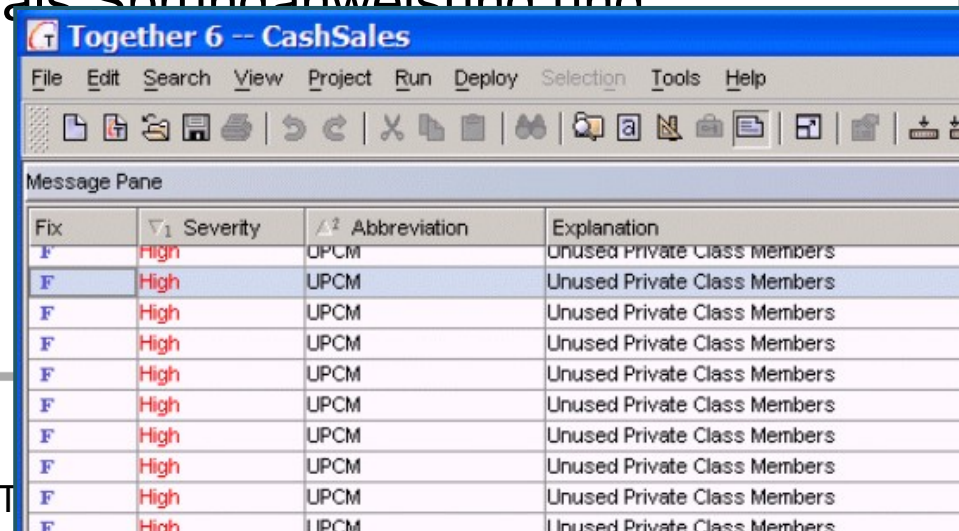
- Rechtschreibprüfprogramme als Art statische Analysatoren, die Fehler in Texten nachweisen. → Trägt zur **Qualitätsverbesserung** bei.
- Compiler führen statische Analyse des Programmtextes durch: Prüfen Einhaltung der Syntax der Programmiersprache.
- Spezielle Werkzeuge prüfen Einhaltung von **Programmierkonventionen**.

Weiteres Ziel: Ermittlung von Messgrößen oder Metriken, um Qualitätsbewertung durchzuführen. → Qualität messen.

- Statische Analyse: Mit **Werkzeugunterstützung** sinnvoll !
- Zu analysierendes Dokument: Muss formaler Struktur unterliegen, um durch Werkzeug überprüft werden zu können.
- Dokumente, die Formalismus unterliegen können, z. B.:
 - Technische Anforderungen
 - Softwarearchitektur
 - Softwareentwurf
- Informeller Text unterhalb Rechtschreibung und elementarer Grammatik nur mit KI analysierbar.
 - **Linguistische semantische Analyse:** aktueller Forschungsgegenstand.
 - **Aber:** Einführung von »**Normsprache**« ermöglicht einfachere Analysen.

- **Compiler:**
 - Führt statische Analyse des Programmtextes durch.
 - Prüft Einhaltung der Syntax der jeweiligen Programmiersprache.
 - Bietet zusätzliche Informationen.
- **Analysatoren:**
 - Einsatz zur gezielten Durchführung der Gruppen von Analysen
- **Fehler(zustände):**
 - Verletzung der Syntax.
 - Abweichungen von Konventionen und Standards.
 - Sicherheitslücken.
 - Kontrollflussanomalien.
 - Datenflussanomalien.

- **Fehler:** Verletzung der Syntax der Programmiersprache.
- Weitere Überprüfungen:
 - Verwendung einzelner Programmelemente.
 - Prüfung **typgerechter Verwendung** der Daten und Variablen bei streng typisierten Programmiersprachen.
 - Ermittlung von nicht deklarierten Variablen.
 - **Unerreichbarer Code.**
 - Fehler bei Über- oder Unterschreitung von Feldgrenzen.
 - Prüfung der **Konsistenz von Schnittstellen.**
 - Prüfung der Verwendung aller Marken als Sprunganweisung und Sprungziel.
- **Ergebnisse** in Form von Listen.
- Fehlerzustand nicht immer auffindbar.
→ **Weitere Untersuchungen.**

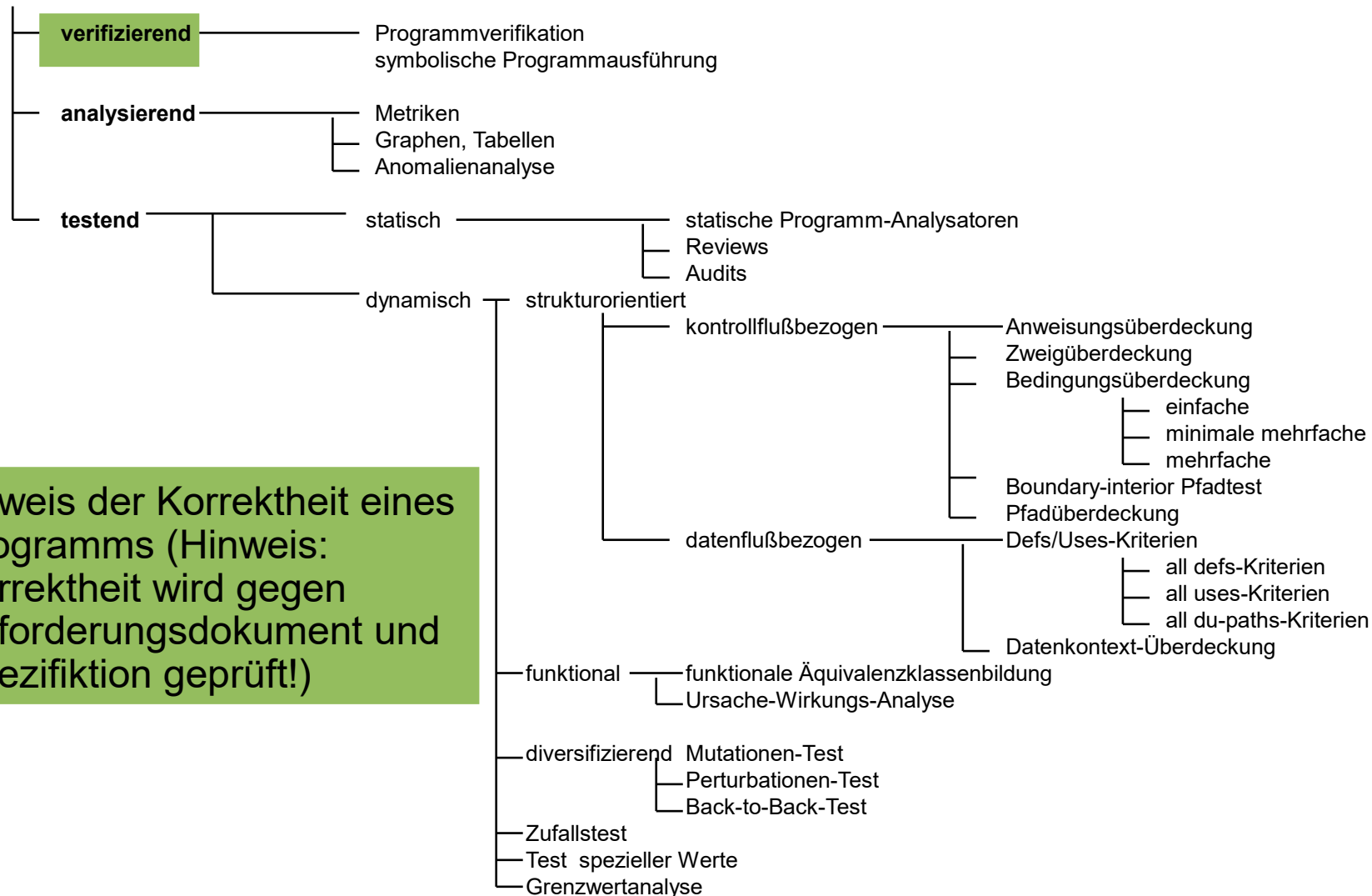


The screenshot shows the 'Together 6 -- CashSales' IDE. The Message Pane displays a list of errors, all with a severity of 'High' and an abbreviation of 'UPCM'. The explanation for all errors is 'Unused Private Class Members'.

Fix	Severity	Abbreviation	Explanation
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members
F	High	UPCM	Unused Private Class Members

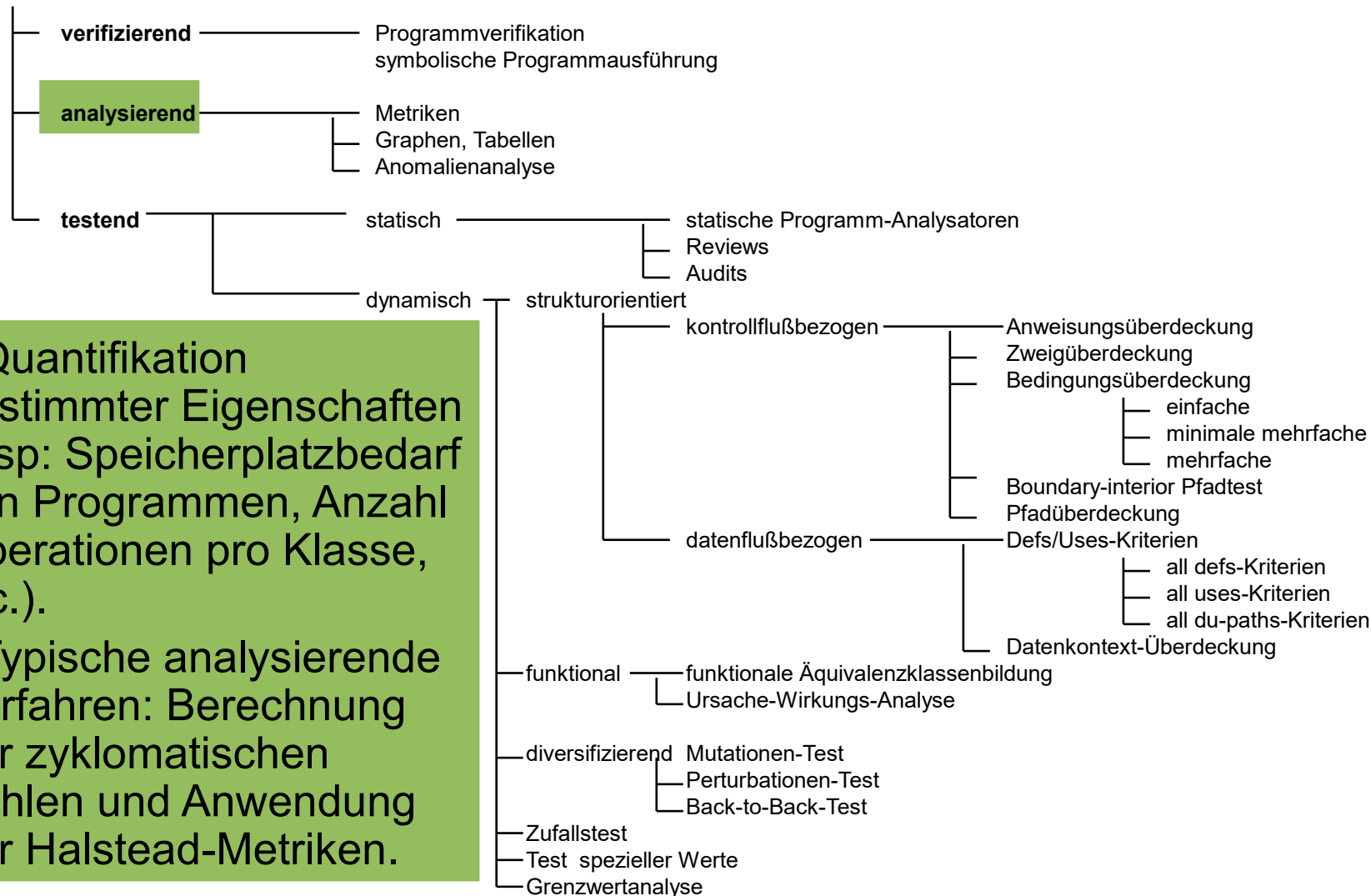
- **Sicherheitsprobleme** durch:
 - Verwendung bestimmter fehleranfälliger Programmkonstrukte.
 - Fehlende notwendige Überprüfungen.
- Beispiele:
 - **Fehlendes Abfangen** von Speicherüberläufen.
 - **Keine Überprüfung des Einhaltens** von Datenbeschränkungen bei Eingabe.
- **Analysewerkzeuge** können diese Mängel aufdecken.
 - Unterliegen einem bestimmten „Muster“.
 - Werden von Werkzeugen gesucht und analysiert.

Übersicht Prüfverfahren:



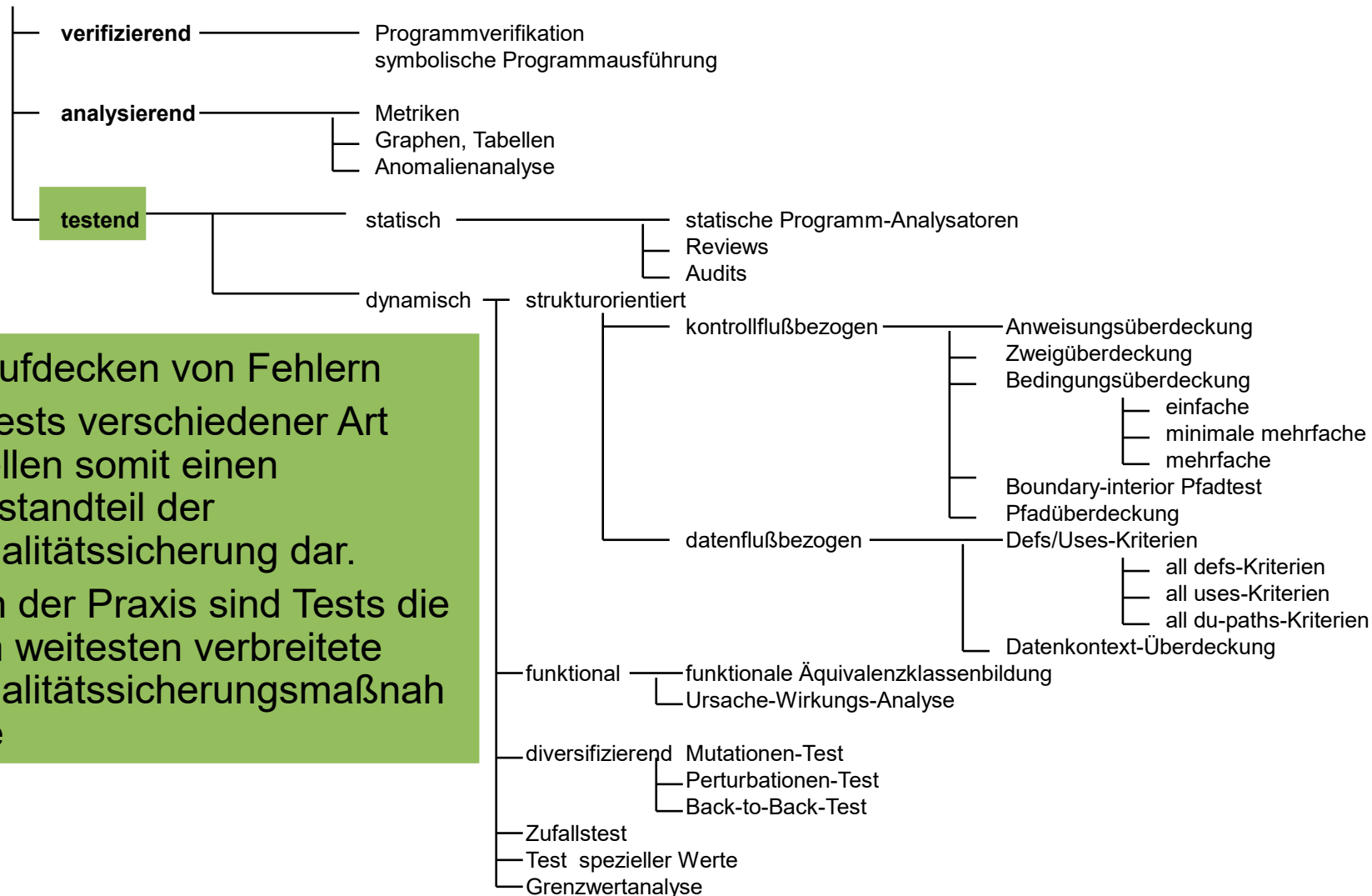
Beweis der Korrektheit eines Programms (Hinweis: Korrektheit wird gegen Anforderungsdokument und Spezifikation geprüft!)

Übersicht Prüfverfahren



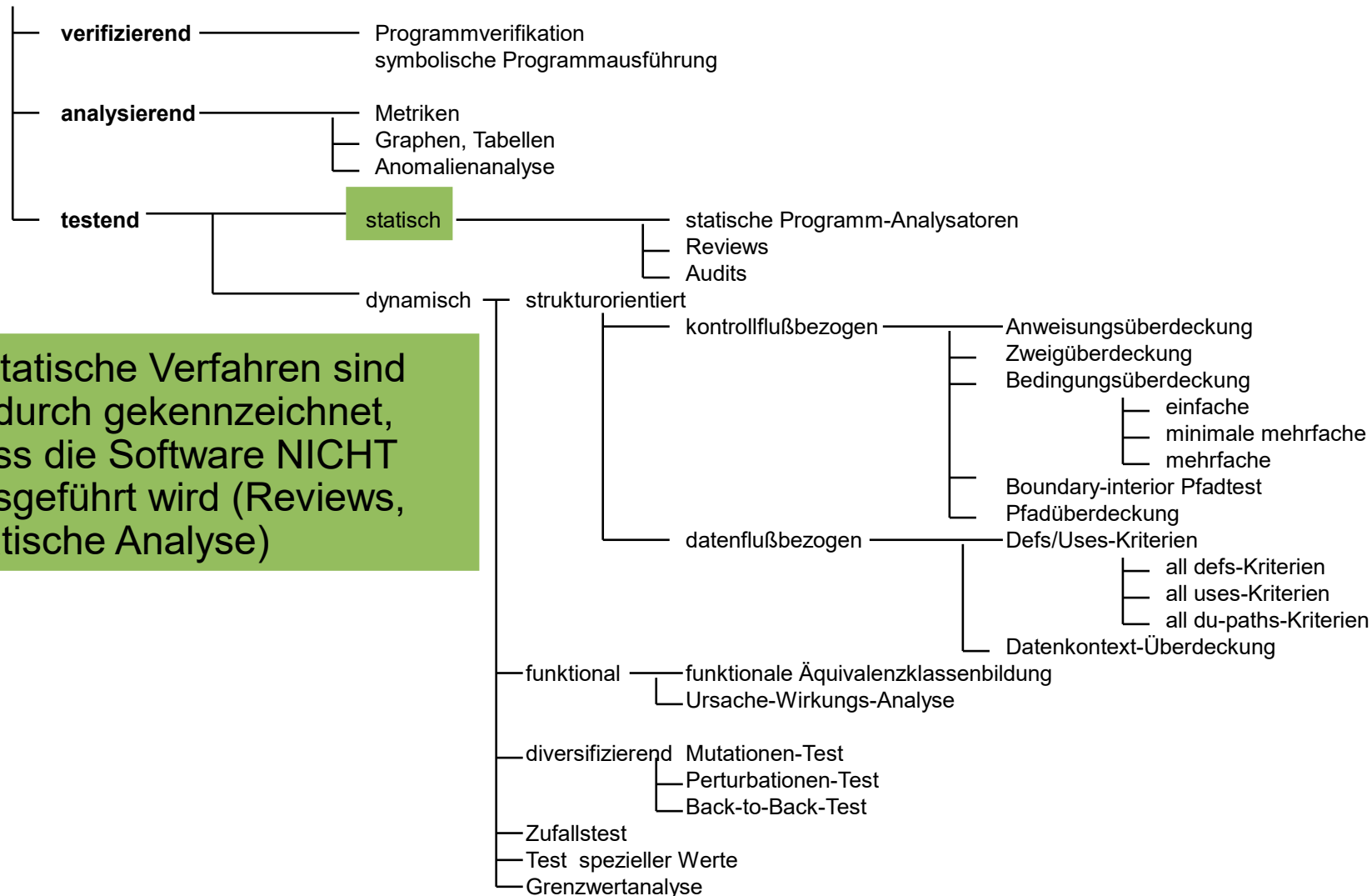
- Quantifikation bestimmter Eigenschaften (Bsp: Speicherplatzbedarf von Programmen, Anzahl Operationen pro Klasse, etc.).
- Typische analysierende Verfahren: Berechnung der zyklomatischen Zahlen und Anwendung der Halstead-Metriken.

Übersicht Prüfverfahren:



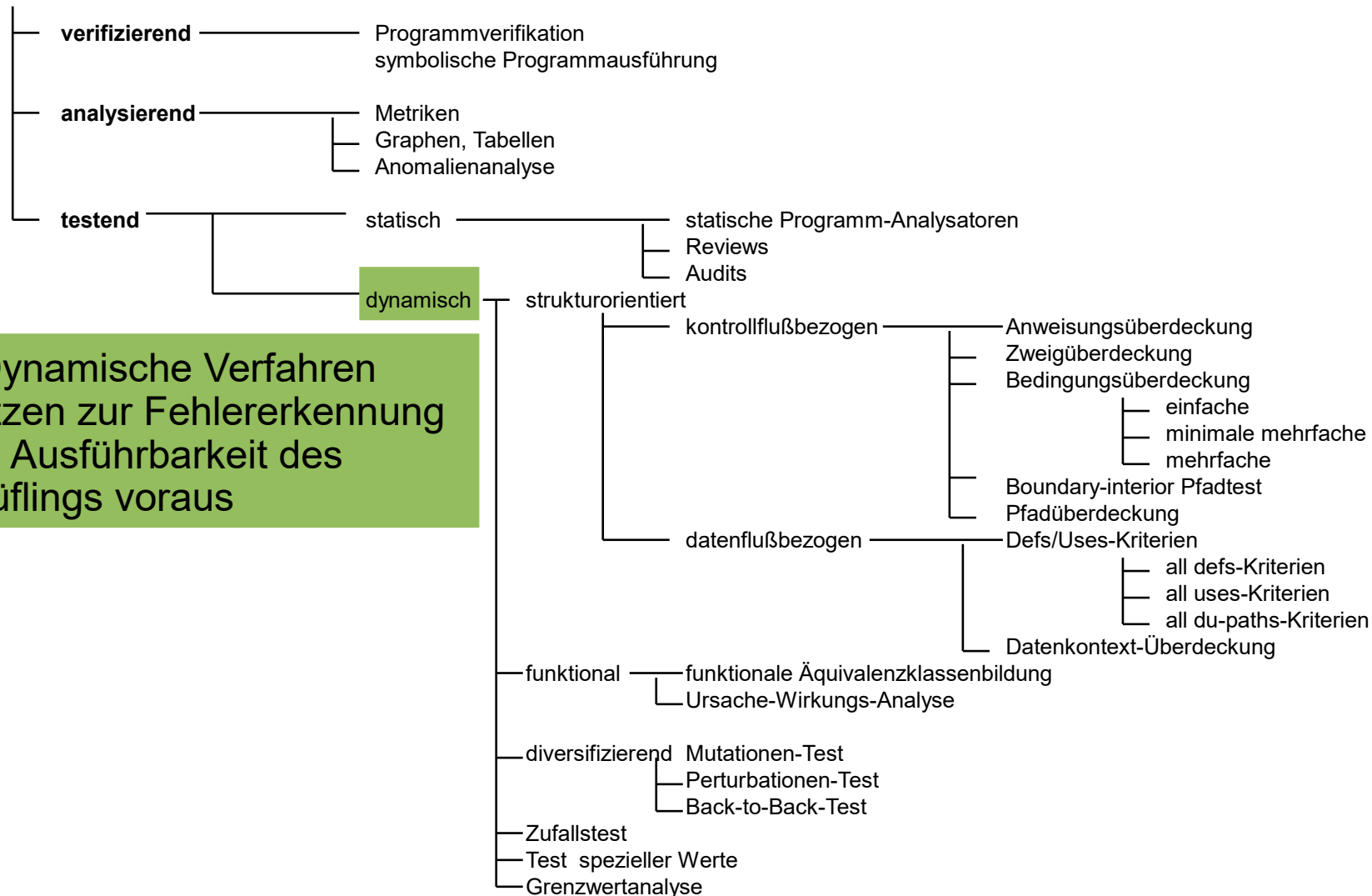
- Aufdecken von Fehlern
- Tests verschiedener Art stellen somit einen Bestandteil der Qualitätssicherung dar.
- In der Praxis sind Tests die am weitesten verbreitete Qualitätssicherungsmaßnahme

Übersicht Prüfverfahren:

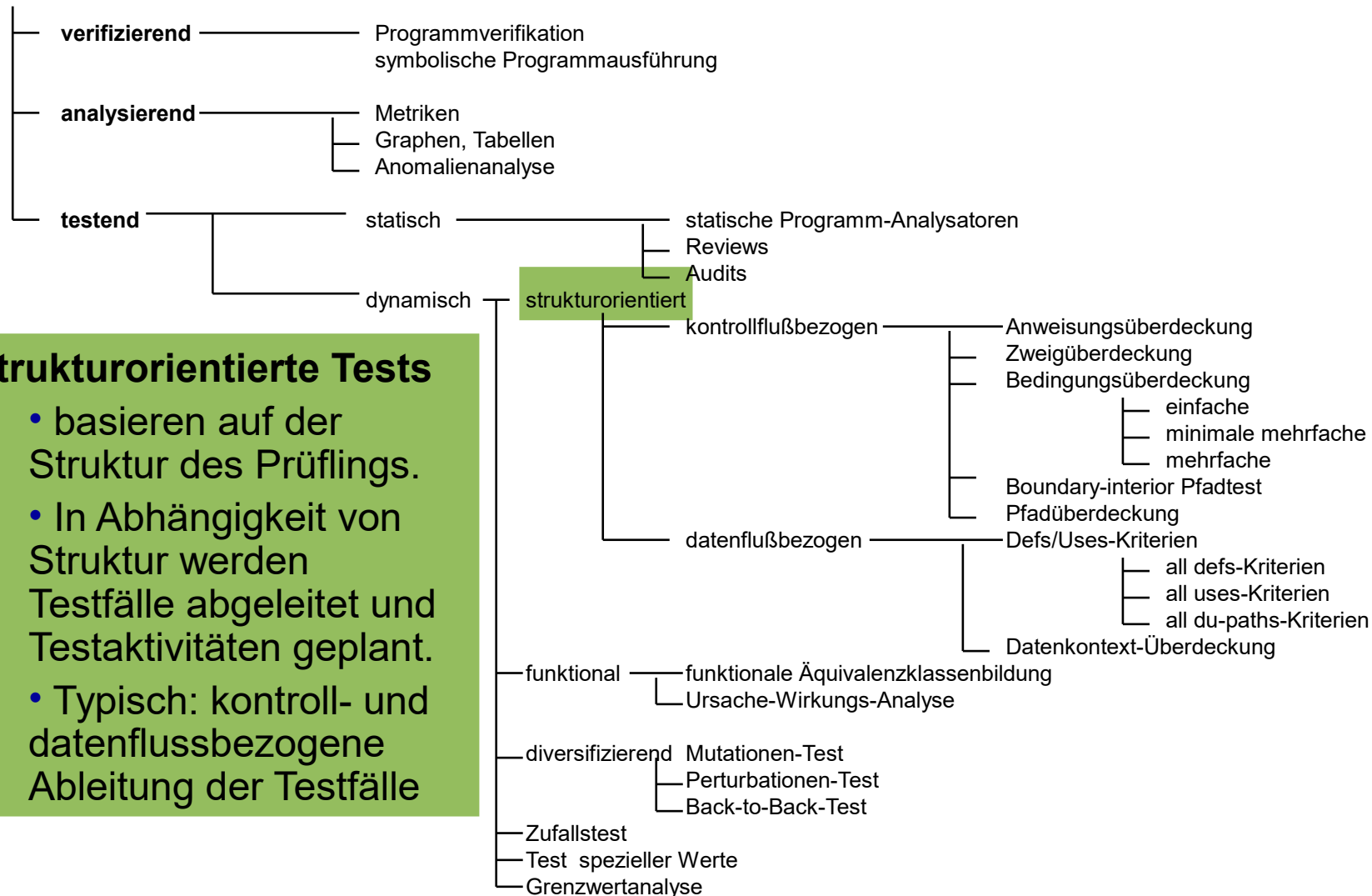


- Statische Verfahren sind dadurch gekennzeichnet, dass die Software NICHT ausgeführt wird (Reviews, statische Analyse)

Übersicht Prüfverfahren:



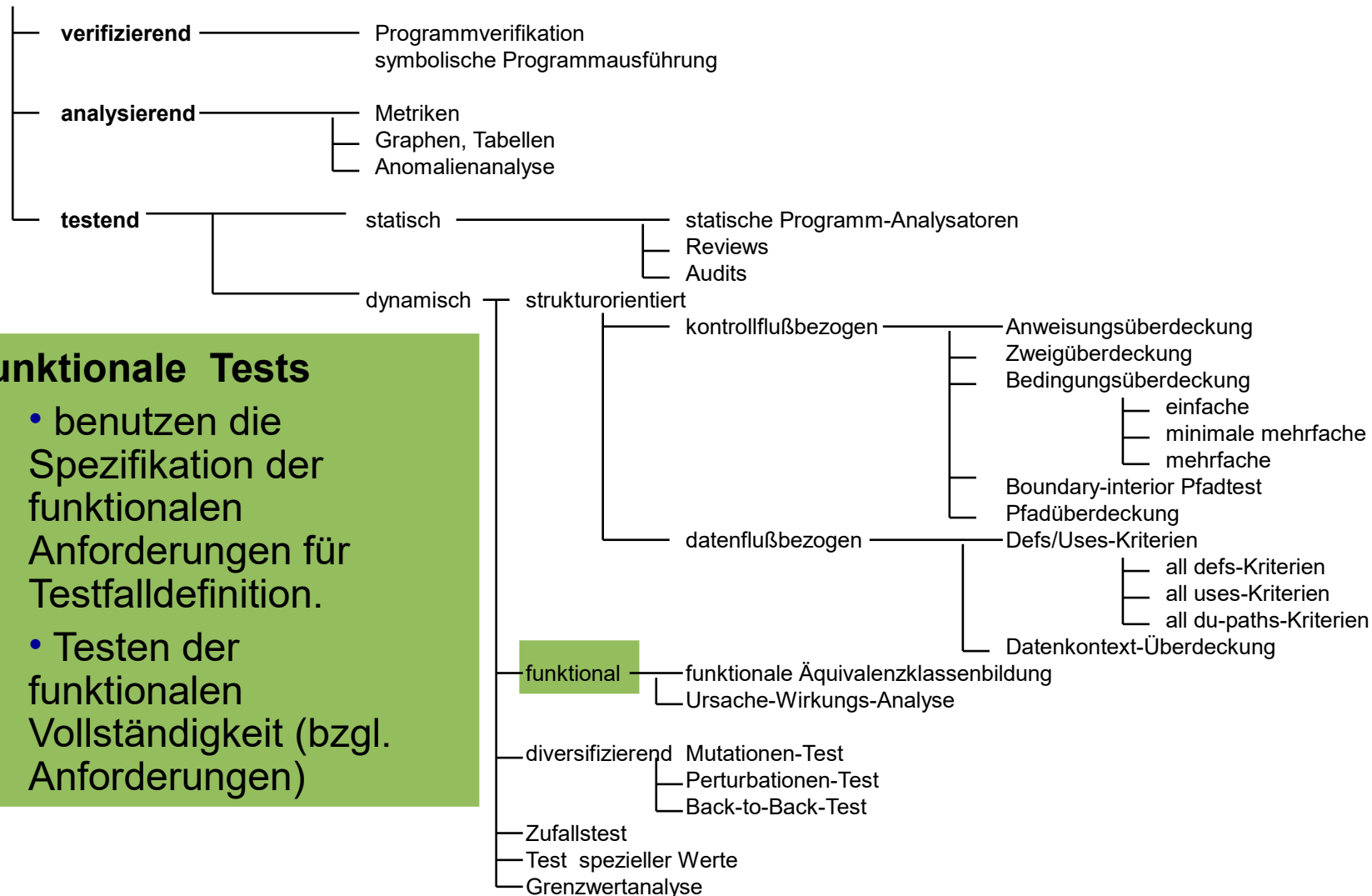
Übersicht Prüfverfahren:



• strukturorientierte Tests

- basieren auf der Struktur des Prüflings.
- In Abhängigkeit von Struktur werden Testfälle abgeleitet und Testaktivitäten geplant.
- Typisch: kontroll- und datenflussbezogene Ableitung der Testfälle

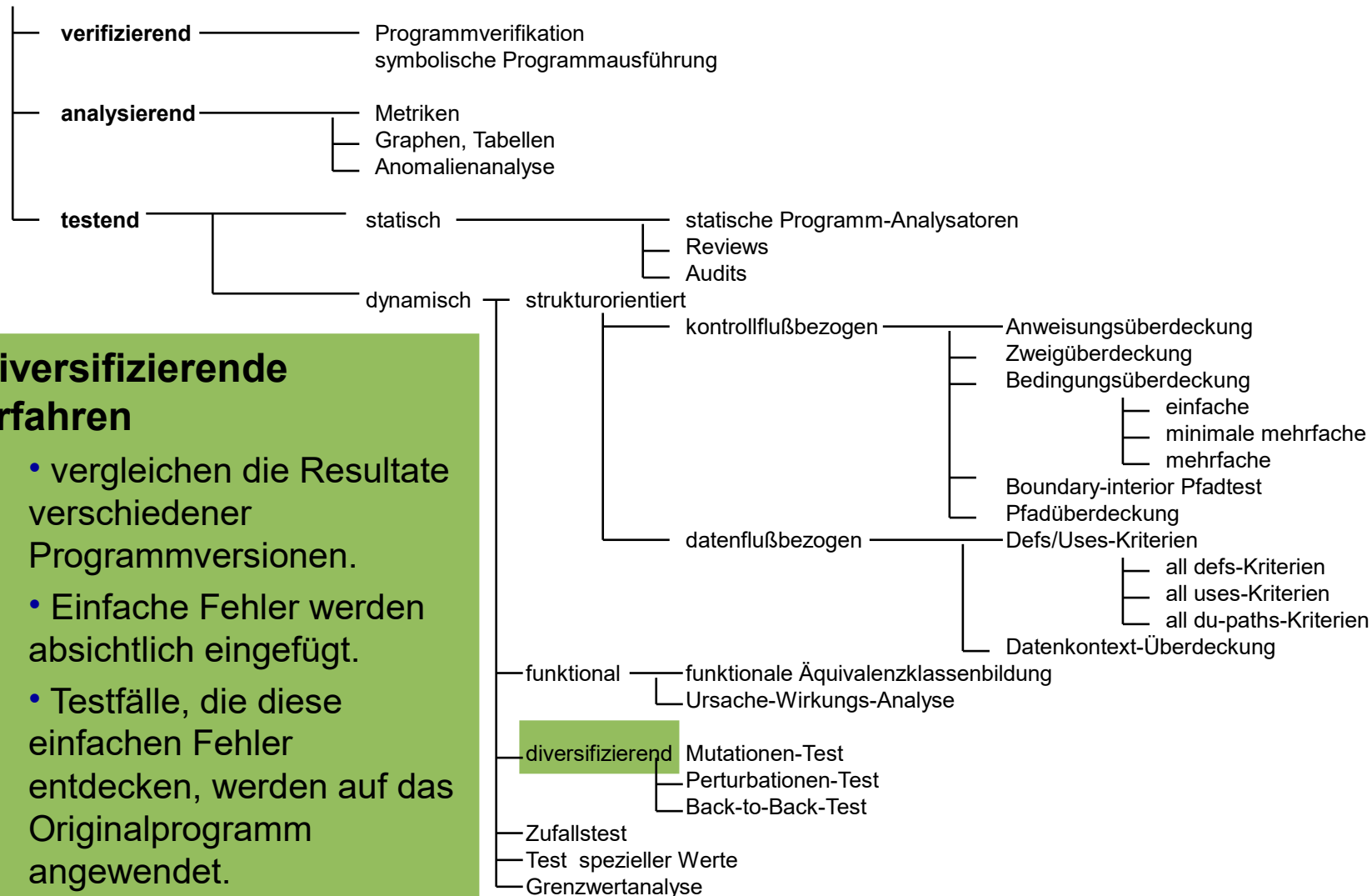
Übersicht Prüfverfahren:



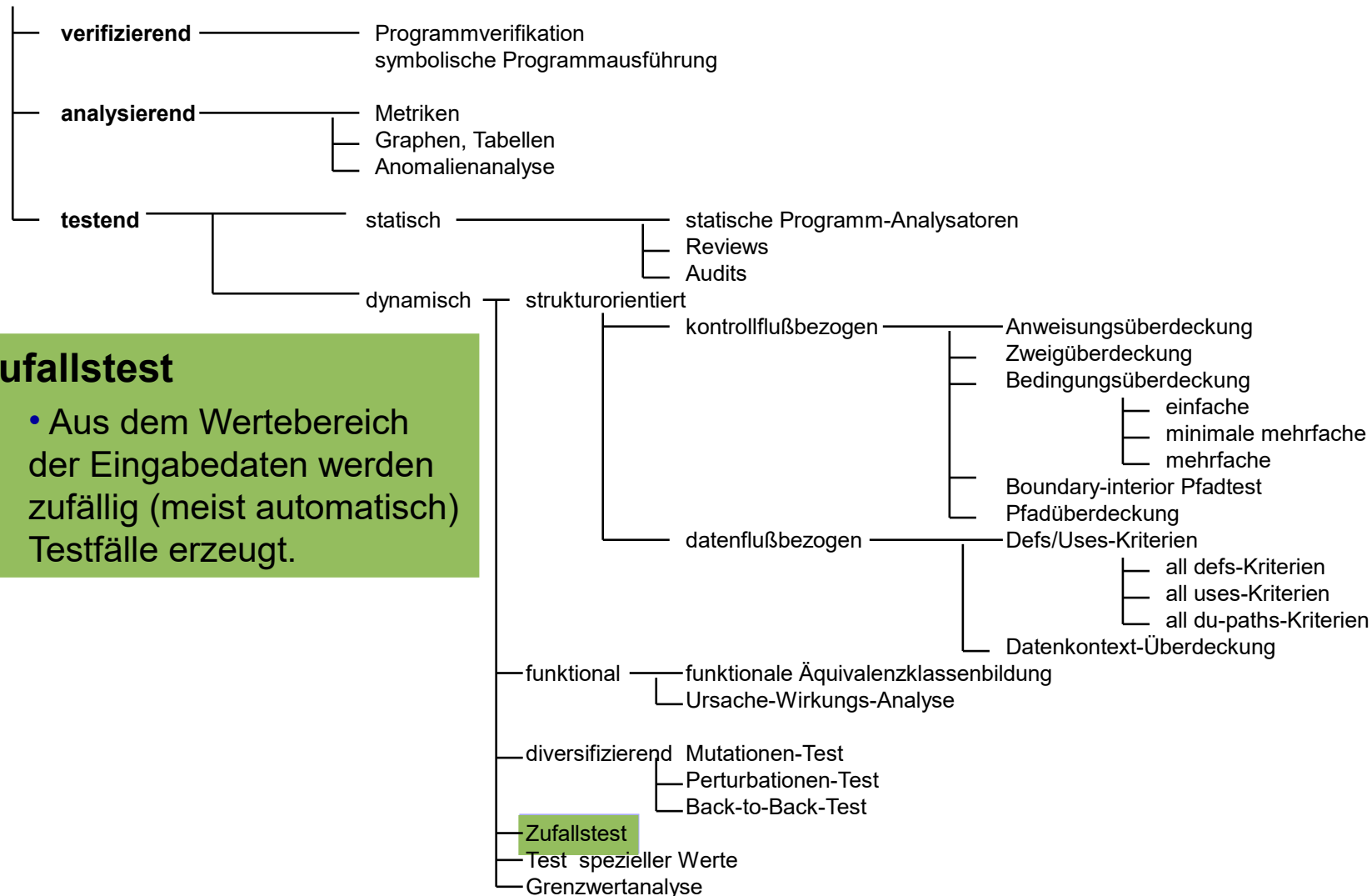
• funktionale Tests

- benutzen die Spezifikation der funktionalen Anforderungen für Testfalldefinition.
- Testen der funktionalen Vollständigkeit (bzgl. Anforderungen)

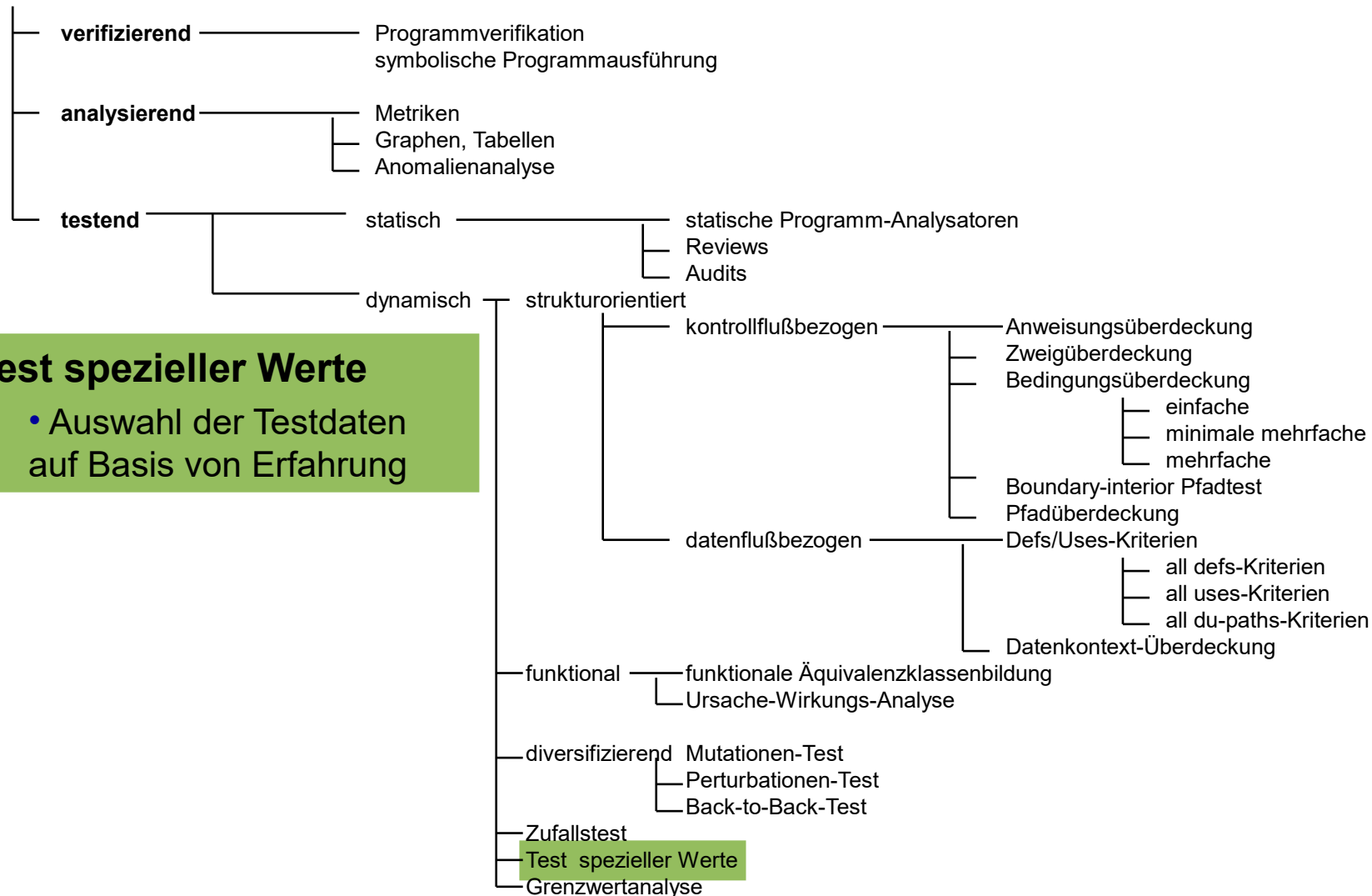
Übersicht Prüfverfahren:



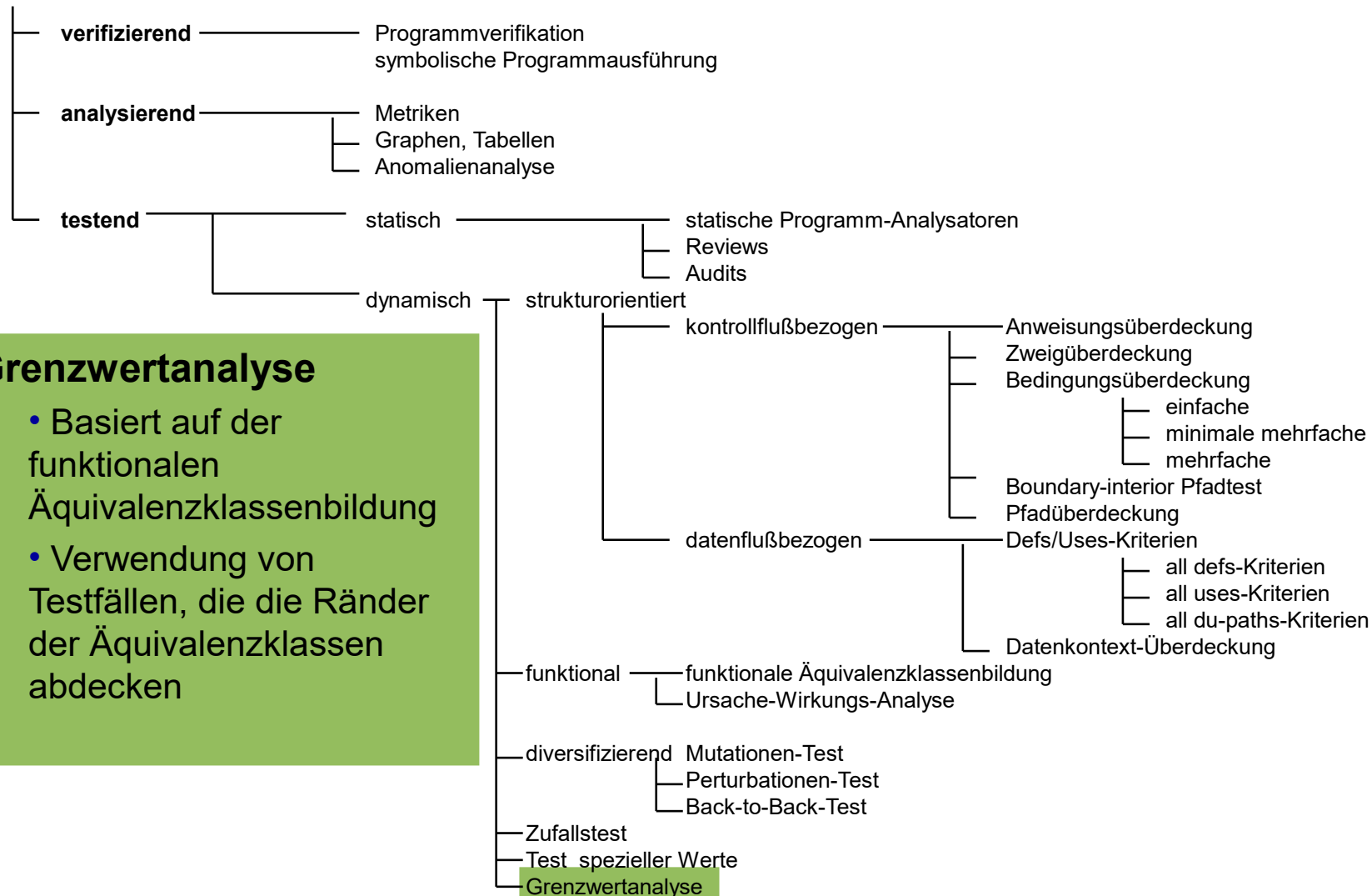
Übersicht Prüfverfahren:



Übersicht Prüfverfahren:



Übersicht Prüfverfahren:



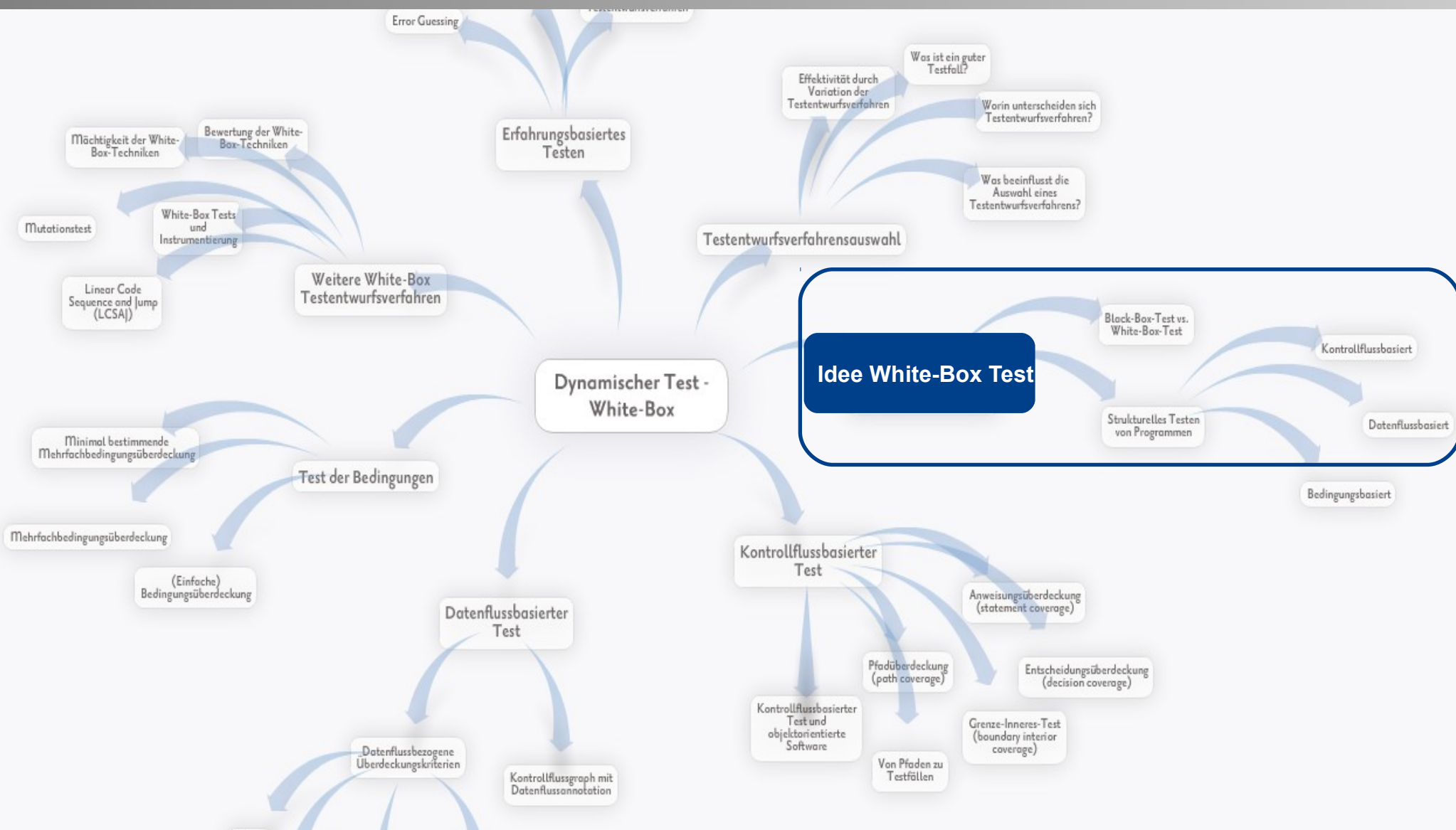
- **Grenzwertanalyse**
 - Basiert auf der funktionalen Äquivalenzklassenbildung
 - Verwendung von Testfällen, die die Ränder der Äquivalenzklassen abdecken

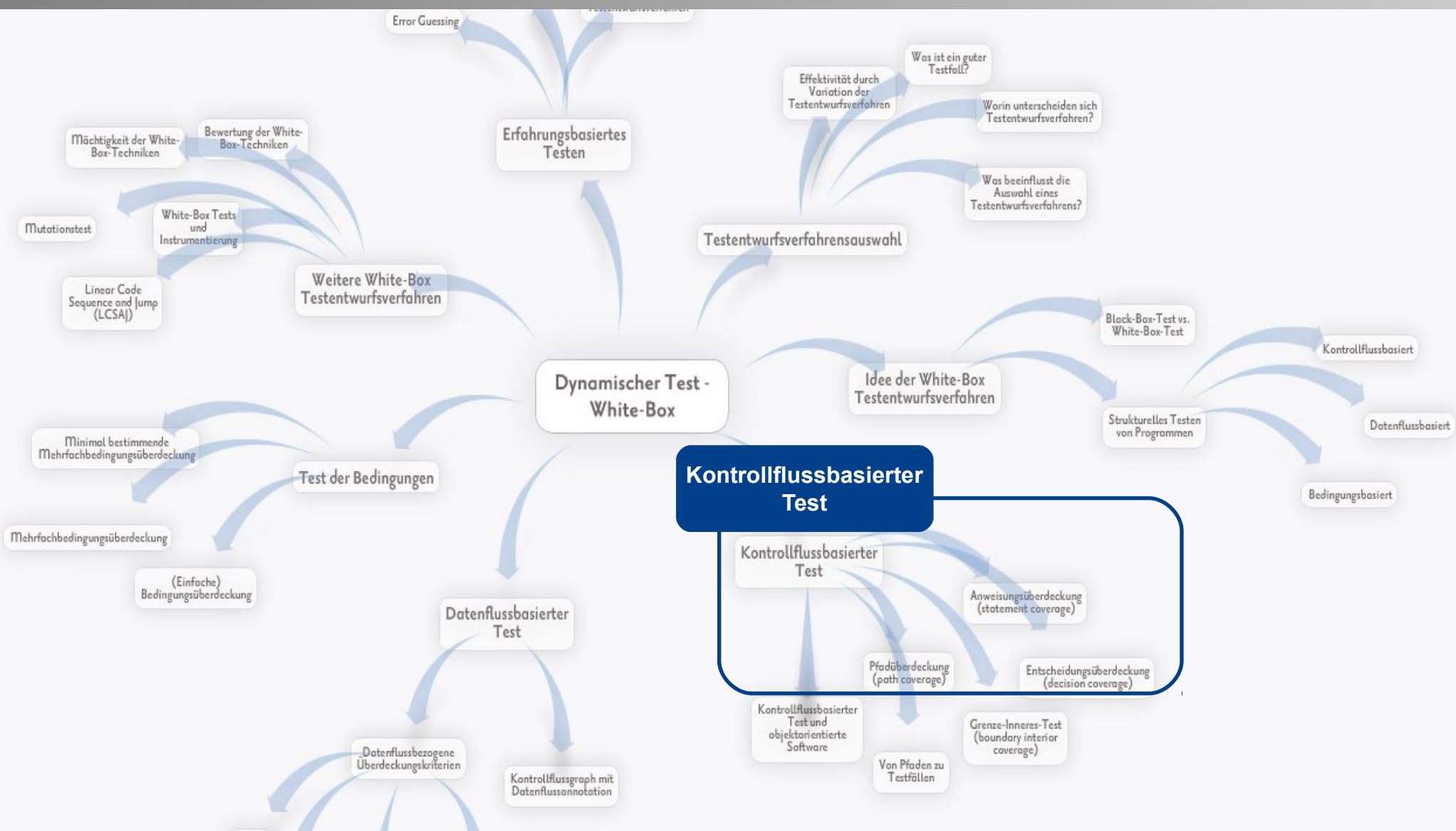
Alte Folien (nicht verwenden)

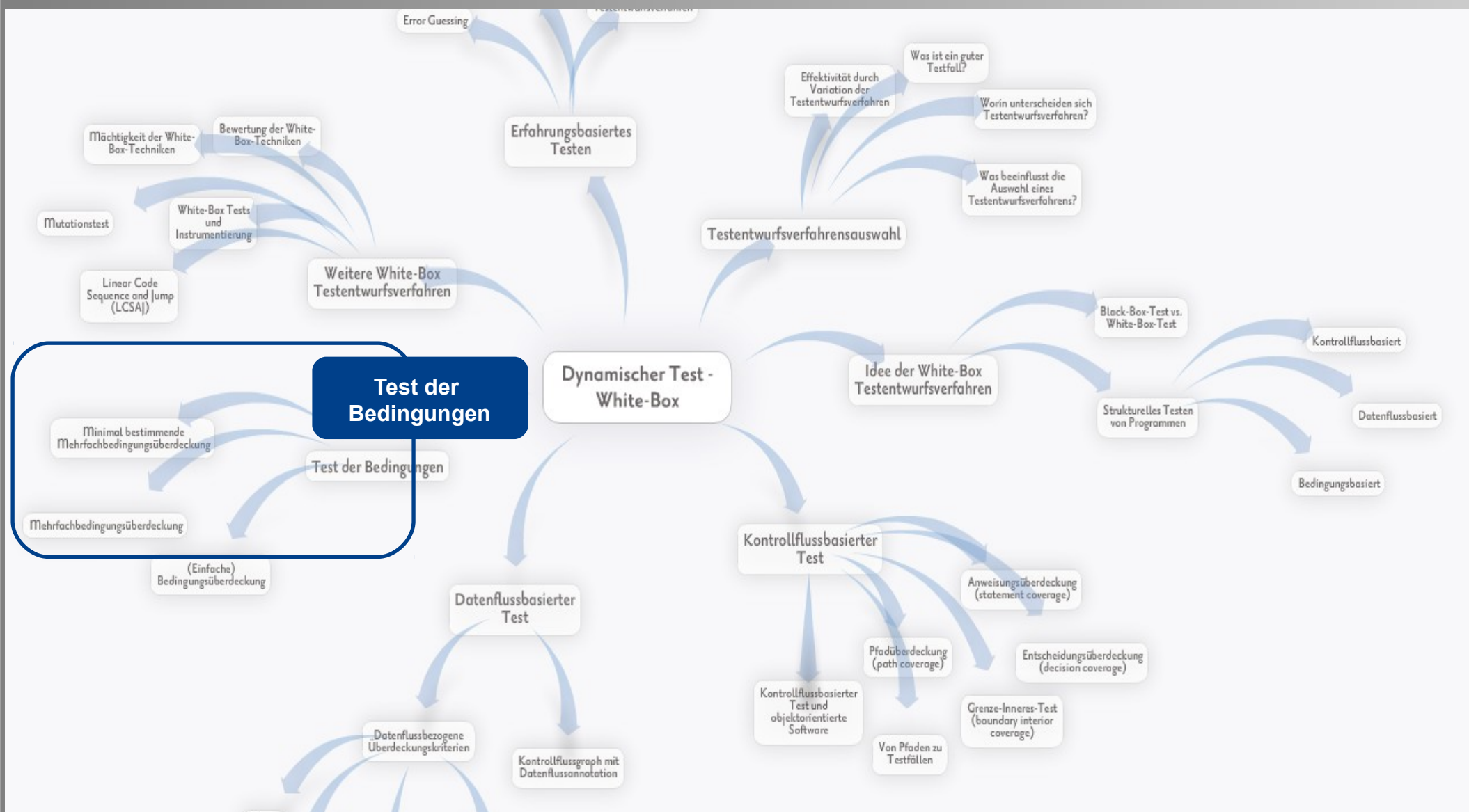
Softwarekonstruktion
WS 2016/17



202







Wodurch unterscheiden sich **verschiedene Methoden** des kontrollflussbezogenen Testens ?

Antwort:

- In **Festlegung der Art der Kontrollflusswege** oder -wegstücke oder Konstrukte.
- In **geforderter Häufigkeit** ihrer Ausführung.

Dynamisches, kontrollflussbasiertes Testentwurfungsverfahren: Fordert min. einmalige Ausführung aller Anweisungen des Testobjekts.

$$\text{Anweisungsüberdeckungsgrad} = \frac{\text{Anzahl durchlaufener Anweisungen}}{\text{Gesamtzahl Anweisungen}}$$

Jeder Testfall anhand eines Pfads durch Kontrollflussgraphen bestimmbar.

- Beim Testfall auf dem Pfad liegende Kanten des Graphen durchlaufen.
→ Anweisungen in entsprechender Reihenfolge zur Ausführung kommen.
- Bei Berechnung nur zählen, ob Anweisung bei Ausführung durchlaufen wurde, **Häufigkeit der Ausführung spielt keine Rolle.**
- **Einzelne Testfälle:** Neben Vor- und Nachbedingungen, auch **erwartete Ergebnisse und erwartetes Verhalten** des Testobjektes vorab bestimmen und mit tatsächlichem Ergebnis vergleichen.

Zuvor **festgelegter Deckungsgrad erreicht.** → Test ausreichend angesehen und beendet.

Auch als **C₀-Maß** bezeichnet (**C**overage = **Ü**berdeckung).

Ist Anweisungsüberdeckung hinreichend, um Fehler aufzudecken?

Antwort:

-

-

Ist **Anweisungsüberdeckung** hinreichend, um Fehler aufzudecken?

Antwort:

- Anweisungsüberdeckung ist zwar ein notwendiges Kriterium, um potentielle Fehler aufzudecken, aber **nicht hinreichend**.
- Gewisse **Kontrollflussfehler** werden **nicht entdeckt**.

Entscheidungstest: White-Box-Testentwurfungsverfahren, bei dem Testfälle im Hinblick auf Überdeckung der Entscheidungsausgänge entworfen werden.

Entscheidung: Stelle im Programm, an der Kontrollfluss in zwei oder mehrere alternative Wege verzweigen kann. Knoten mit zwei oder mehreren ausgehenden Kanten.

Entscheidungsausgang: Ergebnis einer Entscheidung, das einzuschlagenden Weg im Kontrollfluss bestimmt.

Entscheidungsüberdeckung: Anteil an Entscheidungsausgängen, die durch Testsuite geprüft wurden. 100% Entscheidungsüberdeckung bedeutet 100% Zweigüberdeckung und 100% Anweisungsüberdeckung.

Jeden Testfall anhand eines Pfads durch Kontrollflussgraphen zur betrachteten **Entscheidung bestimmen**.

- **Jede Entscheidung:** Zu allen Fällen auswerten (IF / WHILE / FOR-Bedingungen zu wahr und falsch, CASE-Ausdruck zu allen Alternativen).
- Jedes mögliche Ergebnis muss min. einmal vorlegen.
- Für einzelne **Testfälle:** Neben Vor- und Nachbedingungen **erwartete Ergebnisse und erwartetes Verhalten** des Testobjektes vorab bestimmen und **mit tatsächlichem Ergebnis vergleichen**.

$$\text{Entscheidungsüberdeckungsgrad} = \frac{\text{Anzahl getestete Entscheidungsergebnisse}}{\text{Gesamtzahl Entscheidungsergebnisse}}$$

Entscheidungsüberdeckung vs. Zweigüberdeckung

Entscheidungsüberdeckung:

- Betrachtet Entscheidungen: Min. einmal zu beiden Möglichkeiten auswerten.
- Jeder Testfall: **Auswahl anhand Entscheidung.**

Zweigüberdeckung (branch coverage):

- Alle Kanten im Kontrollflussgraphen abdecken.
- Jeder Testfall: Bestimmung anhand eines Pfads durch Kontrollflussgraphen.
- Durch Testfälle **alle Kanten des Graphen durchlaufen** (auch „leere“ Kanten, Knoten „überbrücken“). → Jede Entscheidung zu allen Möglichkeiten auswerten.
- Auch als **C₁-Maß** bezeichnet.

$$\text{Zweigüberdeckungsgrad} = \frac{\text{Anzahl durchlaufener Zweige}}{\text{Gesamtzahl Zweige}}$$

Bei Betrachtung vollständiger Testpfade kommen beide Definitionen **zum selben Ergebnis.**

In Aussage **stärkeres Kriterium** als Anweisungsüberdeckung.

- Berücksichtigt bei
 - **Verzweigung** des Kontrollflusses **beide bzw. alle Möglichkeiten**,
 - **Schleifen** Umgehung des Schleifenkörpers und Rücksprung zum Schleifenanfang.
- Kann **fehlende Anweisungen** im Gegensatz zur Anweisungsüberdeckung **erkennen** !

Entscheidungsergebnisse unabhängig voneinander betrachten und keine bestimmte Kombinationen einzelner Entscheidungen fordern.

Resümee:

- **100%ige Entscheidungsüberdeckung anstreben.**
- Nur wenn **jede mögliche Verzweigung** des Kontrollflusses in Testphase **berücksichtigt** wird, kann Test als ausreichend eingestuft werden.

- Bei **Entscheidungsüberdeckung**: Ermittelte **Ergebnis-Wahrheitswert** einer Bedingung berücksichtigen.
 - Anhand dieses Wertes entscheiden: Welche Verzweigung im Kontrollflussgraphen verfolgt wird bzw. welche Anweisung als nächste im Programm zur Ausführung kommt.
- **Problem**: Bedingung aus **mehreren Teilbedingungen** zusammengesetzt. → Über logische Operatoren miteinander verknüpft.
 - **Strukturelle Komplexität** der Bedingung im Test berücksichtigen.
- **Unterschiedliche Anforderungen und Abstufungen der Testintensität** mit Berücksichtigung der zusammengesetzten Bedingungen unterscheiden.

- **Idee:** Ausgehend von Zweigüberdeckung, Fehlerabdeckung verbessern, indem nicht nur Bedingungen insgesamt als true/false getestet werden, sondern auch **Teilbedingungen**.
- **Überdeckungskriterien:** Verhältnisse zwischen getesteten und maximal möglichen Wahrheitswerten der Teilbedingungen.

Im Folgenden:

- **(Einfache) Bedingungsüberdeckung** (*condition coverage*).
- **Mehrfachbedingungsüberdeckung** (*multiple condition coverage*).
- **Minimal bestimmende Mehrfachbedingungsüberdeckung** (*condition determination coverage*).

Einfache Bedingungsüberdeckung

Bedingungsüberdeckung: Anteil der (atomaren) Teilbedingungen durch Gruppe von Testfällen ausgeführt. 100% Bedingungsüberdeckung: Jede Teilbedingung in jeder Entscheidung mindestens einmal mit Werten True und False ausführen.

Einfacher Bedingungstest: White-Box-Testentwurfsvorgang. → Testfälle so entwerfen, dass Bedingungswege zur Ausführung kommen.

Überdeckung atomarer Teilbedingungen einer Entscheidung mit »wahr« und »falsch« erfordert: Teste **jeden atomaren Ausdruck einmal zu wahr und einmal zu falsch**. Bei n atomaren Ausdrücken mindestens 2, höchstens $2n$ Testfälle.

$$\text{Bedingungsüberdeckungsgrad} = \frac{\text{Anzahl zu wahr und falsch getesteten atom A}}{\text{Gesamtzahl atomarer Ausdrücke}}$$

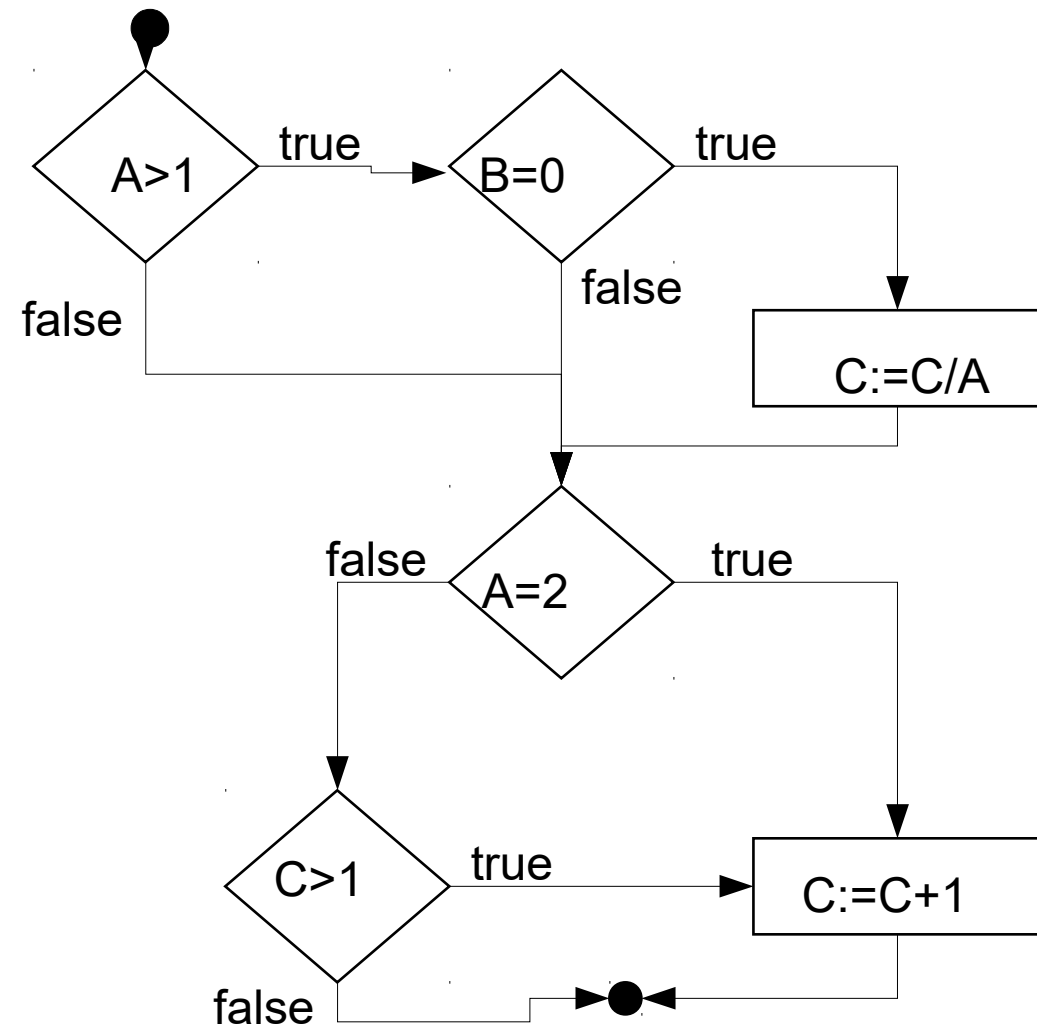
Achtung: Einfache Bedingungsüberdeckung: Schwächeres Kriterium als Anweisungs- oder Entscheidungsüberdeckung.

→ Berücksichtigung unterschiedlicher Wahrheitswerte bei Auswertung gesamter Bedingung im Test nicht verlangt.

**Beschränkung der Zweig- /
Bedingungsüberdeckung:**
Zweigüberdeckung ist nicht
„robust“ unter logisch
äquivalenter Umformung des
Kontrollflussgraphen.

Fehler, die auf einer nicht
getesteten Kombination der
Zweigbedingungen beruhen,
werden daher evt. nicht
gefunden.

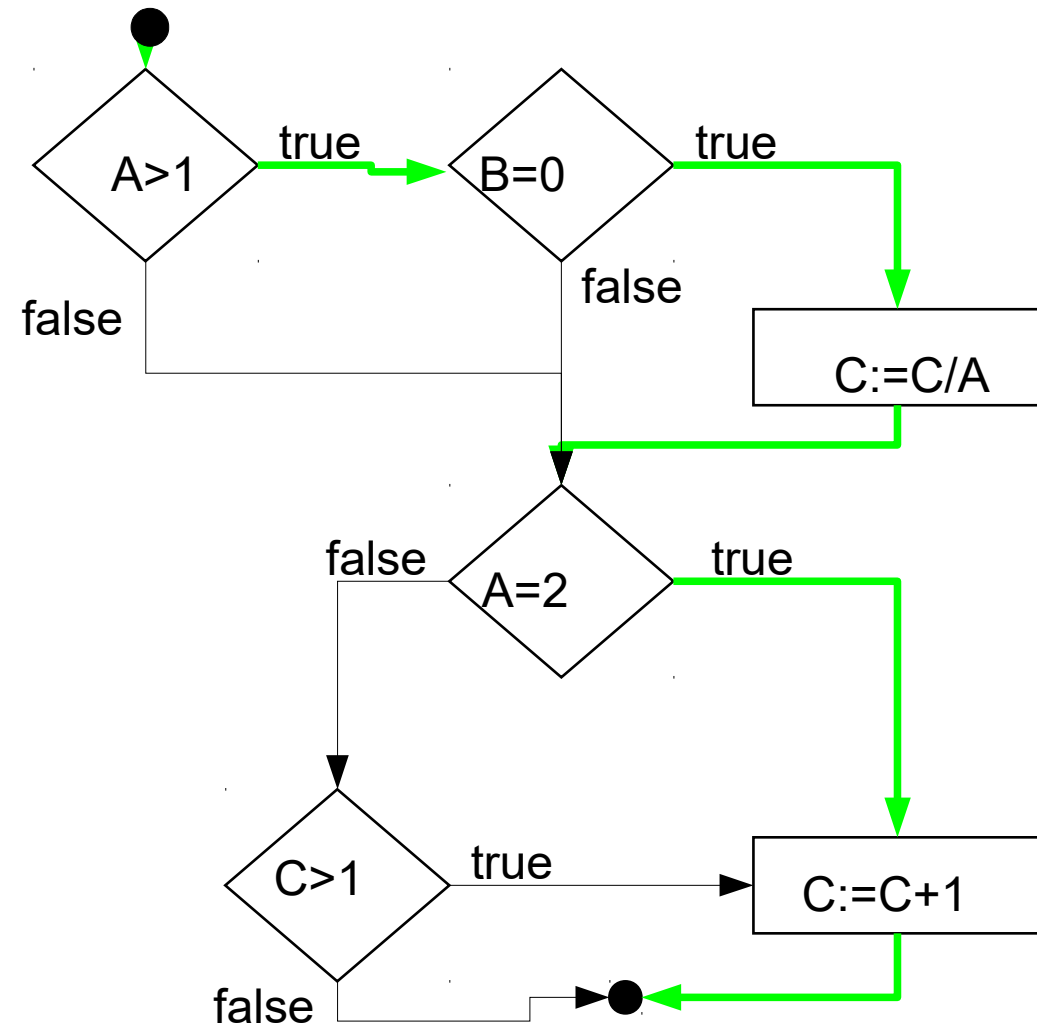
Betrachte nebenstehendes
Beispiel.



Zweig-/Bedingungsüberdeckung: Beschränkung - Beispiel

Beispiel für fehlende Überdeckung
von Zweigkombinationen nach
Umformung des
Kontrollflussgraphen.

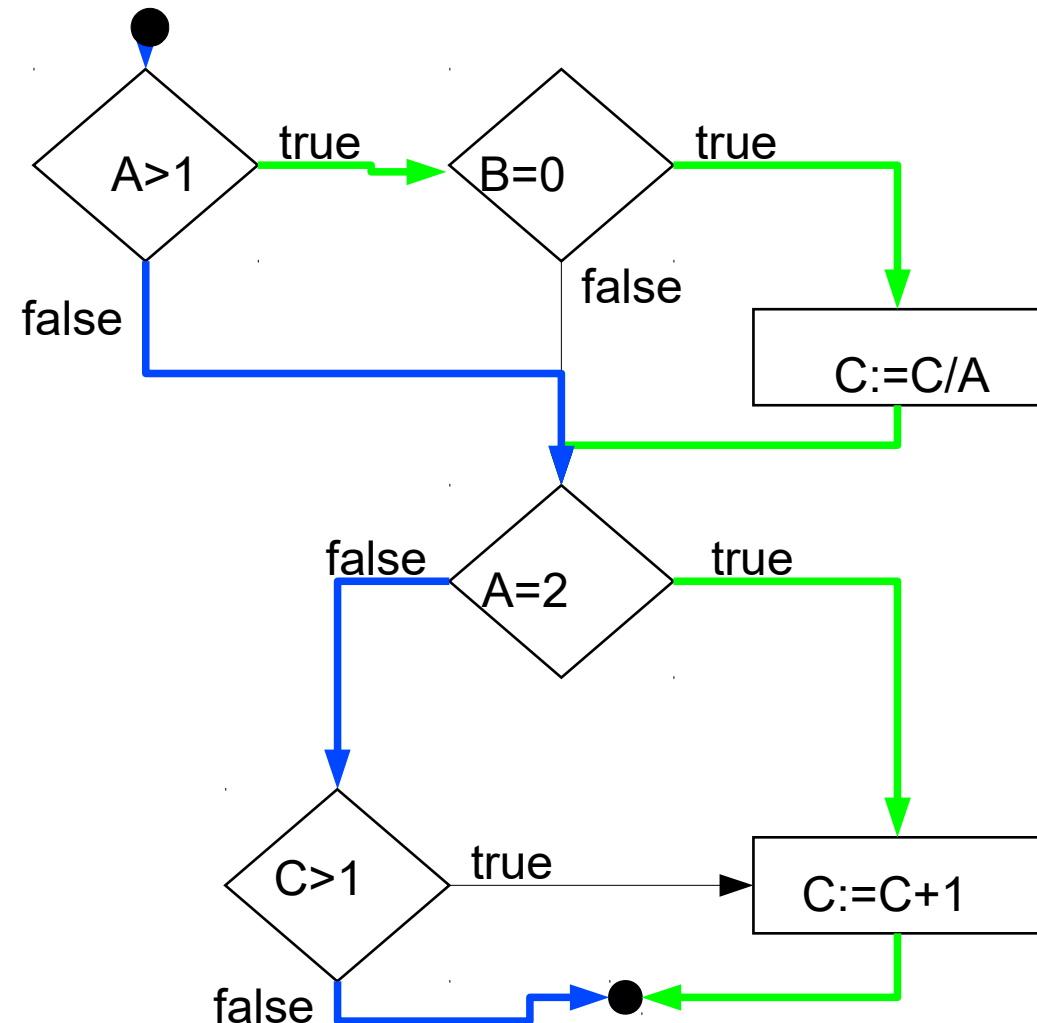
- Testdatum $A=2, B=0, C=4$



Zweig-/Bedingungsüberdeckung: Beschränkung - Beispiel

Beispiel für fehlende Überdeckung von Zweigkombinationen nach Umformung des Kontrollflussgraphen.

- Testdatum $A=2, B=0, C=4$
- Testdatum $A=1, B=1, C=1$



Mehrfachbedingungsüberdeckung (C_3 -Überdeckung):

X (A>1)	Y (B=0)	X and Y
false	false	false
false	true	false
true	false	false
true	true	true

Jeweils 2^n Variationen bei n
atomaren Prädikattermen.

X (A=2)	Y (C>1)	X or Y
false	false	false
false	true	true
true	false	true
true	true	true

Mehrfachbedingungstest: White-Box-Testentwurfsverfahren, das Überdeckung atomarer Teilbedingungen einer Entscheidung mit WAHR und FALSCH in allen Kombinationen fordert.

Testfall-Anzahl bei n atomaren Ausdrücken: 2^n .

- **Wächst exponentiell** mit Anzahl unterschiedlicher atomarer Ausdrücke !

$$\text{Mehrfachbedingungsüber.grad} = \frac{\text{Anzahl getesteter Kombinationen atom A}}{2 \text{ Gesamtzahl atomarer Ausdrücke}}$$

Bei **Auswertung** der Gesamtbedingung ergeben sich i.d.R. **beide Wahrheitswerte** (sonst: Tautologie !).

- **Erfüllt** somit Kriterien der **Anweisungs- & Entscheidungsüberdeckung**.
- **Umfassenderes Kriterium:** Berücksichtigt Komplexität bei zusammengesetzten Bedingungen.

Problem: Manche Kombinationen nicht durch konkrete Testfälle realisierbar:

- Wenn Teilbedingungen voneinander abhängig sind.
- Z.B. bei $(x > 0) \ \&\& \ (x < 5)$ ist (falsch, falsch) nicht realisierbar.

Minimal bestimmende Mehrfachbedingungsüberdeckung

Minimal bestimmende Mehrfachbedingungsüberdeckung: Anteil aller

Teste (Gesamt-)Ausdruck **einmal zu wahr und einmal zu falsch** sowie jede Kombination von Wahrheitswerten, bei denen **Änderung des Wahrheitswertes** eines atomaren Ausdrucks Wahrheitswert des zusammengesetzten Ausdrucks ändern kann!

Teste:

- (Gesamt-)Ausdruck **einmal zu wahr und einmal zu falsch** sowie,
- jede Kombination von Wahrheitswerten, bei denen **Änderung des Wahrheitswertes** eines atomaren Ausdrucks Wahrheitswert des zusammengesetzten Ausdrucks ändern kann (MM-Kombinationen) !

$$\text{Min. best. Mehrfachbed.üb.grad} = \frac{\text{Anzahl getesteter MM Kombinationen}}{\text{Gesamtzahl MM Kombinationen}}$$

100% minimal bestimmende Mehrfachbedingungsüberdeckung impliziert 100% Entscheidungsüberdeckung.

Gesamtzahl der MM-Kombinationen: Bei reinen and- bzw. or-Bedingungen mit n atomaren Ausdrücken nur n+1.

- ***REF(k)***: Menge der Variablen x , für welche Anweisungsfolge f , die zum Knoten k gehört, die Variable x referenziert, ohne dass x vorher in f undefiniert wird.

Berücksichtigen:

- Rein **lokale Datenflüsse** vermeiden (wenn intern Referenz auf Definition folgt).
→ Keine Berücksichtigung bei datenflussbezogenen Testkriterien.

Datenflussschema:

- Entsteht durch Ersetzen der Anweisungen und Entscheidungsprädikate durch formale Bezeichner.

Folgende Fragen sollten Sie jetzt beantworten können

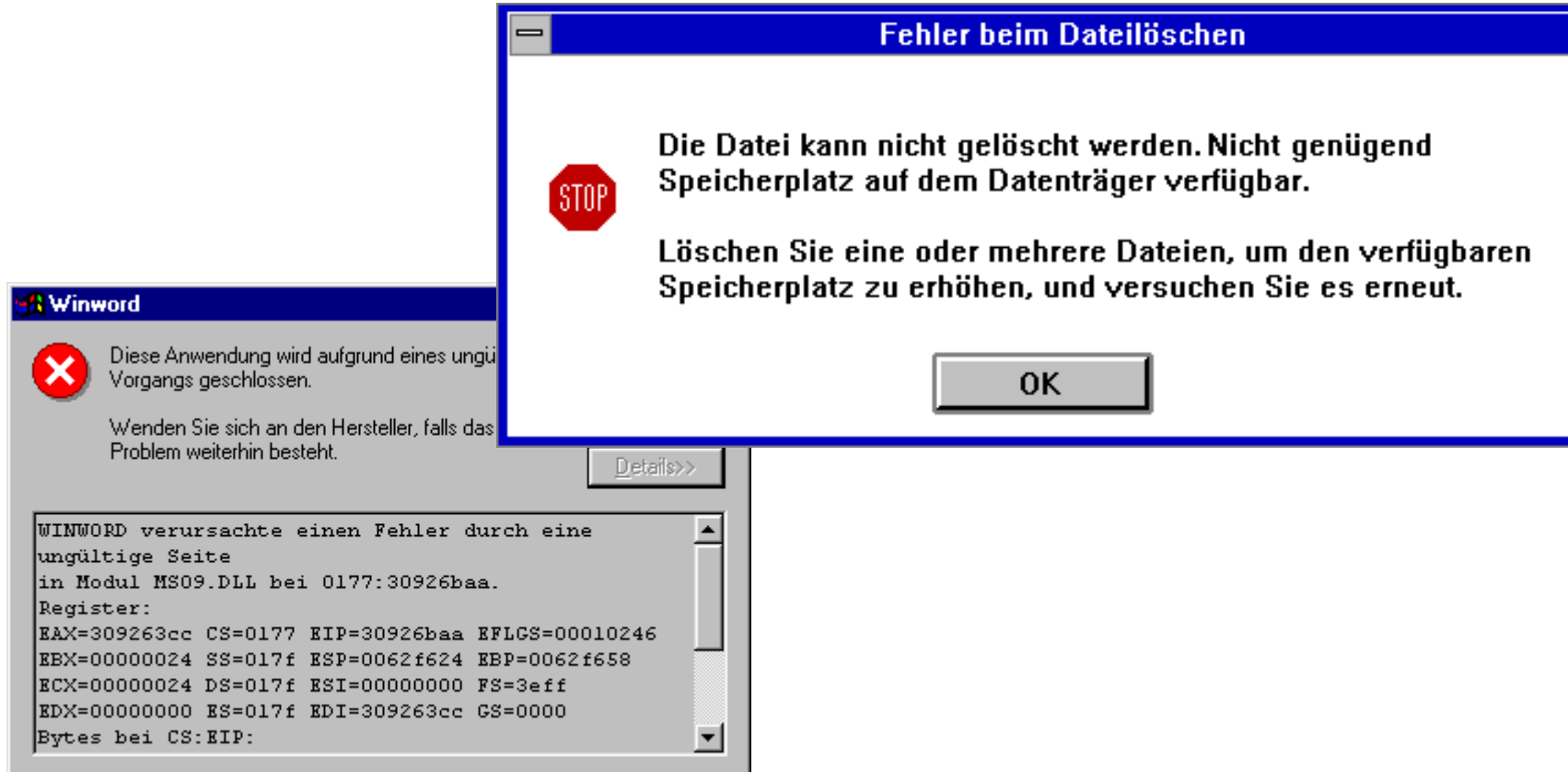
- Was bedeutet der Begriff Anweisungsüberdeckung ?
- Worin unterscheiden sich Anweisungs- und Entscheidungsüberdeckung ?
- Nach welcher Formel wird erreichte Anweisungsüberdeckung berechnet ?
- Wozu dient Instrumentierung ?
- Exkurs: Worauf zielt Bedingungsüberdeckung ab ?
- Exkurs: Worin unterscheiden sich einfache Bedingungsüberdeckung und Mehrfachbedingungsüberdeckung ?
- Was ist unter erfahrungsbasierten (exploratives) Testen zu verstehen ?



- **Grundidee der White-Box Testentwurfungsverfahren** (strukturorientierte Testentwurfungsverfahren) erläutern können.
- White-Box Testentwurfungsverfahren zur **Ermittlung von Testfällen** charakterisieren und voneinander abgrenzen können.
- **Anweisungsüberdeckung** und **Entscheidungsüberdeckung** für kontrollflussbasiertes Testen kennen und auf einfache Beispiele anwenden können.
- Ideen der **Grenze-Inneres-Überdeckung** und datenflussbasierten Testens kennen und erläutern können.
- Unterschiedliche Arten der **Bedingungsüberdeckung** kennen und auf einfache Beispiele anwenden können.
- Weitere Arten von White-Box Testentwurfungsverfahren aufzählen können.
- **Error Guessing** und **exploratives Testen** als erfahrungsbasierte Testentwurfungsverfahren kennen und charakterisieren können.
- **Dynamische Tests** bez. ihres Einsatzbereiches erläutern und mit anderen Testentwurfungsverfahren zu einer **Teststrategie** kombinieren können.

Diese Begriffe sollten Sie kennen...

Erfahrungsbasiertes Testentwurfsverfahren
(Fehler-)Angriff
White-Box-Testentwurfsverfahren
Kontrollfluss
Anweisungsüberdeckung



Bilder von <http://www.moffem.de/de/keller2/>

- **4.4 Strukturorientierte oder White-Box-Verfahren (K4)**
 - LO-4.4.1 Das Konzept und den Wert von Codeüberdeckung beschreiben können. (K2)
 - LO-4.4.2 Die Konzepte von Anweisungs- und Entscheidungsüberdeckung erklären können und Gründe nennen können, warum diese Überdeckungsmaße auch auf anderen Teststufen als im Komponententest eingesetzt werden können (z.B. bei Geschäftsprozessen auf Systemebene). (K2)
 - LO-4.4.3 Testfälle zu vorgegebenen Kontrollflüssen schreiben können unter Verwendung von Anweisungs- und Entscheidungsüberdeckungsverfahren. (K3)
 - LO-4.4.4 Vollständigkeit von Anweisungs- und Entscheidungsüberdeckung in Bezug auf definierte Ausgangskriterien überprüfen können. (K4)

- **4.5 Erfahrungsbasierte Verfahren (K2)**

- LO-4.5.1 Gründe wiedergeben können, warum Testfälle auf der Grundlage von Intuition, Erfahrung und Wissen über typische Fehlerzustände geschrieben werden sollten. (K1)
- LO-4.5.2 Erfahrungsbasierte Verfahren mit spezifikationsorientierten Testverfahren vergleichen können. (K2)

- **4.6 Auswahl von Testverfahren (K2)**

- LO-4.6.1 Testentwurfsverfahren in einem vorgegebenen Zusammenhang gemäß ihrer Tauglichkeit, für die Testbasis und bezüglich Modellen und Softwaremerkmalen klassifizieren können. (K2)

Polynom- bzw. Multinom-Kriterium

Beispiel-Ausdruck: **MID +1** (Grad $n=1$, eventuell fehlerhaft)

Nr.	korrekter Ausdruck	Bedingung für Testdaten
1	$k * MID + 1 \ (k \neq 1)$	$MID \neq 0$
2	$k * (MID + 1) \ (k \neq 1)$	$MID \neq -1$
3	$MID + c \ (c \neq 1)$	beliebig
4	$k * MID + c \ (k \neq 1)$	$MID \neq \frac{1-c}{k-1}$
5	$MID^2 + 1$	$MID \neq 0$ und $MID \neq 1$

Fehler bei 5 erfüllt **nicht** die Bedingung bzgl. Exponenterniedrigung, d.h. $n+1 = 2$ Testdaten (**MID=0 und MID=1**) reichen evtl. nicht !

Arithmetisches Relations-Kriterium

Beispiel-Ausdruck: $F > A[MID]$ (eventuell fehlerhaft)

Nr.	korrekter Ausdruck	Bedingung für Testdaten
1	$F = A[MID]$	$F = A[MID]$
2	$F \geq A[MID]$	$F = A[MID]$
3	$F \leq A[MID]$	$F = A[MID]$
4	$F < A[MID]$	$F = A[MID] + d, d > 0$
5	$F \neq A[MID]$	$F = A[MID] - d, d > 0$
6	$F > A[MID] + k, k \neq 0$	$F = A[MID] + d, 0 < d < k $ mit $d > 0$ g. d. w. $k > 0$

!! Obige Tabelle enthält Fehler bei Nr.1
($F=A[MID]+d, d \geq 0$), Nr.3 („beliebig“), Nr.4 ($d \neq 0$)
und Nr. 6 (auch $d=k$ erlaubt für $k > 0$ und $d=0$
erlaubt für $k < 0$)

Was ist **Nachteil** von einfacher Bedingungstest?

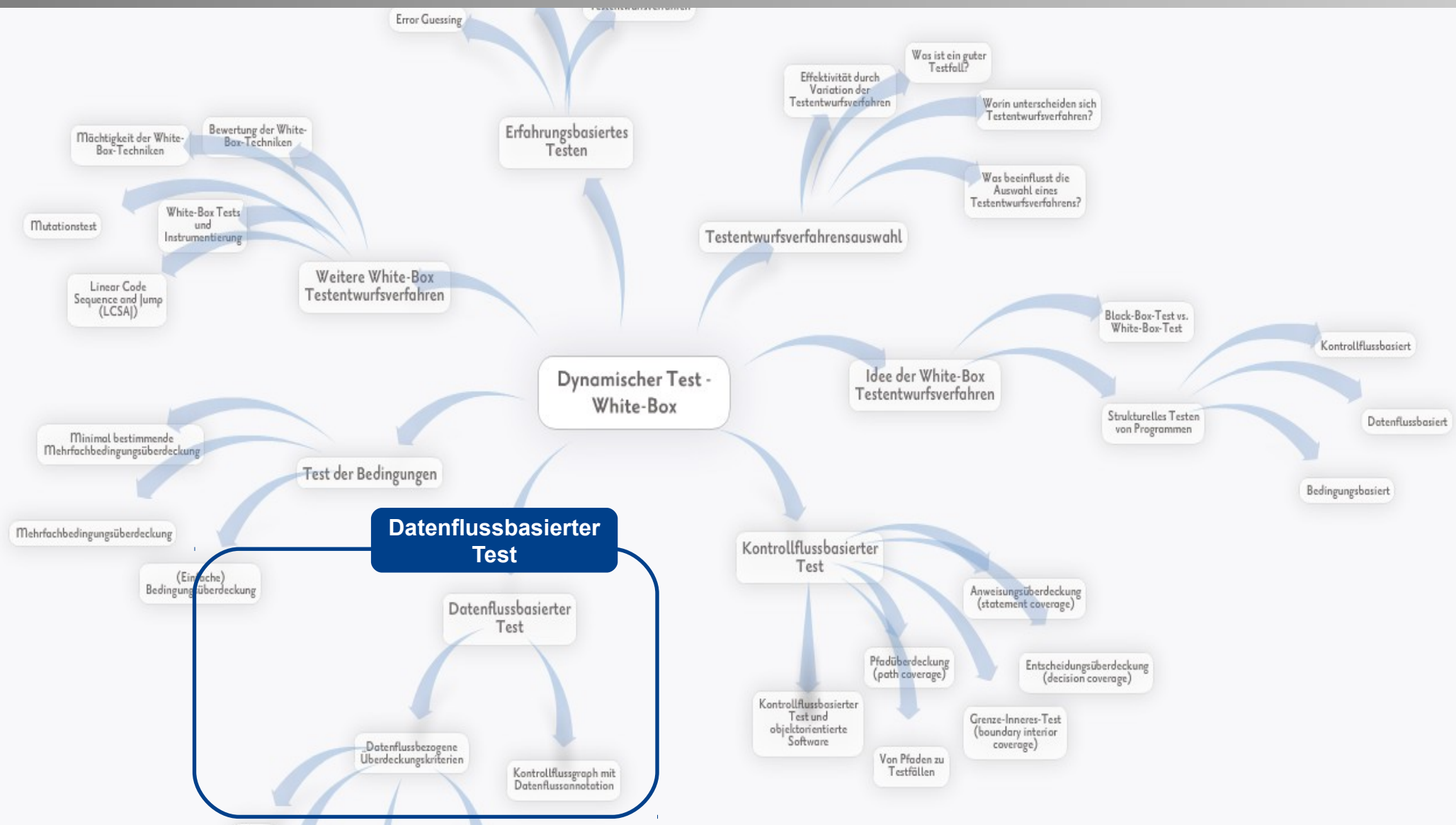
Antwort:

-
-

Was ist **Nachteil** von einfacher **Bedingungstest**?

Antwort:

- Prüft **boolesche Ausdrücke** nur **innerhalb IF-Anweisung**.
- Es wird nicht erkannt, dass IF-Entscheidung aus mehreren Bedingungen zusammengesetzt ist.



Auswahl eines Kontrollfluss-Testverfahrens

Liegt eine Prozedur, Funktion oder Operation einschließlich Quellcode vor?									
ja						nein			
... nur Anweisungen	... nur Anweisungen und atomare, nichtzusammengesetzte Bedingungen	Enthält die Prozedur, Funktion oder Operation ...				Black-Box-Testen			
		... nur Anweisungen, atomare, nichtzusammengesetzte Bedingungen und Schleifen	... nur Anweisungen und zusammengesetzte Bedingungen	... nur Anweisungen, zusammengesetzte Bedingungen und Schleifen					
Anwendungsüberdeckungstest	Zweigüberdeckungstest	Genügt es, Schleifen einmal zu wiederholen?		Reicht Überprüfung aller atomaren Bedingungen aus?				Genügt es, Schleifen einmal zu wiederholen?	
		ja	nein	ja	nein			ja	nein
		boundary interior-Pfadtest	strukturierter Pfadtest	Zweigüberdeckungstest und einfacher Bedingungsüberdeckungstest	minimaler Mehrfach-Bedingungsüberdeckungstest			boundary interior-Pfadtest	strukturierter Pfadtest
		Reicht die Überprüfung aller atomaren Bedingungen aus?							
ja		nein							
		einfacher Bedingungsüberdeckungstest	minimaler Mehrfach-Bedingungsüberdeckungstest						

Einige Werkzeuge für Überdeckungskriterien

Softwarekonstruktion
WS 2016/17



LEHRSTUHL 14
SOFTWARE ENGINEERING

BullseyeCoverage - The Authority in C++ Code Coverage

Java-Source.net - Open Source Code Coverage Tools in Java

Semantic Designs: Test Coverage - SEMANTIC DESIGNS, INC.

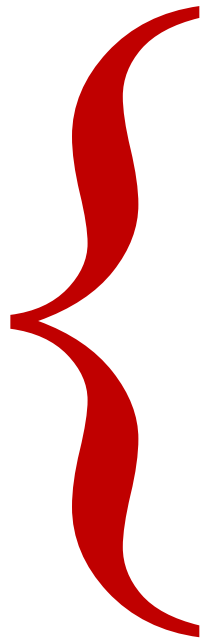
McCabeTest - Analyze the

C++ Test Coverage Analyzer

Counter	Coverage	Covered	Missed	Total
Instructions	65.9 %	924	478	1402
Branches	76.9 %	83	25	108
Lines	66.9 %	234	116	350
Methods	54.9 %	73	60	133
Types	94.4 %	17	1	18
Complexity	57.2 %	107	80	187

236

3.1 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Datenflussbasierter Test

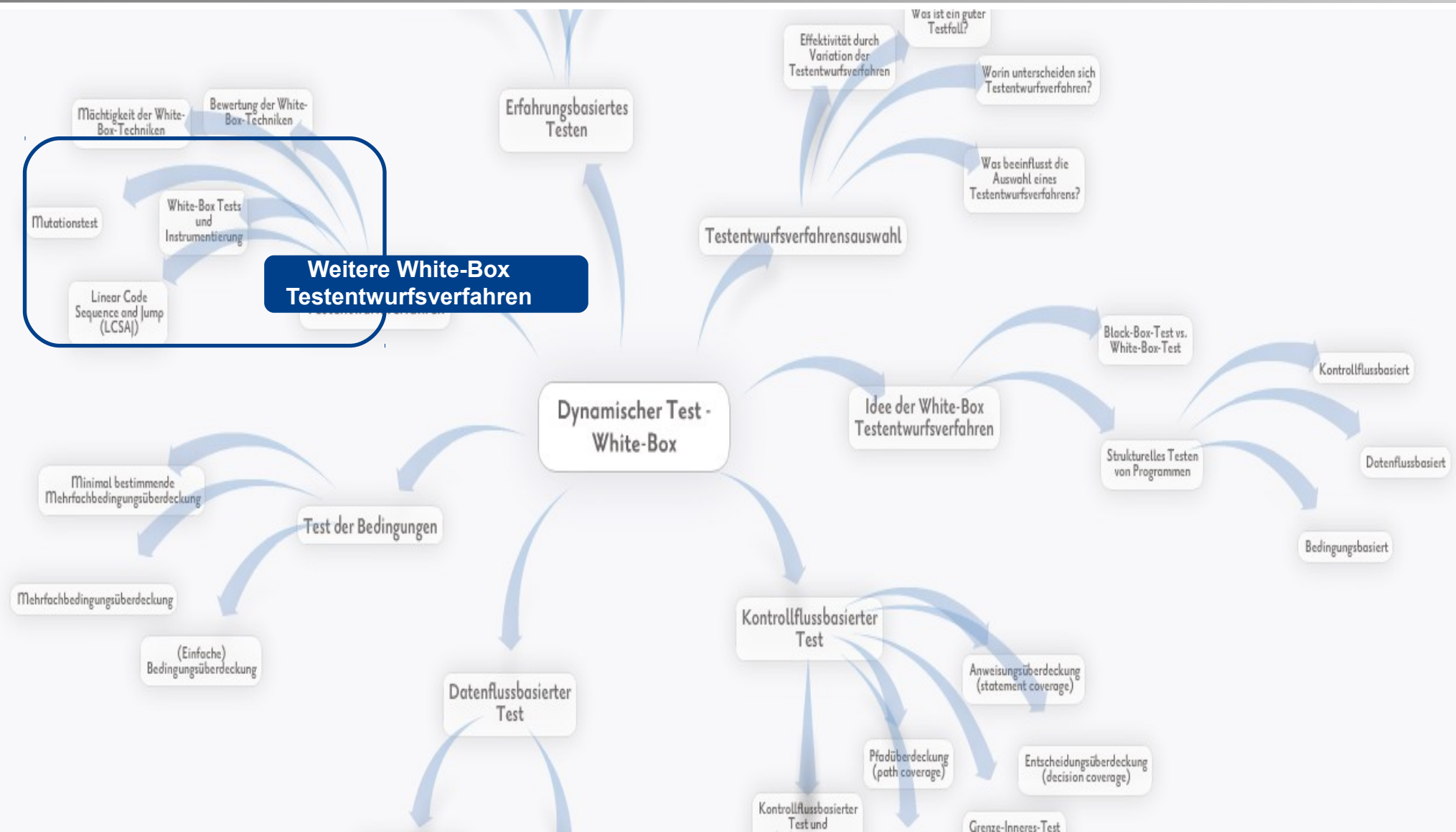
Test der Bedingungen

Weitere White-Box Testentwurfsverfahren

Erfahrungsbasiertes Testen

Dynamischer Test: Testentwurfsvorgangsauswahl
und Zusammenfassung

Weitere White-Box Testentwurfungsverfahren



Konzept: Mögliche Fehler in Ausdrücken und Anweisungen aufspüren.

Grobe Modellierung: **Anweisungen:**

- Zugriff auf Variablen (Datenzugriff).
- Speicherung in Variablen (Datenspeicherung).

Feinere Modellierung: **Ausdrücke** und **Relationen:**

- Arithmetische Ausdrücke.
- Arithmetische Relationen.
- Boolesche Ausdrücke (→ **bedingungs-basiertes Testen**).

Datenzugriff:

- Fehlerart: Zugriff auf falsche Variable.
- Testdaten: **Datenzugriffskriterium:**
 - Alle Variablen im Programmteil müssen vor Zugriff verschiedene Werte haben.

Datenspeicherung:

- Fehlerart: Speicherung in falsche Variable.
- Testdaten: **Datenspeicherungskriterium:**
 - Wird einer Variablen Wert zugewiesen, muss er anders als bisheriger Wert sein.

Arithmetische Ausdrücke:

- Mit Variablen, Konstanten und $+$, $-$, $*$, $/$ und $**$.
- Ohne „/“ ist es Polynom oder Multinom.

Fehlerarten:

- Einfache additive oder multiplikative Fehler.
- Fehler in Polynom oder Multinom, der Variablenmenge nicht ändert und höchsten Exponenten nicht erniedrigt.

Testdaten:

- **Additives/multiplikatives Fehler-Kriterium:**
 - Ausdruck muss Wert ungleich 0 erhalten. → Multiplikative Fehler entdecken.
- **Polynom- bzw. Multinom-Kriterium:**
 - Für Polynom vom Grad n : $n+1$ unabhängige Testdaten ausreichend.
 - Für Multinom mit höchstem Exponenten n : Kaskadenmenge von k -Tupeln vom Grad $n+1$ ausreichend, wenn k Anzahl Variablen (s. Howden 1978d).

- **Arithmetische Relation:**

- Seien A und B arithmetische Ausdrücke.
- Sei „r“ eines der sechs Relationssymbole $<, >, \geq, \leq, =, \neq$.

- **Fehlerarten:**

- Relation „A r B“ enthält falsches Relationssymbol r.
- Relation „A r B“ ist falsch, es müsste „(A+k) r B“ mit $k \neq 0$ realisiert werden.

- **Testdaten: Arithmetisches Relations-Kriterium:**

3 Testdaten notwendig und hinreichend, bei denen Differenz A-B folgende Werte annimmt (mit minimalem positivem Wert ϵ):

- $-\epsilon$ (größter negativer Wert – knapp unterhalb 0).
- 0.
- ϵ (kleinster positiver Wert – knapp oberhalb 0).

Konzept: Betrachtung der Zusammensetzung von (Eingabe- und Ausgabe-) **Daten** auf verschiedenen **Abstraktionsstufen**.

- **Datenkapsel** (zusammenhängende Daten, z.B. Tabelle, Klasse, struct).
- **Datenfelder** (aus denen Datenkapsel besteht).
- **Repräsentative Werte pro Datenfeld** (ergeben sich aus Spezifikation, z.B. Äquivalenzklassen, Grenzwerte).
- **Datenzustände** (alle möglichen Kombinationen von allen repräsentativen Werten für alle Felder einer Datenkapsel).

Testdaten für Datenüberdeckungen:

- **Datenkapselüberdeckung:** Durch Test min. ein Ein- oder Ausgabefeld jeder Datenkapsel „ansprechen“ (d.h. einlesen bzw. ausgeben).
- **Feldüberdeckung:** Durch Test jedes Ein- und Ausgabefeld jeder Datenkapsel „ansprechen“.
- **Überdeckung repräsentativer Werte:** Durch Test jeden repräsentativen Wert jedes Ein- und Ausgabefeldes jeder Datenkapsel „ansprechen“.

Testverfahren unterscheiden sich nach Aufwand, aber wie genau?

→ Fehleraufdeckungsfähigkeit:

- **Datenzugriffskriterium:** 79% aller Bereichsfehler (**100%** bei falsch platzierter Anweisung).
- **Arithmetisches Relationskriterium:** 100% aller Bereichsfehler durch fehlerhafte arithmetische Operatoren, falsche konstante Werte und falsch platzierte Anweisungen, aber nur **70%** aller fehlerhaften relationalen Operatoren.
- Kriterium **additive / multiplikative Fehler:**
 - **56%** von fehlenden Berechnungen,
 - **13%** falsche konstante Werte,
 - **11%** fehlerhafte Variablenreferenzen;
 - **10%** falsche relationale Operatoren und
 - **alle anderen Fehler gar nicht.**
- **Keine** Fehler, die **nur** mit obigen Methoden gefunden wurden !

[Quelle: Riedemann: Spezialvorlesung Test-Methoden.]

Programme testen, die in älteren Programmiersprachen vorliegen.

→ Testobjekte können **Sprünge** enthalten.

- Bisher vorgestellte Testentwurfungsverfahren berücksichtigen Test der Sprünge **unzureichend**.

»**Linear Code Sequence and Jump**« (LCSAJ):

Folgen von Anweisungen:

- Im Mittelpunkt der Untersuchung.
- **Sequentiell ausführbar** und durch Sprung auf andere Anweisung als nächst folgende Anweisung beendbar.
 - Kombinationen linearer Codesequenzen mit Sprung am Ende beim Test berücksichtigen.
 - **Prüft** durch Kombination der Sequenzen **mehr als Entscheidungsüberdeckung**.
 - **Nicht umfangreich** wie Pfadüberdeckung.

- Mit Black- oder White-Box Testentwurfsverfahren Testfälle erstellen und soll über deren Güte, d.h. **Wahrscheinlichkeit Fehler aufzudecken**, aussagen.
 - **Fehler im Programm einbaubar** und prüfbar, ob Testfälle Fehler finden.
- »Mutationstest« systematisiert Idee durch Definition von **Mutationsoperatoren**: Programme automatisch veränderbar:
 - Mathematische Operatoren durch inverse ersetzen ($+$ $\rightarrow$ $-$, $*$ $\rightarrow$ $/$).
 - Logische Operatoren in Bedingungen durch inverse ersetzen.
 - Variablenbezeichner vertauschen.
- **Messung**: Welche Testfälle »killen« welche Mutanten? → Min. einen Fehler im Mutanten entdecken.
- Mengen von Testfällen: »gut«, wenn sie alle Mutanten »killen«.

Programme **weichen geringfügig** vom vorgesehenen Verhalten ab.

→ Bestehen aus **einfachen Fehlern** wie Definition oder Referenz falscher Variablen.

Ziel:

- Erzeugung von **mehreren Versionen** eines Programms P, die jeweils Abweichung repräsentieren.
- Spezifikation einer Testdatenmenge T, die diese **Abweichungen in Programmversionen entdecken** kann.
 - **Korrespondierende Fehler** im ursprünglichen Programm P erkennbar.

Erzeugung einer **Menge von Programmen** $P_1, \dots, P_n$.

→ Unterscheiden sich „in einer Kleinigkeit“. P_i : Mutanten von P .

Mutanten durch **Mutationstransformationen** erzeugbar, z.B.

- **Referenz** einer anderen Variablen.
- **Definition** einer anderen Variablen.
- **Verfälschung arithmetischer Ausdrücke** um multiplikative oder additive Konstanten oder Veränderung von Koeffizienten.
- **Änderung arithmetischer Relationen** um additive Konstanten oder Verfälschung der Relation.
- **Verfälschung boolescher Ausdrücke.**

Anwendung der Tests t aus T auf jede Mutante P_i $i=1, \dots, n$ solange und Ergebnisse $P_i(t)$ mit Ergebnis $P(t)$, **bis Mutante „tot“ ist.**

- **Mutante P_i : lebendig** (bzgl. T und P) g.d.w. $P_i(t) = P(t)$ für alle Tests t aus T .
- **Mutante P_i : tot** (bzgl. T und P) g.d.w. $P_i(t) \neq P(t)$ für mindestens einen Test t aus T .



Wie stark **wächst** die **Zahl der notwendigen Mutanten** und Testdaten mit der **Programmgröße** an?

Antwort:

-

Wie stark **wächst** die **Zahl der notwendigen Mutanten** und Testdaten mit der **Programmgröße** an?

Antwort:

- Zahl der Mutanten **maximal quadratisch** mit der Zahl der Variablenreferenzen, Zahl der Testdaten etwa linear.

Programm P:

```
int grow(int* input){  
    int j, result;  
    Result = 0;  
    for (j =1; j < 3; j++) {  
        if (input[j] > input[result])  
            result = j;  
    }  
    return result;  
}
```

Wie könnte ein Mutant bspw. aussehen ?

Programm P:

```
int grow(int* input){  
    int j, result;  
    Result = 0;  
    for (j =1; j < 3; j++) {  
        if (input[j] > input[result])  
            result = j;  
    }  
    return result;  
}
```

Mutant P':

```
int grow(int* input){  
    int j, result;  
    Result = 0;  
    for (j =1; j < 3; j++) {  
        if (input[j] >= input[result])  
            result = j;  
    }  
    return result;  
}
```

Resultat:

- Mutationsanalyse-Ergebnis: **Alle Mutanten tot.** → Erhöht Zuversicht, dass Testdatenmenge T „entsprechende“ Fehler im Programm P findet.

Grenzen des Verfahrens:

- Mutant und Originalprogramm äquivalent. → Keine Existenz von Testfällen, die Mutanten töten.
- **Lebendige Mutante P_i :** Zum Programm P äquivalent. → Für alle Eingabedaten e gilt $P_i(e) = P(e)$.
- Beispiel:

```
int max(int* input){
    int j, result;
    Result = 0;
    for (j = 0; j < 3; j++) {
        if (input[j] > input[result])
            result = j;
    }
    return result;
}
```

Grenzen des Verfahrens ff.:

- Geeignet **viele Mutanten erzeugen**, die alle „typischen“ Fehler enthalten.
- Zweifelhaft, ob Kopplungseffekt immer gilt:
 - **Kopplungseffekt:** Testdaten, die einfache Fehler aufdecken, entdecken auch große Fehler..

- **Betrachtung einzelner Komponenten** eines Mutanten und nicht des gesamten.
- **Forderung:** Direkt nach Ausführung fehlerhafter Anweisung Vorlage eines fehlerhaften Programmzustands.
- **Nicht warten** bis Fehler zur Programmausgabe fortpflanzt.

Beim Datenzugriff:

- Fehlerart: Zugriff auf falsche Variable.
- Testdaten: So wählen, dass alle Variablen im Programm vor Zugriff verschiedene Werte haben.

Bei Datenspeicherung:

- Fehlerart: Speicherung eines Wertes in falscher Variablen.
- Testdaten: So wählen, dass eine Variable anderen Wert zugewiesen bekommt als bisher.

Bei arithmetischen Ausdrücken:

- Ausdruck besteht aus **Operanden** (Variablen, Konstanten) und **Operatoren** (+, -, *, /, **).
- **Fehlerart:** Einfache additive oder multiplikative Fehler.
- **Testdaten:** Betroffener Ausdruck: Wert ungleich 0 erhalten, um multiplikative Fehler zu finden. Additive Fehler wirken sich immer aus.
- Z.B. statt Ausdruck $k \cdot x + 1$ wird fälschlicherweise Ausdruck $x+1$ verwendet ($k \neq 1$).

x	$k \cdot x + 1$	$x+1$
0	1	1
1	$k + 1$	2

Bei arithmetischen Ausdrücken ff.

- **Fehlerart:** Fehler im Polynom oder Multinom, der Variablenmenge nicht ändert und höchsten Exponenten im Ausdruck nicht erniedrigt.
- **Testdaten:**
 - Polynom vom Grad n : $n+1$ unabhängige Testdaten.
 - Z.B. statt Ausdruck x^2+1 wird fälschlicherweise Ausdruck $x+1$ verwendet.
 - $(2+1)$ Testdaten finden diesen Fehler:

x	$x^2 + 1$	$x + 1$
0	1	1
1	2	2
2	5	3

- Multinom: hier nicht betrachtet

Schwache Mutationsanalyse:

Zur Aufdeckung von Fehlern (4)

Bei arithmetischen Relationen:

- A, B arithmetische Ausdrücke und r eines der Relationssymbole $<$, $=$, $>$, $\leq$, $\geq$, $\neq$.
- **Fehlerarten:**
 - Relation der Form „A r B“ enthält falsches Relationssymbol.
 - „A r B“ statt „(A+k) r B“ verwendet.
- **Testdaten:** 3 Testdaten, bei denen Differenz A-B größten negativen Wert, Wert 0 und kleinsten positiven Wert annimmt:
 - Z.B. statt Ausdruck $x=y$ fälschlicherweise Ausdruck $x>y$ verwendet.

x	y	$x>y$	$x=y$
0	ε	F	F
1	1	F	T
ε	0	T	F

Schwache Mutationsanalyse:

Zur Aufdeckung von Fehlern (5)

Bei booleschen Ausdrücken:

- S. Bedingungsüberdeckung.

Bei allgemeinen Ausdrücken:

- **Fehlerart:** Überflüssige Teilausdrücke (Kriterium „kürzere Ausdrücke“).
- **Testdaten:** Für jeden Ausdruck und jeden seiner Teilausdrücke: Testdatum, das diesen Ausdruck ausführt, so dass bei Ausführung (Teil-)Ausdruck verschiedene Werte annehmen. → Überflüssigen Rest erkennen.
 - Beispiel: für Ausdruck $x*(x-2) + (x-1)$ Testdaten $x=0,1$ oder 2 nicht geeignet.

x	$x*(x-2)$	$(x-1)$	$x*(x-2) + (x-1)$
0	0	-1	-1
1	-1	0	-1
2	0	1	1

Bei allgemeinen Ausdrücken ff.:

- **Fehlerart:** Vertauschte Teilausdrücke (Kriterium „alle kürzeren Ausdrücke“).
- **Testdaten:** Für jeden Ausdruck und jeden seiner Teilausdrücke: Testdatum, das diesen Ausdruck ausführt und bei dessen Ausführung alle (Teil-)Ausdrücke im Programm verschiedene Werte haben. → Auffallen von Vertauschungen.

Vorteil: Fehler-Orientierung der Methode.

Probleme / Nachteile:

- **Viele Mutanten** erzeugen, die alle **typischen Fehler** abdecken.
- **Kopplungseffekt** müsste gelten:
Testdaten killen **einfache** Mutanten (mit Abweichung).
→ Testdaten killen auch **komplexe** Mutanten (mit Menge von Abweichungen).
- **Hoher Testaufwand** (ca. 50%) zum Killen **letzter 10%** nicht äquivalenter Mutanten.
- **Regressionstests** aufwändig.
- **Äquivalente Mutanten nicht** killtbar und **Äquivalenzproblem nicht** entscheidbar !
→ **Mutationstest nicht vollständig automatisierbar !**

Forderung bei White-Box Testentwurfsv erfahren:

- Ausführung bestimmter Programmteile bzw.
- Annahme unterschiedlicher Wahrheitswerte der Bedingungen.

Auswertung des Tests: Ermittlung ausgeführter und nicht zur Ausführung gekommener Programmteile.

Testobjekt an strategisch wichtigen Stellen vor Testausführung instrumentieren:

- Zusätzlich Anweisungen wie z.B. Zähler einbauen und mit Null initialisieren. → Beim Durchlauf an entsprechenden Stellen inkrementieren.
- Am Ende der Testläufe enthalten Zähler Anzahl von Durchläufen durch jeweilige Programmteile.
- Zähler = 0: Entsprechende Programmteile nicht ausgeführt.

Instrumentierung größerer Programme nur mit Werkzeug sinnvoll !

Welche **Anforderungen** werden durch Whitebox-Verfahren **aufgedeckt**?

Antwort:

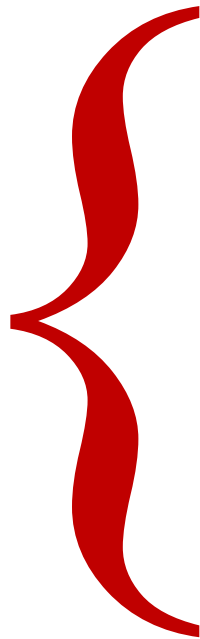
-
-

Welche **Anforderungen** werden durch Whitebox-Verfahren **aufgedeckt**?

Antwort:

- Anforderungen, die **im Programm umgesetzt worden sind**, werden bei Whitebox-Verfahren überprüft.
- Zur Aufdeckung von fehlenden Programmteilen: Andere Testverfahren einsetzen.

3.1 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Datenflussbasierter Test

Test der Bedingungen

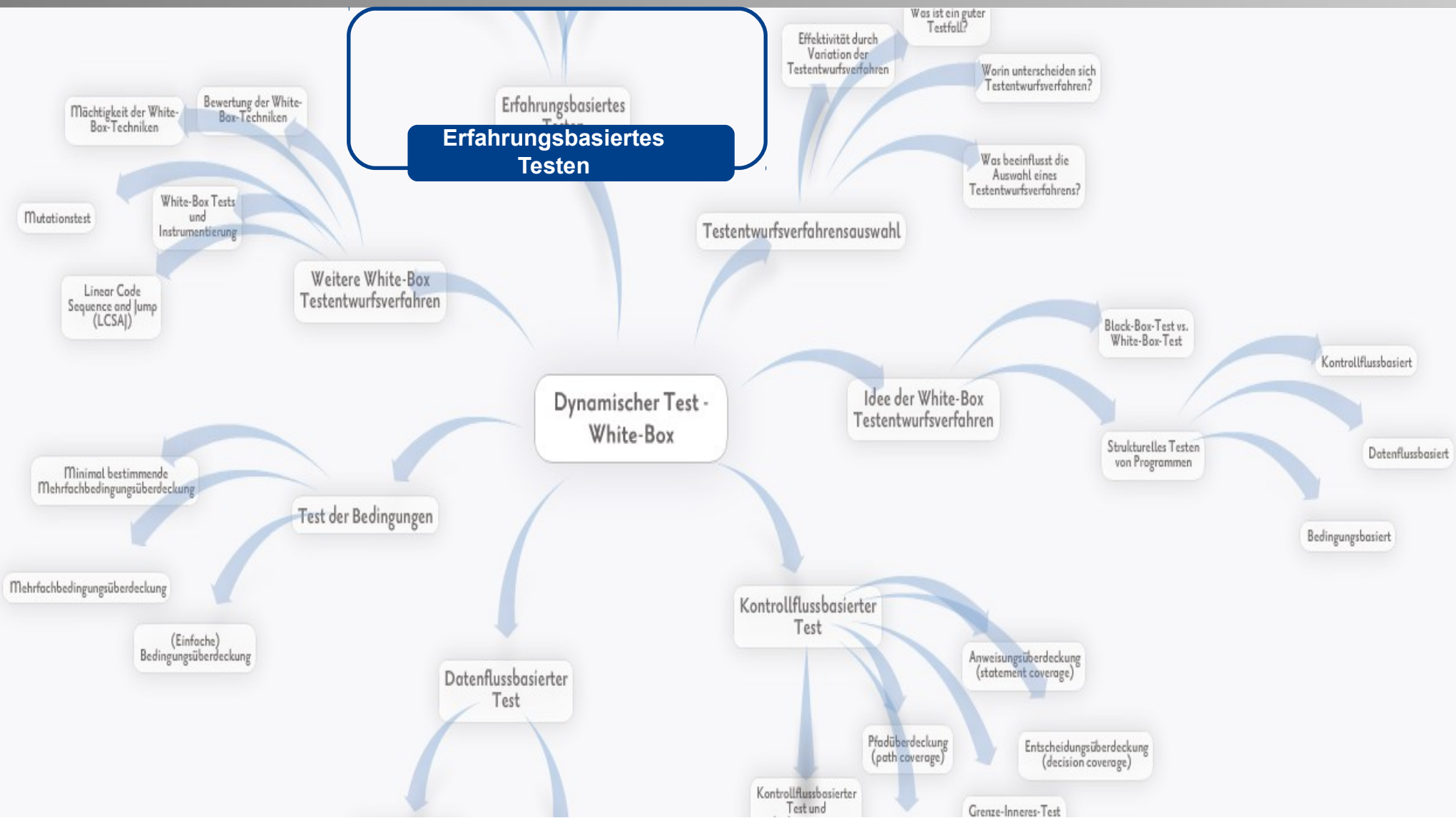
Weitere White-Box Testentwurfsverfahren

Erfahrungsbasiertes Testen

Dynamischer Test: Testentwurfsvorgangsauswahl
und Zusammenfassung

Dynamischer Test – Erfahrungsbasiertes Testen

Softwarekonstruktion
WS 2016/17



Erfolg systematischer Testentwurfsverfahren: Auf Grundlage **Qualität der Dokumente** Testfälle definieren.

- **Qualität:** Lesbarkeit, Testbarkeit, Vollständigkeit, Eindeutigkeit, Widerspruchsfreiheit...
- **Formalisierungsgrad und Granularität nicht adäquat**, um formale bzw. systematische Testentwurfsverfahren anzuwenden.

Grundidee:

- **Wissen** der Tester bei Definition von Testfällen einbeziehen.
- **Kompensation mangelnder Qualität** der Eingangsdokumentation durch Erfahrung und Wissen der Tester.
- **Definition von Testfällen:** Schwer durch systematische Testentwurfsverfahren ableitbar.

Ergänzung bisher vorgestellten systematischen Testentwurfsverfahren.

- **Beispiele:** Error Guessing (intuitives Testen) und exploratives Testen.

1. Testentwurfsverfahren mit Nutzung von **Erfahrung und Wissen der Tester**.
→ Vorhersage:

- Welche Fehlerzustände kommen in Komponente oder System aufgrund Fehlhandlungen vor.
- **Testfälle ableiten** um diese Fehler aufzudecken.

2. Methode und Herleitung oder Auswahl der Testfälle, gestützt auf Erfahrung und Wissen des Testers.

Am weitesten verbreitete Testentwurfsverfahren (intuitive Testfallermittlung).

Tests durch **Können und Intuition des Testers**, aus seiner Erfahrung mit ähnlichen Applikationen und Technologien, ableiten.

Systematische Vorgehensweise:

- **Fehlerkatalog** mit möglichen Fehlerzuständen und -wirkungen erstellen.
- **Testfälle**, die auf diese Fehler abzielen, entwerfen.

- Einsetzung zur **Unterstützung systematischer Testentwurfsv Verfahren.**
- Ermittelt von systematischen Testentwurfsv Verfahren nicht/schwer erfassbare Tests.
- **Vorsicht:** Unterschiedliche Grade von Effizienz erreichbar.
→ Abhängig von **Erfahrung des Testers.**
- Strukturierte **Herangehensweise:**
 - **Liste möglicher Fehler** erstellen und Testfälle entwerfen, die auf diese Fehler abzielen.
 - Durch Erfahrung, verfügbaren Daten über Fehlerzustände, Fehlerwirkungen und Allgemeinwissen erstellbar, warum Software sich falsch verhalten kann.
 - **Nutzen für Entwickler:** Hinweis auf evtl. Probleme vor Implementierung, während Implementierung berücksichtigen.
→ **Fehlervermeidung.**

Liste mit möglichen Fehlern und fehlerverdächtigen Situationen:

- Umfangreiches Erfahrungswissen oft in Köpfen erfahrener Tester.
- Erfahrungen über wieder auftretende Fehler vermerken und allen Testern zur Verfügung stellen.

Zahlreiche publizierte Fehlerlisten:

- Cem Kaner, Jack Falk, & Hung Quoc Nguyen, Testing Computer Software: enthält eine Liste von über 400 Fehlern
- James A. Whittaker: How to Break Software: A Practical Guide to Testing: enthält mögliche „Angriffe“, die zu einem Fehler führen können
James A. Whittaker: How to Break Software Security: enthält mögliche „Sicherheits-Angriffe“
- Mike Andrews and James A. Whittaker: How to Break Web Software: Functional and Security Testing of Web Applications and Web Services
- Greg Hogg and Gary McGraw: Exploiting Software: How to Break Code

Problem: Aktualität, Fehlerlisten nicht domänen- und produktspezifisch.

Anpassung allgemeiner Fehlerlisten und **Definition und Pflege** eigener Fehlerkataloge immer notwendig.

- **Informelles Testen:** Keine Testvorbereitung und keine Verwendung erkennbarer Testentwurfsverfahren.
- **Keine Spezifikation** erwarteter Ergebnisse **vorab** und willkürliche Testdurchführung.
- **Gleichzeitige Testfallanalyse** und -entwurf, Testrealisierung und -durchführung, Testprotokollierung und Lernen.
 - **Grundlage:** Test-Charta, der Testziele und mögliche Testideen entnimmt.
 - Durchführung innerhalb **festgelegter Zeitfenster**.
 - **Gut geeignet:**
 - Wenig oder ungeeignete Spezifikationen, unter hohem Zeitdruck zu testen,
 - Andere, formälere Testentwurfsverfahren zu unterstützen oder zu ergänzen.
 - **Exploratives Testen:** Zur Überprüfung des Testprozesses dienbar.
→ **Lücken aufspüren**, falls dabei Fehler findbar.

Eigenschaften:

- **Paralleler Testentwurf und Testdurchführung:** Entwicklung von Strategien zur Untersuchung des Produktes, Definition und Ausführung von Testfällen.
- **Überprüfbare Ergebnisse:** Ergebnisse aus vorherigen Tests dokumentieren, auswerten. → Beeinflussen nächste Tests.
- **Exploration:** »Kennenlernen« des Produktes (Funktionen, verarbeitete Daten, unreife Produktbereiche, ...).
- **Heuristiken:** Leitfäden und Faustregeln, erleichtern Auswahl von Testfällen.

Aktivitäten:

- Testvorbereitung.
- Testdurchführung.
- Dokumentation.
- Testauswertung.

Einheiten:

Abgeschlossene Teile des Produktes

Testaufwand 1-2 Tage

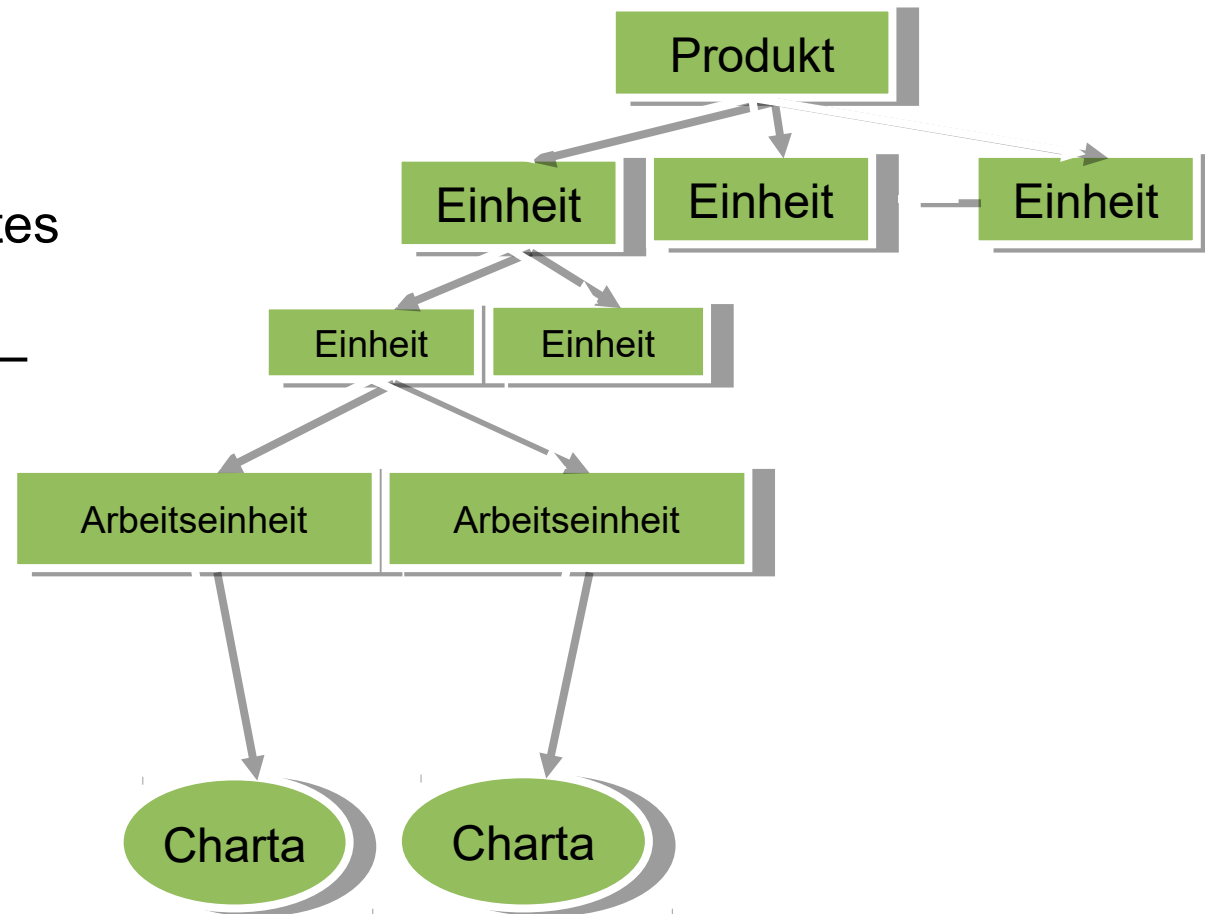
Arbeitseinheit:

Abarbeiten innerhalb einer Sitzung
„an einem Stück“ möglich

Testaufwand 1-2 Stunden

Charta:

Gibt Ziel und Aufgabe für
die Sitzung vor



Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach

Gibt klares Ziel und Aufgabe einer Sitzung vor:

- Warum soll entsprechende Einheit getestet werden ?
- Was soll getestet werden ?
- Wie getestet werden soll (Testentwurfsverfahren) ?
- Welche Probleme sollen adressiert werden ?

Zusatzinformationen, die enthalten sein können:

- Werkzeuge, die benutzt werden können
- Explorationsstrategien
- Dokumente, die einbezogen werden müssen

Kein detaillierter Testplan !

Am Anfang: Allgemeine Test-Chartas zur Überprüfung und zum Kennenlernen der Grundfunktionalität.

- Beispiel: “Analysiere Grafik-Einfüge-Funktion eines Textverarbeitungsprogramms”

Später: Detaillierte Test-Chartas zum detaillierten Test der entsprechenden Funktionalität, umfassen auch Test von Qualitätsanforderungen wie beispielsweise Performanz oder Usability.

- Beispiel: “Teste das Einfügen eines Cliparts”, “Teste das Einfügen eines Bildes aus einer Datei”.

Zeitlich begrenzte Sitzung:

- Abgeschlossene Arbeitseinheit.
- Zielgerichtet: Durchführung der in Charta „geplanten“ Tests.
- Dauer: Kurz um flexibel auf Testergebnisse zu reagieren; lang um abgeschlossenen Teilbereich des Produktes zu testen und um aussagekräftige Auswertungen zu erzeugen.
- Kurz: 60 Minuten, Normal: 90 Minuten, Lang: 120 Minuten
- Ohne Unterbrechung (z.B. Email, Telefon, usw..).
- Überprüfbar (Ergebnisse jeder Sitzung werden protokolliert).

Während Sitzung:

- Ziel nicht aus Augen verlieren.
- Notizen während Durchführung: Um später nachvollziehen zu können, welche Tests warum durchgeführt wurden und um Testauswertung zu ermöglichen.

Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach

Dokumentation:

Chartas: Geben Überblick über das zu testende Objekt.

Sitzungsprotokolle erlauben Auswertung und Fortschrittsüberwachung:

- Beginn der Sitzung.
- Charta, getestete Einheit(en).
- Tester.
- Dauer gesamt, für Vorbereitung, Testdurchführung, Fehleranalyse, Dokumentation.
- Testdaten.
- Notizen (Was habe ich gemacht? Warum? Hinweise, Fragen, Anomalien, aufgetretene Schwierigkeiten, ...).
- Fehlerbericht (adäquate Granularität, um Tests wiederholen zu können).

Auswertung:

- Nach jeder Sitzung: Ergebnisse der Sitzung auswerten und mit Testmanager besprechen.
- Auswertungsergebnisse und Erfahrungen aktueller Sitzung fließen in nächste Sitzung ein.

Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach

Explorationsstrategien: Helfen, Produkt zu analysieren und Testfälle auszuwählen. **Zahlreiche »Muster«:**

- Intuition (zufälliges Ausprobieren, Suche nach ähnlichen Fehlern, Funktionen, ...).
- Modelle (mental oder explizit modelliert, UML, ...).
- Beispiele (realistische Benutzungsszenarien, User Stories, ...).
- Interferenz (Störung des „normalen“ Ablaufs, Hardwarefehler, ...).
- Fehleranalyse (Schritte weglassen, alternative Schritte ausführen, Konfiguration ändern, ...).
- Gruppenwissen (pair testing, brainstorming, ...).
- Aktives Lesen (Mehrdeutigkeiten, Unvollständigkeiten, ...).

Heuristiken: Systematische Gewinnung neuer Erkenntnisse auf Basis von Erfahrung. Nützlich beim explorativen Testen.

Achtung: Keine Garantie, dass sie zur richtigen Lösung führen. **Daher:** Kontext und Gründe hinter Heuristik vor Anwendung verstehen.

Beispiele: Heuristiken zur Identifikation von potenziellen Fehlern.

- Neue Funktionalität fehleranfälliger als reife Funktionalität.
- Geänderte Funktionalität fehleranfälliger als reife Funktionalität.
- Späte Änderungen verursachen Fehler.
- Grenzwerte testen.

Vorteile:

- Ergänzt systematische Testentwurfungsverfahren.
- Durch systematische Testentwurfungsverfahren schwer aufdeckbare Fehler aufdecken.
- Kurzfristig einsetzbar, erfordert keine lange Vorplanung.
- Fördert Kreativität und Spontaneität.
- In Gruppen ergeben sich Synergien zwi. erfahrenen und unerfahrenen Testern.
- Anwendbar, wenn wenig Dokumentation oder Domänenwissen vorhanden (Kennenlernen des Produktes, »Exploration«).

Schwächen:

- Verfall in bestimmte Denkmuster verhindert Aufdecken wichtiger Probleme.
- Vom Wissen und Erfahrung der Tester abhängig.
- Keine Automatisierbarkeit.

- **Nicht eindeutig Black- oder White-Box Testentwurfsverfahren** zuzuordnen.
→ Weder Anforderungen noch Programmtext ausschließliche Grundlage für Überlegungen und Prüfungen.
- **Anwendungsbereich in höheren Teststufen.** → Auf niedrigeren stehen ausreichende Informationen für anderen Testentwurfsverfahren zur Verfügung, wie Programmtext oder detaillierte Spezifikation.
- Unabhängig vom Testobjekt **mit gutem Erfolg einsetzbar.**
- **Probleme:**
 - **Vor- und Nachbedingungen**, erwartete Ausgaben und erwartetes Verhalten des Testobjektes bei intuitiven Testfällen oft **schwierig festzulegen.**
 - Keine Messungen der Intensität oder Vollständigkeit der Testfallermittlung möglich.
 - **Kein Ausgangskriterium** (Endekriterium) wie bei systematischen Testentwurfsverfahren.
 - Existiert Fehlerkatalog, gewisse Vollständigkeit überprüfbar.

Fazit: Nicht als primäre, sondern zur Abrundung und Unterstützung der methodischen Testentwurfsverfahren einsetzen.

Wovon hängt **Erfolg der Vorgehensweise** und deren **Effektivität** ab?

Antwort:

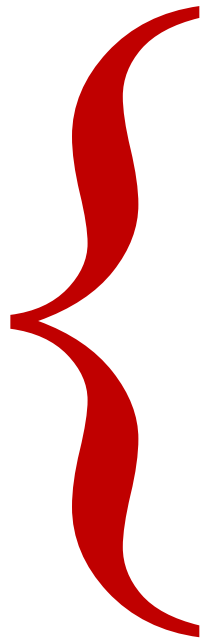
-

Wovon hängt **Erfolg der Vorgehensweise** und deren **Effektivität** ab?

Antwort:

- Sie hängen sehr stark von Fähigkeiten und Intuition der Tester und ihren bisherigen Erfahrungen mit ähnlichen Anwendungen und verwendeten Technologien ab.

2.2 White-Box- Test



Idee der White-Box Testentwurfsverfahren

Kontrollflussbasierter Test

Datenflussbasierter Test

Test der Bedingungen

Weitere White-Box Testentwurfsverfahren

Erfahrungsbasiertes Testen

Dynamischer Test: Testentwurfsvorgangsauswahl
und Zusammenfassung

Dynamischer Test – Auswahl von Testentwurfungsverfahren

Softwarekonstruktion
WS 2016/17

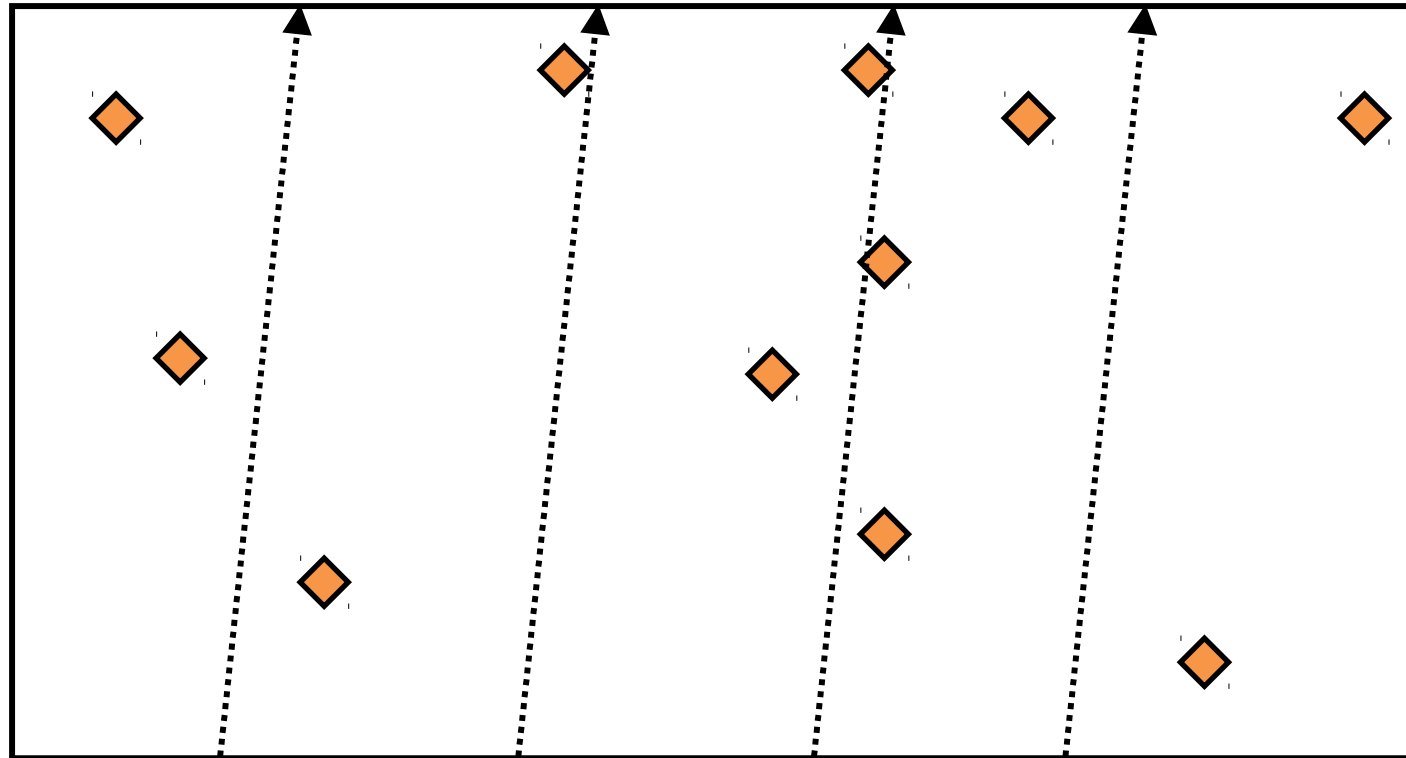


LEHRSTUHL 14
SOFTWARE ENGINEERING



Die Gefahr einer einseitigen Teststrategie (1)

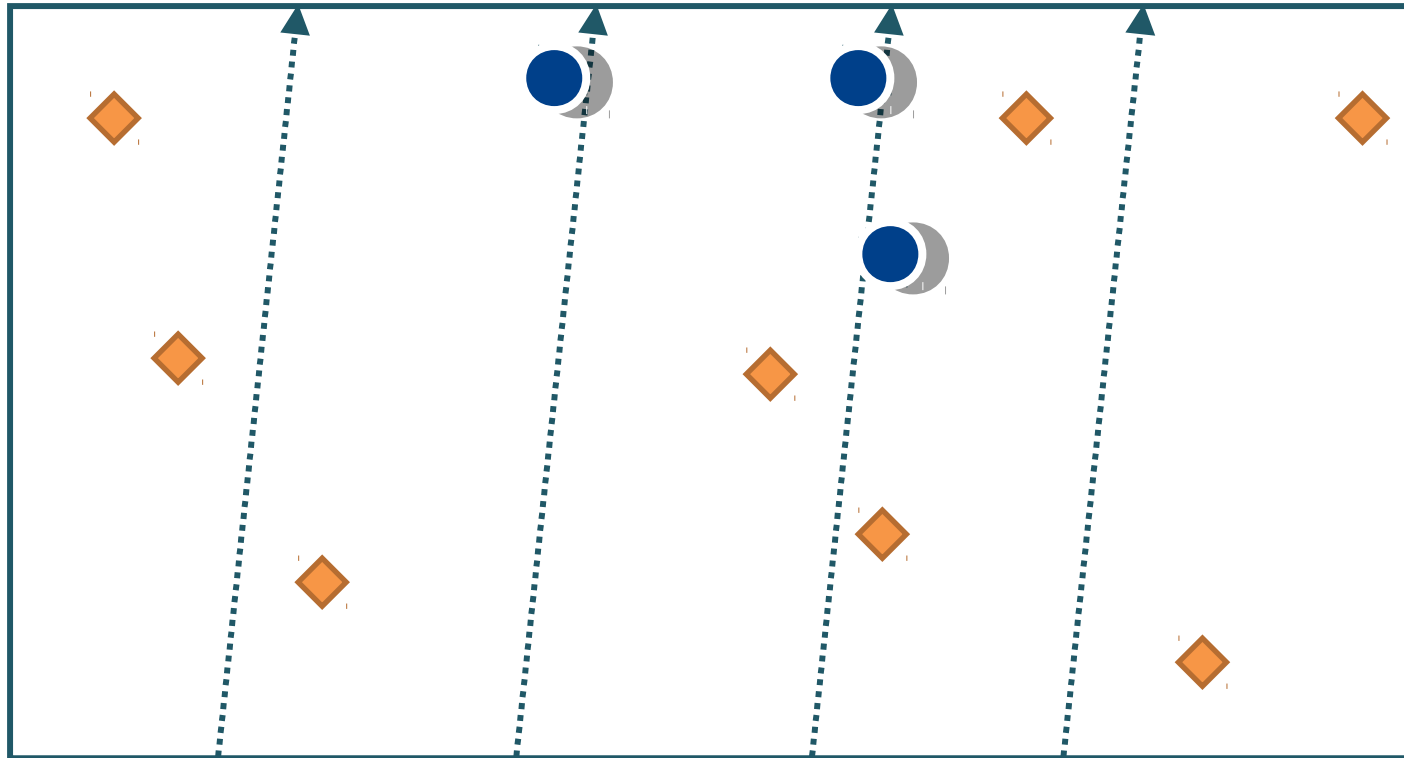
Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach



◆ Fehlerzustand («Mine«)

Die Gefahr einer einseitigen Teststrategie (2)

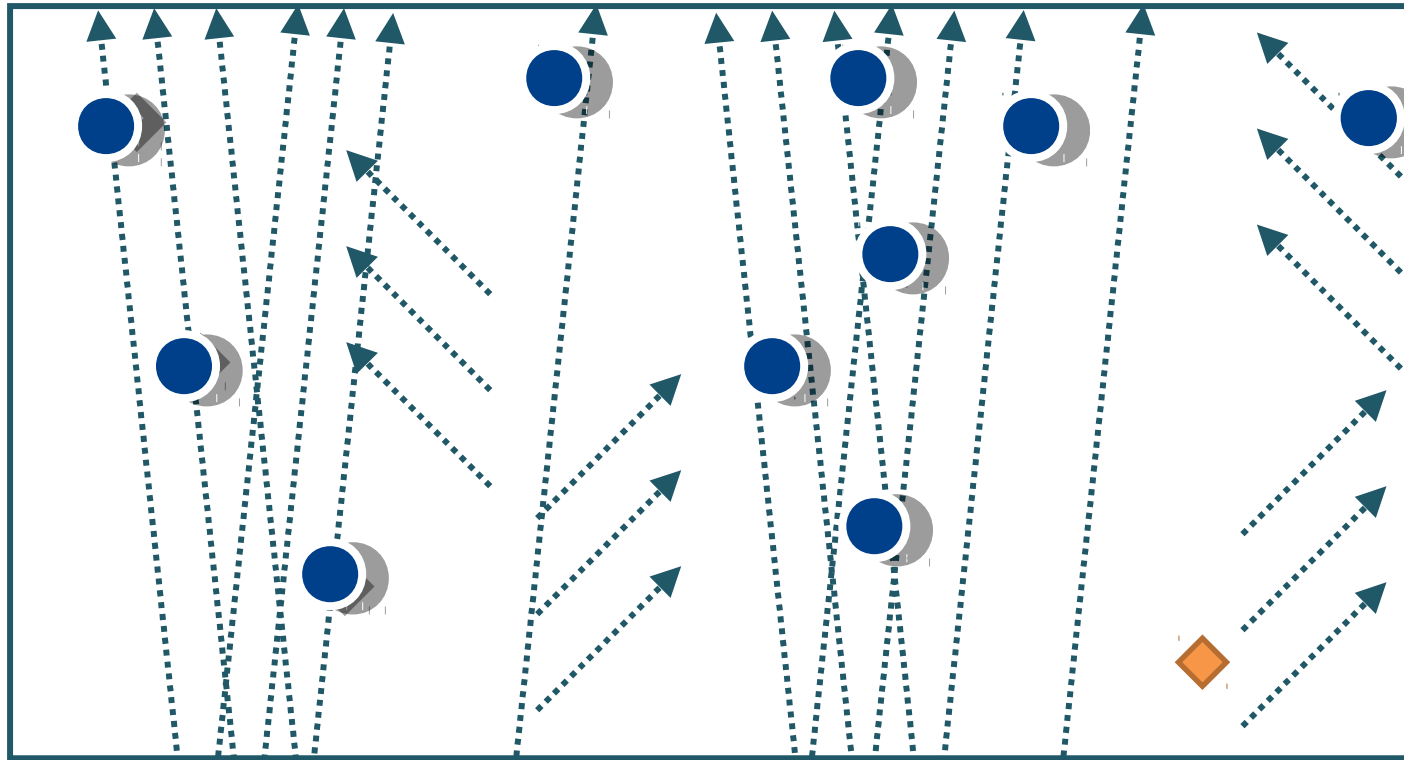
Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach



◆ Fehlerzustand (»Mine«) ● behoben (zur Wirkung gebracht)

Effektivität durch Variation der Testentwurfungsverfahren

Nach: Rapid Software Testing,
copyright © 1996-2002 James Bach



◆ Fehlerzustand (»Mine«) ● behoben (zur Wirkung gebracht)

Attribute guten Testfalls:

- **Mächtig** – Testfall deckt Fehlerwirkung auf, falls diese existiert.
- **Valide** – Testfall deckt tatsächliche Fehlerwirkung auf.
- **Relevant** – Testfall deckt Fehlerwirkung auf, die für Kunden relevant sind.
- **Glaubwürdig.**
- **Repräsentativ.**
- **Nicht-redundant** – Fehlerwirkungen werden nicht durch einen anderen Testfall abgedeckt.
- **Ausführbar** – Testfall kann, so wie er spezifiziert ist, ausgeführt werden.
- **Wartbar**, wiederholbar, einfach evaluierbar, adäquate Komplexität, geringe Kosten, ...

Ausprägung einzelner Attribute hängt von verwendeter Technik ab. Beispiele:

- **Anwendungsfallbasierter Test** – fokussiert auf Validität, Relevanz, Glaubwürdigkeit, Komplexität, weniger auf Wartbarkeit.
- **Äquivalenzklassenbildung** – nicht-redundant, wiederholbar, einfach wartbar. Keine Fokussierung auf Repräsentativität oder Glaubwürdigkeit.

»Parameter« eines Testentwurfsverfahrens

Jedes Testentwurfsverfahren **adressiert einen oder mehrere dieser »Parameter«**, wobei restliche »Parameter« offen gelassen werden.

»Parameter«:

- Testart, Teststufe, Tester, Abdeckung, potenzielle Fehler, Artefakte, Charakteristika des Produktes (Domäne, Technologie).
- Verfügbare Dokumentation und ihre Qualität.
- Regulatorische Anforderungen / Standards, Kunden- / Vertragsanforderungen.
- Projekt- und Produktrisiken.
- Testziele, angestrebter Automatisierungsgrad.
- **Projektfaktoren:**
Qualifikation der Tester, Erfahrung, Zeit und Geld, Softwareentwicklungsmodell

Was beeinflusst die Auswahl eines Testentwurfsverfahrens ?

- Kombination von Testentwurfsverfahren mit jeweils unterschiedlichem Fokus.
- **Einige Testentwurfsverfahren:** Im bestimmten Kontext für bestimmte Situationen und Teststufen besser geeignet.
- **Andere Testentwurfsverfahren:** In allen Teststufen gleichermaßen einsetzbar.
- **Fehlerannahme:** Annahme über mögliche Fehlerwirkungen. → Durch Anwendung des entsprechenden Testentwurfsverfahrens aufdeckbar:
 - **Folge:** Unterschiedliche Testentwurfsverfahren finden unterschiedliche Arten von Fehlerwirkungen.
 - **Empfehlung:** Verwendung unterschiedlicher Testentwurfsverfahren !

Bisher nicht ausgeführte Teile des Testobjektes: Gezielt einem der White-Box Testentwurfsverfahren unterziehen.

- Je nach Kritikalität und Beschaffenheit des Testobjektes entsprechendes White-Box Testentwurfsverfahren wählen.
- **Minimales Kriterium:** Entscheidungsüberdeckung.
- Bei White-Box Testentwurfsverfahren:
 - Struktur des Testobjektes: Grundlage bei Auswahl der Testentwurfsverfahren.
 - Komplexe Bedingungen im Testobjekt vorhanden. → Minimal bestimmende Mehrfachbedingungsüberdeckung: Adäquates Testentwurfsverfahren.
→ Fehlerhafte Bedingungen erkennen.
- Achtung bei Überdeckungsmessungen: Schleifen mehr als einmal wiederholbar.
- Bei kritischen Systemteilen: Prüfung der Schleifen durch entsprechende Methoden erfolgen.
- Pfadüberdeckung: Theoretisches Maß und keine Bedeutung für Praxis auf Grund des riesigen Aufwands.

- **White-Box** Testentwurfungsverfahren auf **unteren Teststufen** einsetzen.
- Auf **oberen Teststufen: Black-Box** Testentwurfungsverfahren sind adäquate Methoden.
- Auf **erfahrungsbasierte Ermittlung** der Testfälle als Ergänzung nicht verzichten.
- Erfahrungen der Tester nutzen, um **weitere Fehler aufzudecken**.
- Testen umfasst **Kombination von unterschiedlichen Testentwurfungsverfahren**.
 - Kein Testentwurfungsverfahren vorhanden, dass alle Aspekte, die beim Testen zu berücksichtigen sind, gleich gut abdeckt.
 - **Auswahl der Testentwurfungsverfahren** und Intensität der Durchführung: Durch Kritikalität und erwartender Risiko **im Fehlerfall festlegen**.